

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN MÔN HỌC

Môn học: CE118.Q11 – Thiết kế luận lý số

Giảng viên hướng dẫn: PGS.TS. Lâm Đức Khải

Sinh viên thực hiện:

- | | |
|---------------------------|----------|
| 1. Nguyễn Đăng Phương Duy | 23520372 |
| 2. Vũ Đại Dương | 23520359 |

TP. HỒ CHÍ MINH, 2025

Mục Lục

A.	Thiết kế BCD Counter có chức năng Paralel Load.....	5
1.	Thiết kế thành phần.....	5
2.	Thiết kế BCD Counter 4 bit.....	8
B.	Thiết kế mạch dịch đa năng 4 bit	10
1.	Ý tưởng thiết kế	10
2.	Thiết kế mạch dịch đa năng	10
C.	Register-file 2 Write Port & 1 Read Port	13
1.	Cơ sở lý thuyết.....	13
2.	Thiết kế mạch.....	14
3.	Waveform.....	16
D.	Thiết kế ram 32x32	18
1.	Memory_cell.....	18
2.	4x4ram	19
3.	4x32ram	20
4.	32x32ram	21
5.	Waveform.....	22
E.	Thiết kế Push-down Stack (LIFO)	23
1.	Cơ sở lý thuyết.....	23
2.	Thiết kế các khối cần thiết cho mạch	23
3.	Thiết kế mạch Push-down Stack.....	26
4.	Waveform.....	27
F.	Thiết kế queue (FIFO).....	28
1.	Cơ sở lý thuyết.....	28
2.	Thiết kế thành phần.....	28
3.	Thiết kế tổng quan queue.....	30
4.	Waveform.....	30
G.	Thiết kế bộ đếm số lượng bit 1	31
1.	Cơ sở lý thuyết.....	31
2.	Thiết kế datapath.....	31

3.	Thiết kế controller.....	33
4.	Mạch hoàn chỉnh.....	36
5.	Waveform.....	37
H.	Thiết kế bộ tính tổng từ 1 đến 8	38
1.	Ý tưởng thiết kế	38
2.	Thiết kế Datapath.....	40
3.	Thiết kế Controller.....	47
4.	Thiết kế mạch tính tổng từ 1 đến 8 (Sum8).....	49
5.	Waveform.....	50
I.	Thiết kế khối tính căn bậc 2 của tổng bình phương 2 số 8 bit có dấu ..	51
1.	Cơ sở lý thuyết.....	51
2.	Thiết kế mạch Datapath.....	52
3.	Thiết kế mạch Controller.....	60
4.	Thiết kế mạch tổng	64
5.	Waveform.....	65
J.	Thiết kế sử dụng Function-unit Sharing	66
1.	Cơ sở lý thuyết.....	66
2.	Thiết kế Datapath.....	66
3.	Thiết kế Controller.....	71
4.	Thiết kế mạch theo Function-Unit Sharring.....	76
5.	Waveform.....	76
K.	Thiết kế sử dụng Bus-sharing	77
1.	Cơ sở lý thuyết.....	77
2.	Thiết kế datapath.....	77
3.	Thiết kế controller.....	81
4.	Thiết kế sử dụng Bus Sharing.....	85
5.	Waveform.....	85
L.	Thiết kế sử dụng Register Merging.....	87
1.	Cơ sở lý thuyết.....	87
2.	Thiết kế Datapath.....	87
3.	Thiết kế Controller.....	89

4.	Thiết kế mạch sử dụng Register Merging.....	92
5.	Waveform.....	92
M.	Thiết kế sử dụng pipelined functional unit	93
1.	Cơ sở lý thuyết.....	93
2.	Thiết kế datapath.....	93
3.	Thiết kế controller.....	96
4.	Mạch hoàn chỉnh.....	100
5.	Waveform.....	100
N.	Thiết kế datapath pipelined	102
1.	Cơ sở lý thuyết.....	102
2.	Thiết kế datapath.....	102
3.	Thiết kế controller.....	107
4.	Mạch hoàn chỉnh.....	111
5.	Waveform.....	111
O.	Datapath pipeline với pipelined units	113
1.	Cơ sở lý thuyết.....	113
2.	Thiết kế datapath.....	113
3.	Controller	116
4.	Mạch hoàn chỉnh.....	122
5.	Waveform.....	122

Bảng phân công bài tập

STT	Tên bài tập	Thành viên thực hiện
1	Thiết kế BCD Counter có chức năng Paralel Load	Đại Dương
2	Thiết kế mạch dịch đa năng 4 bit	Đại Dương
3	Register-file 2 Write Port & 1 Read Port	Đại Dương
4	Thiết kế Ram 32x32	Phuong Duy
5	Thiết kế Push-down Stack (LIFO)	Phuong Duy
6	Thiết kế queue (LIFO)	Phuong Duy
7	Thiết kế bộ đếm số lượng bit 1	Phuong Duy
8	Thiết kế bộ tính tổng từ 1 đến 8	Đại Dương
9	Thiết kế khối tính căn bậc 2 của tổng bình phương 2 số 8 bit có dấu	Đại Dương
10	Thiết kế sử dụng Function-unit Sharing	Đại Dương
11	Thiết kế sử dụng Bus-sharing	Đại Dương
12	Thiết kế sử dụng Register Merging	Đại Dương
13	Thiết kế sử dụng pipelined functional unit	Phuong Duy
14	Datapath pipelined	Phuong Duy
15	Datapath pipeline với pipelined units	Phuong Duy

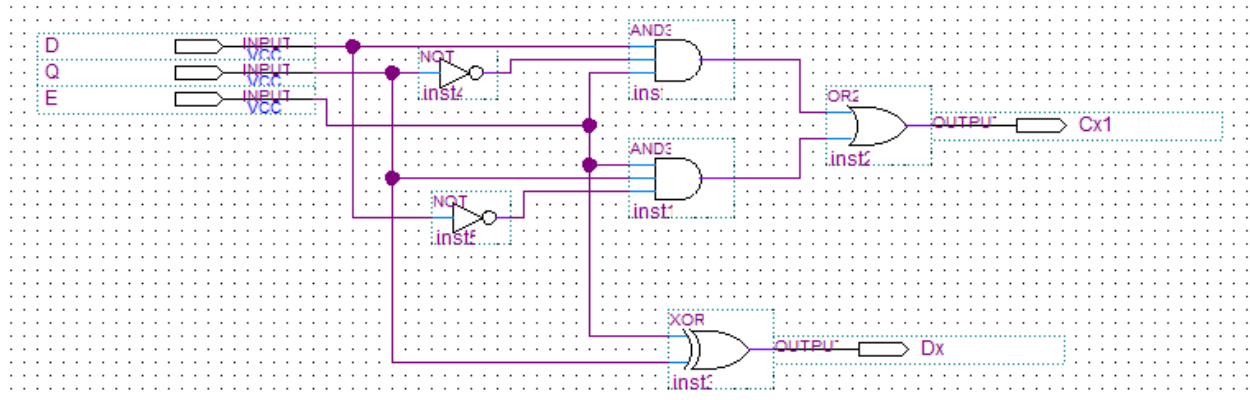
Bảng đóng góp

Họ và tên	Phần trăm đóng góp
Nguyễn Đặng Phương Duy	50%
Vũ Đại Dương	50%

A. Thiết kế BCD Counter có chức năng Paralel Load

1. Thiết kế thành phần

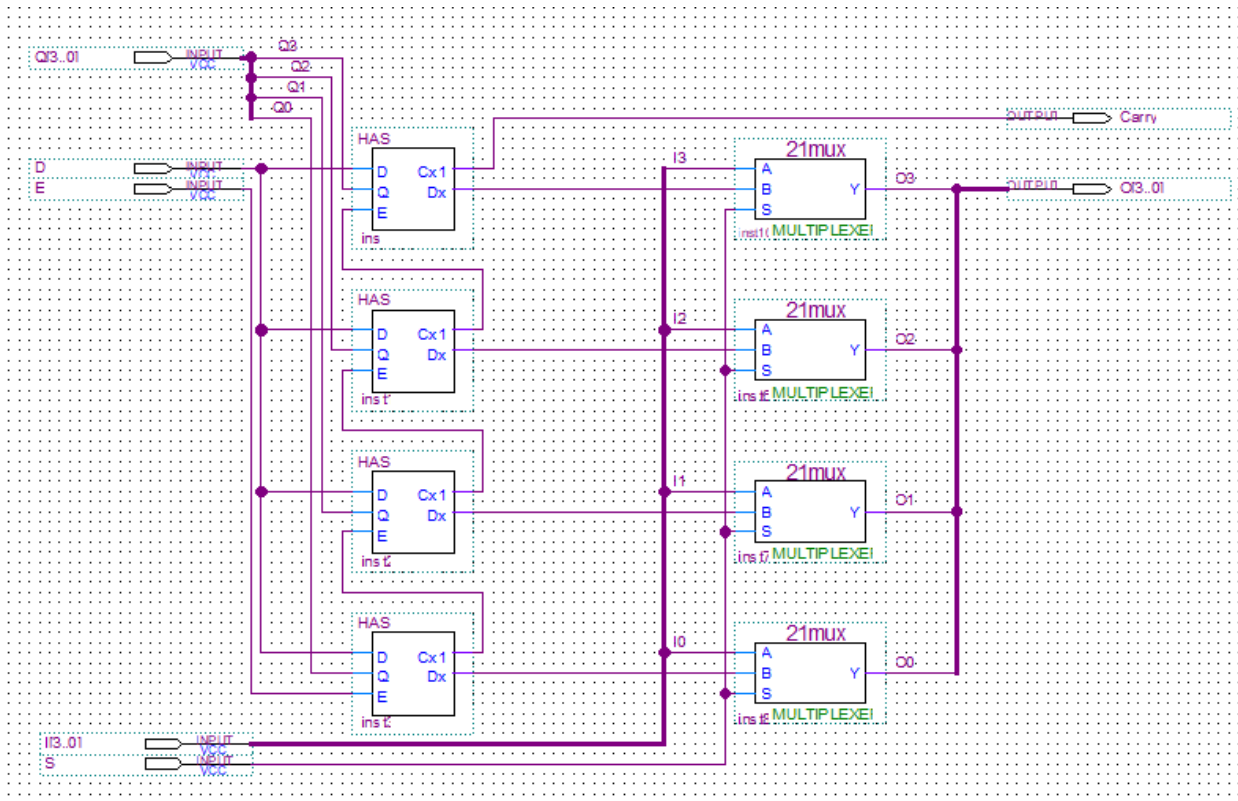
1.1. Thiết kế bộ HAS



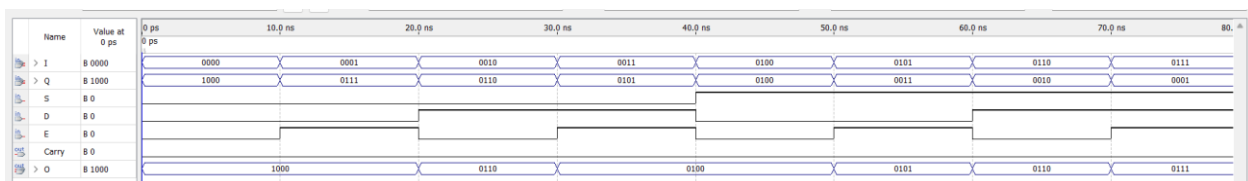
D	Q	E	Cx1	Dx
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	0

1.2. Thiết kế Comb1

- Chức năng: thực hiện Paralel Load hoặc tăng/giảm giá trị Q với
 - S: chọn chức năng
 - E: cho phép hoạt động tăng/giảm
 - D: = 0 → tăng; = 1 → giảm



- Waveform:



- 0 – 40ns: $S = 0 \rightarrow O = Q \pm 1$
 - 0 – 10ns: $E = 0 \rightarrow O = Q$
 - 10 – 20ns: $E = 1, D = 0 \rightarrow O = Q + 1$
 - 20 – 30ns: $E = 0 \rightarrow O = Q$
 - 30 – 40ns: $E = 1, D = 1 \rightarrow O = Q - 1$

- 40 – 80ns: $S = 1 \rightarrow O = I$

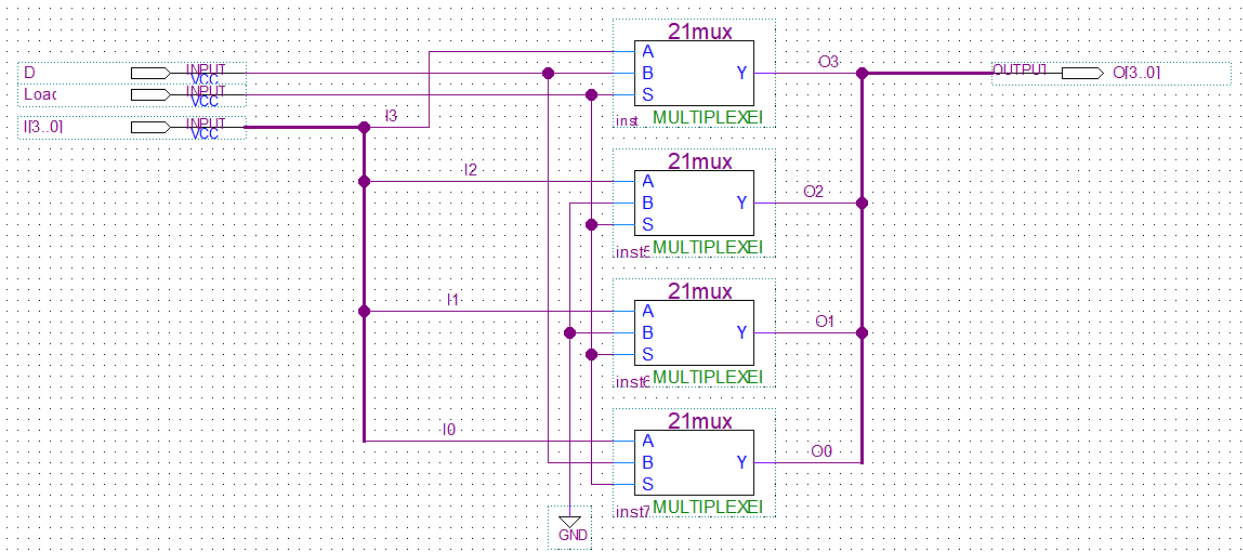
⇒ Mạch thực hiện đúng chức năng

1.3. Thiết kế mạch IP

- Chức năng: Chọn output $O = I$ hoặc $O = 0000$ hay 1001 thông qua tín hiệu Load

- Load = 1 $\rightarrow O = I$
- Load = 0:
 - $D = 0 \rightarrow O = 0000$
 - $D = 1 \rightarrow O = 1001$

⇒ Parallel load hoặc tiền xử lý cho chức năng nạp 0 khi thực hiện tăng, nạp 9 khi thực hiện giảm

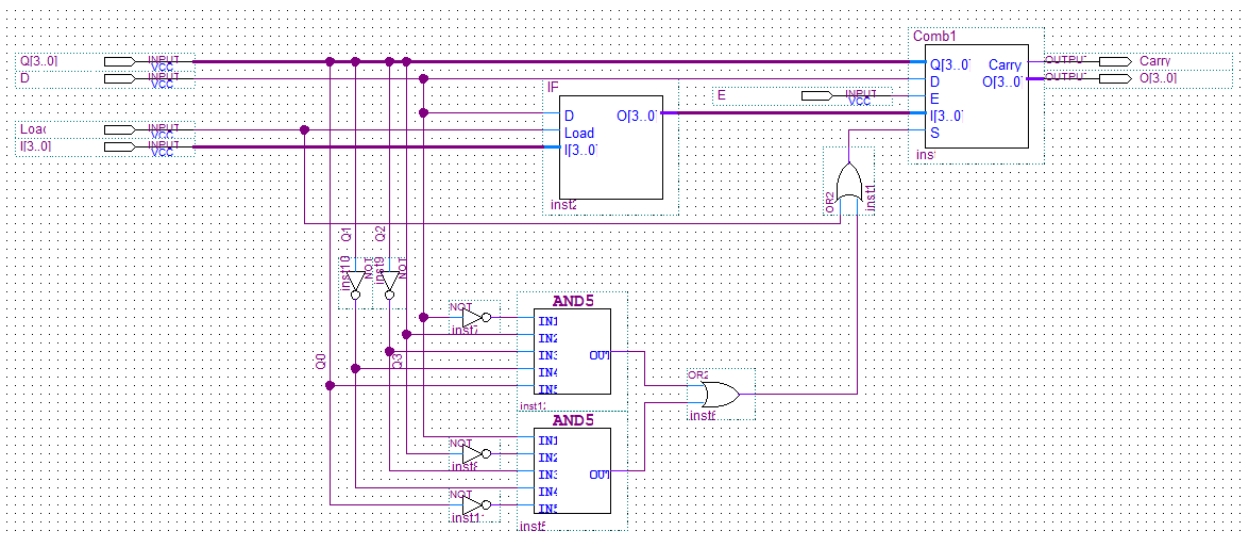


1.4. Thiết kế mạch Comb

- Chức năng:

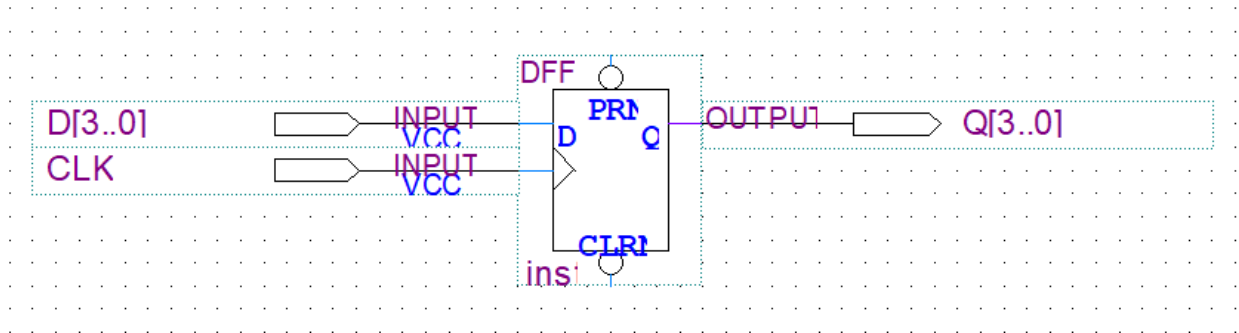
- Thực hiện nạp hoặc tăng/giảm giá trị hiện tại (Q) thông qua tín hiệu Load
 - Load = 1 $\rightarrow$ O = I
 - Load = 0 $\rightarrow$ O = Q $\pm$ (0 | 1)
- E: cho phép thực hiện tăng/giảm hoặc giữ nguyên giá trị
 - E = 0 $\rightarrow$ O = Q
 - E = 1 $\rightarrow$ O = Q $\pm$ 1
- D: tăng/giảm
 - D = 0 $\rightarrow$ O = Q + 1
 - D = 1 $\rightarrow$ O = Q - 1
- Tín hiệu Select tại Comb:

$$S = D' \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0 + D \cdot Q_3' \cdot Q_2 \cdot Q_1 \cdot Q_0' + \text{Load}$$



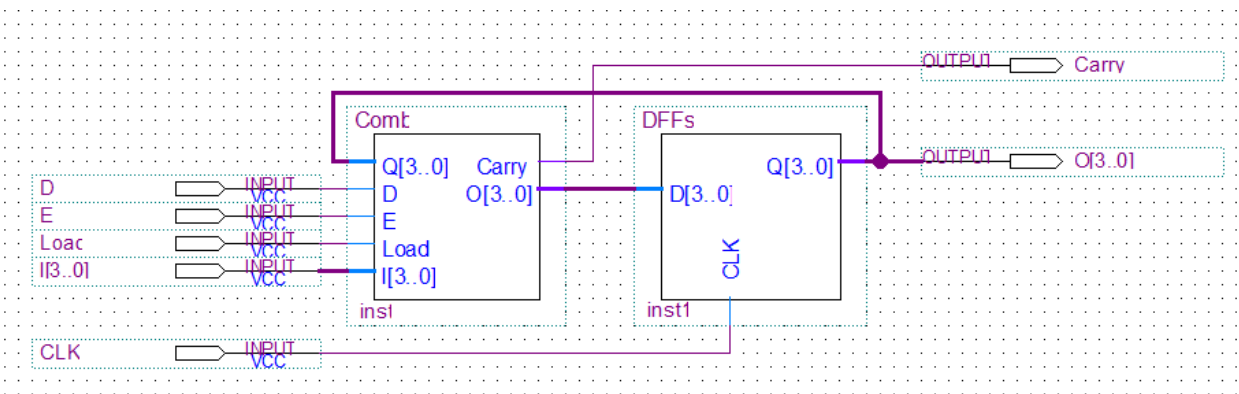
1.5. Thiết kế mạch DFFs

- Chức năng: đưa các tín hiệu về đồng bộ để dễ quan sát

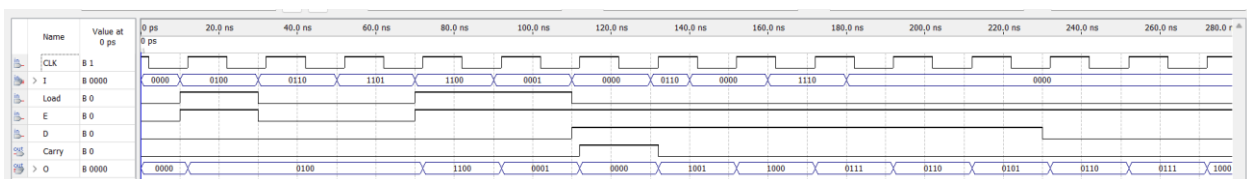


2. Thiết kế BCD Counter 4 bit

- Chức năng:
 - o Thực hiện nạp hoặc tăng/giảm giá trị hiện tại (Q) thông qua tín hiệu Load
 - Load = 1 $\rightarrow O = I$
 - Load = 0 $\rightarrow O = Q \pm (0 | 1)$
 - o E: cho phép thực hiện tăng/giảm hoặc giữ nguyên giá trị
 - E = 0 $\rightarrow O = Q$
 - E = 1 $\rightarrow O = Q \pm 1$
 - o D: tăng/giảm
 - D = 0 $\rightarrow O = Q + 1$
 - D = 1 $\rightarrow O = Q - 1$
 - o Giá trị được cập nhật mỗi khi CLK tích cực cạnh lên (đồng bộ)



- Waveform:



- o 0 – 12ns: Load = 0, E = 0 $\rightarrow O$ giữ nguyên = 0000 (giá trị khởi tạo)

- 12 – 32ns: Load = 1 $\rightarrow$ O = I = 0100
- 32 – 72ns: Load = 0, E = 0 $\rightarrow$ O giữ nguyên = 0100
- 72 – 112ns: Load = 1 $\rightarrow$ O = I
- 112 – 232ns: Load = 0, E = 1, D = 1 $\rightarrow$ O = Q – 1
 - 112 – 132ns: Load = 0, E = 1, D = 1 & O = 0000 $\rightarrow$ Carry=1
- 232 – 280ns: Load = 0, E = 1, D = 0 $\rightarrow$ O = Q + 1

B. Thiết kế mạch dịch đa năng 4 bit

1. Ý tưởng thiết kế

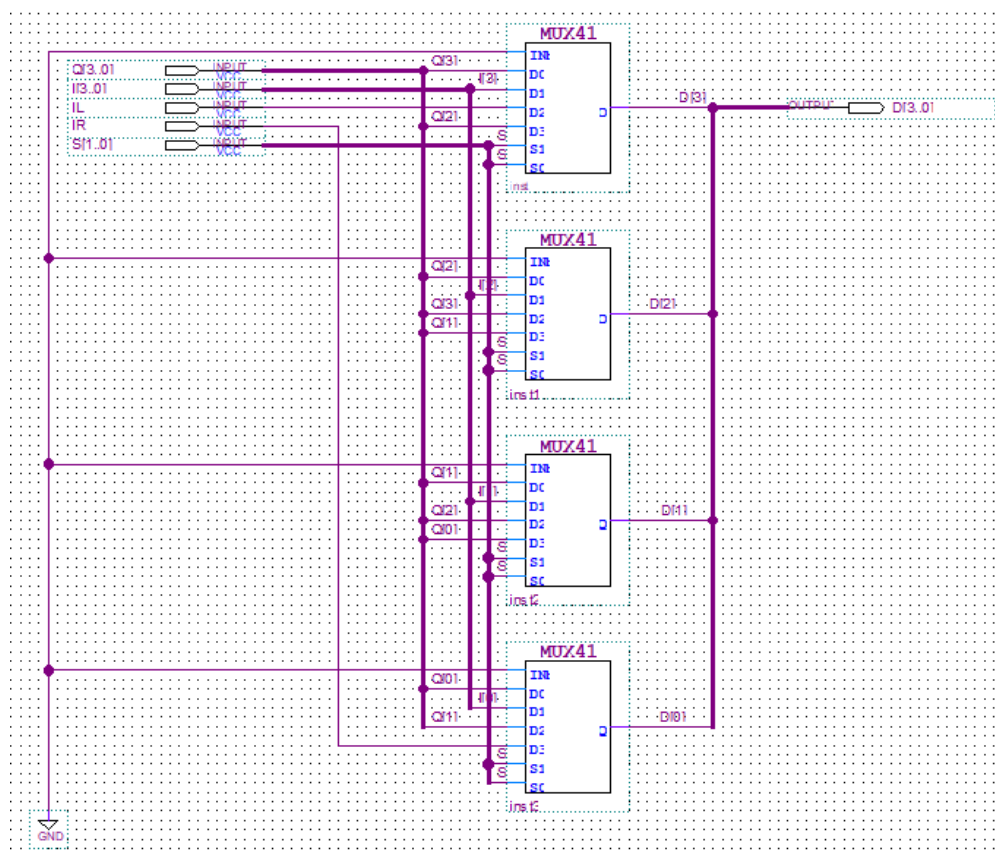
- Mạch dịch đa năng có 4 chức năng: giữ nguyên, nạp Input, dịch trái & dịch phải
- Sử dụng mux4:1 được điều khiển bởi tín hiệu S[1..0] để chọn Output đầu ra (Res[3..0]) với

S[1..0]	Res[3..0]	Operation
00	Q[3..0]	No change
01	I[3..0]	Load input
10	IL_Q[3..1]	Shift right
11	Q[2..0]_IR	Shift left

- Sử dụng thêm DFFs để tín hiệu đồng bộ

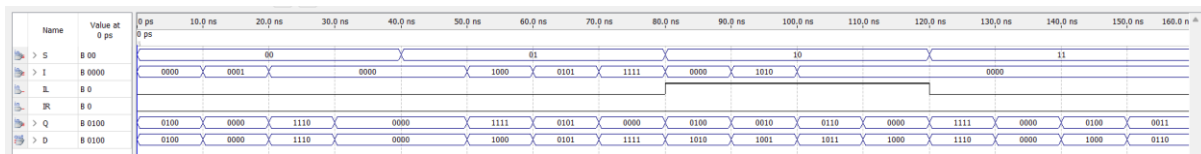
2. Thiết kế mạch dịch đa năng

2.1. Thiết kế khối Comb



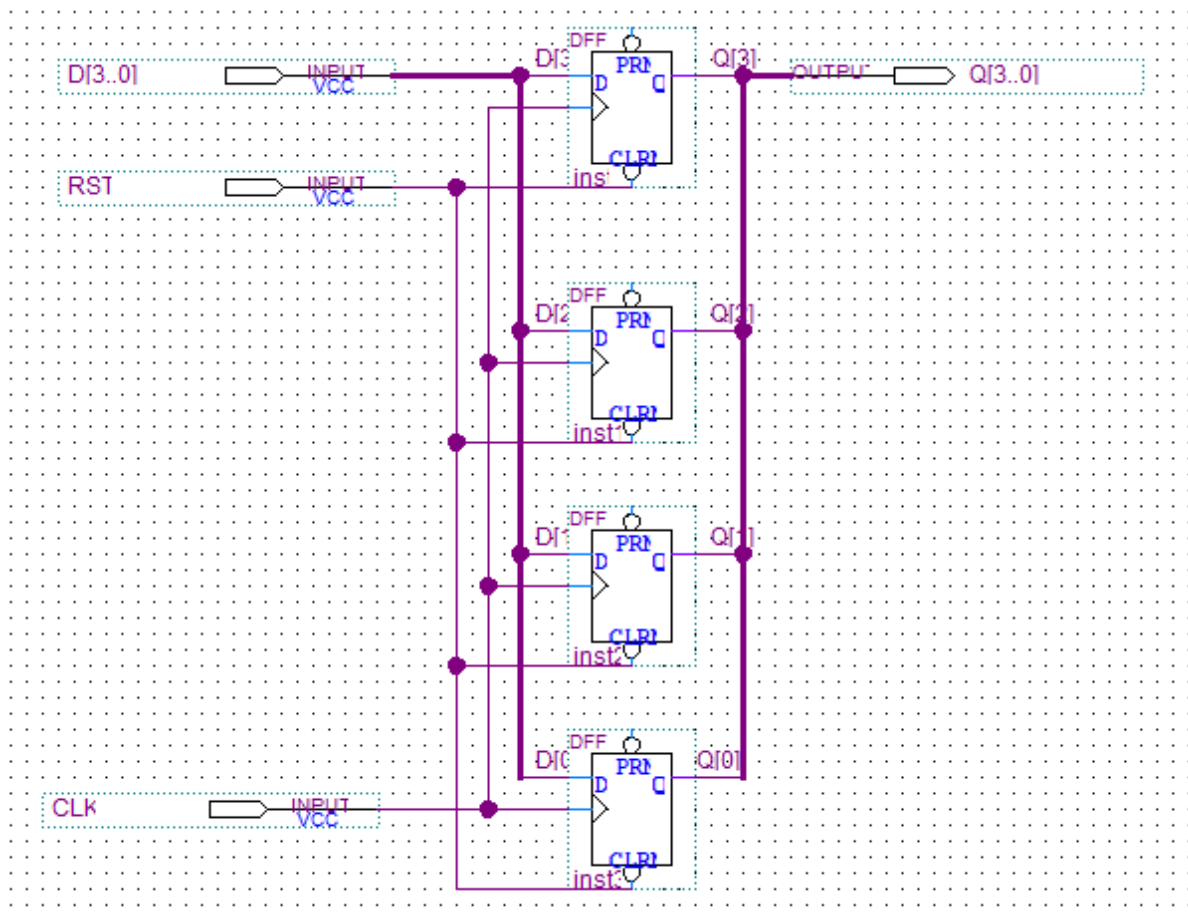
- Comb là khối xử lý input đầu vào theo từng chức năng để cho ra Input của DFFs

- Waveform:



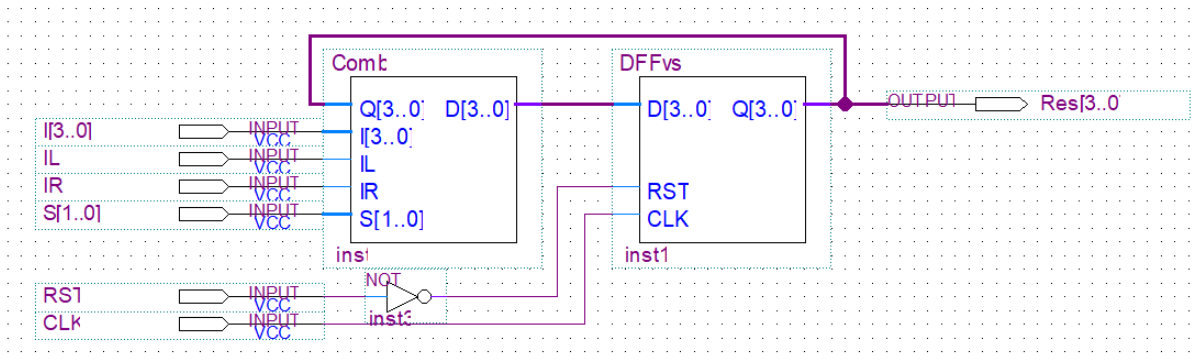
- 0 – 40ns: $S[1..0] = 00 \rightarrow$ Giữ nguyên: $D = Q$
- 40 – 80ns: $S[1..0] = 01 \rightarrow$ Nạp Input: $D = I$
- 80 – 120ns: $S[1..0] = 10 \rightarrow$ Dịch phải: $D = ILQ[3..1]$
- 120 – 160ns: $S[1..0] = 11 \rightarrow$ Dịch trái: $D = Q[2..0]IR$

2.2. Thiết kế khối DFFs

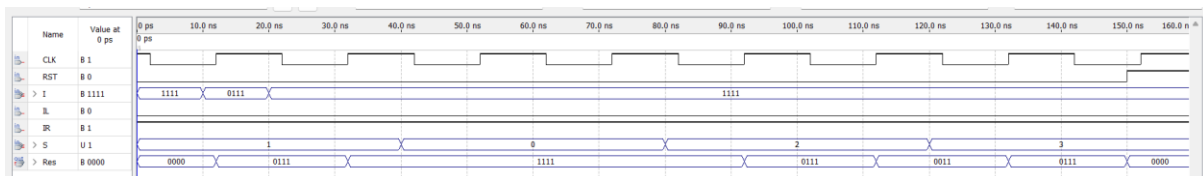


- Chức năng: đưa Output về tín hiệu đồng bộ để dễ quan sát

2.3. Thiết kế mạch dịch đa năng



- Mạch sẽ thực hiện các chức năng đã nêu ra trong ý tưởng thiết kế
- Tín hiệu RST sẽ được tích cực mức cao để đưa Q[3..0] về 0000
- Output cập nhật giá trị tại mỗi cạnh tích cực mức lên của CLK
- Waveform:



- 12 – 52ns: S[1..0] = 01 → Nạp Input: Res = I
- 52 – 92ns: S[1..0] = 00 → Giữ nguyên: Res = Q
- 92 – 132ns: S[1..0] = 10 → Dịch phải: Res = ILQ[3..1]
- 132 – 150ns: S[1..0] = 11 → Dịch trái: Res = Q[2..0]IR
- 150 – 160ns: RST = 1 → Res = 0000

C. Register-file 2 Write Port & 1 Read Port

1. Cơ sở lý thuyết

1.1. 2-to-4 Write Decoder Block

WE	WA0	WA1	W3	W2	W1	W0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

- Công thức Output:

- $W3 = WE \cdot WA1 \cdot WA0$
- $W2 = WE \cdot WA1 \cdot WA0'$
- $W1 = WE \cdot WA1' \cdot WA0$
- $W0 = WE \cdot WA1' \cdot WA0'$

1.2. Register-file Cell (RFC)

RS	WS1	WS0	O
0	X	X	Z
1	0	0	Q
1	0	1	I0
1	1	X	I1

*Z: High impedance

- Ưu tiên W1

- Công thức Output:

- $O_{data} = (WS1 \cdot I1) + (WS1' \cdot WS0 \cdot I0) + (WS1' \cdot WS0' \cdot Q)$
- $O = \begin{cases} O_{data} & \text{nếu } RS = 1 \\ Z & \text{nếu } RS = 0 \end{cases}$

1.3. 2-to-4 Read Decoder Block

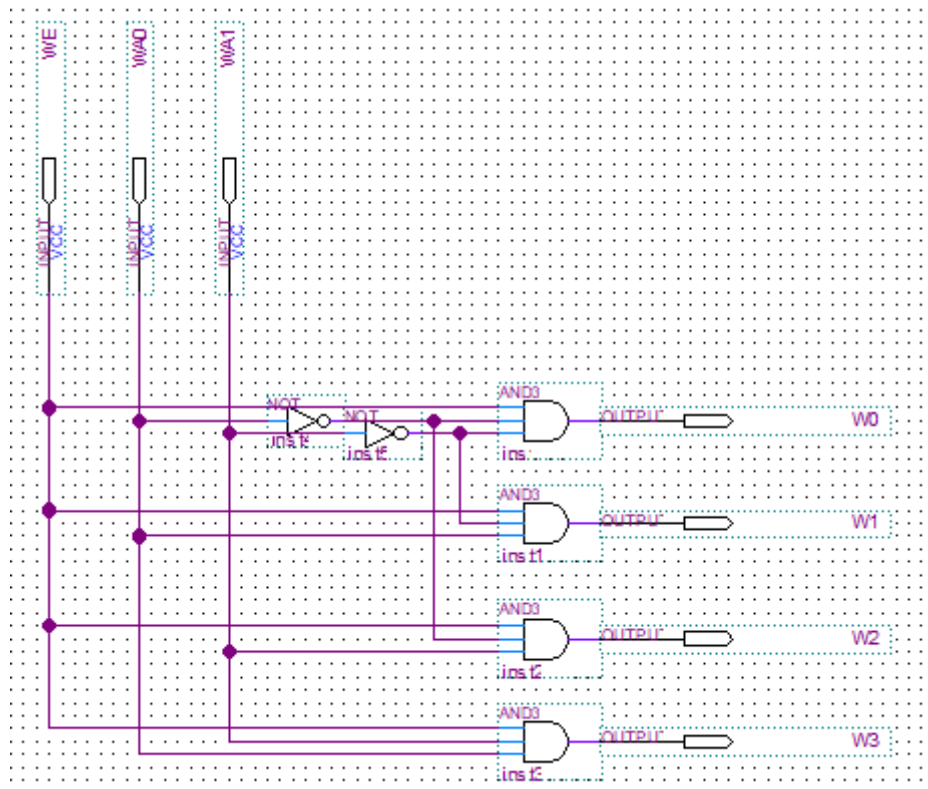
RE	RA1	RA0	R3	R2	R1	R0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

- Công thức Output:

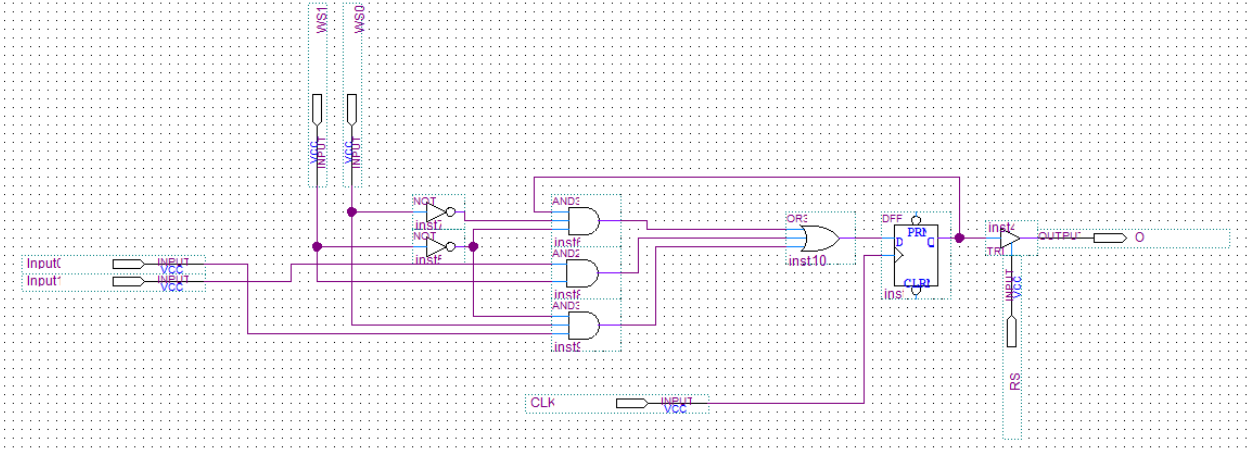
- $R3 = RE.RA1.RA0$
- $R2 = RE.RA1.RA0'$
- $R1 = RE.RA1'.RA0$
- $R0 = RE.RA1'.RA0'$

2. Thiết kế mạch

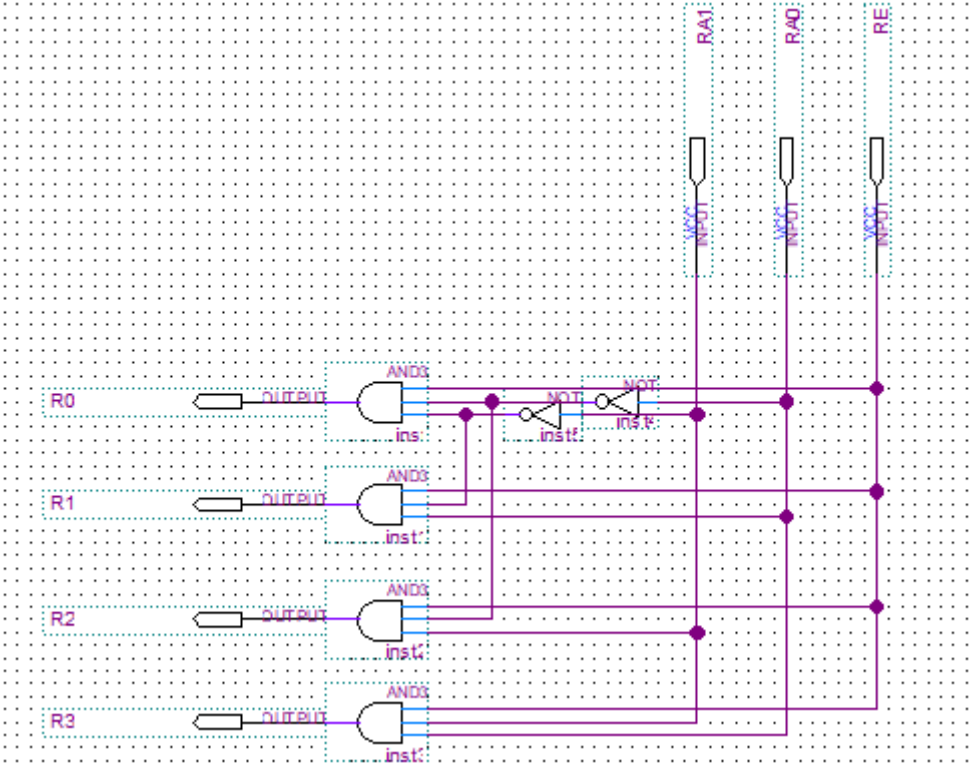
2.1. 2-to-4 Write Decoder Block



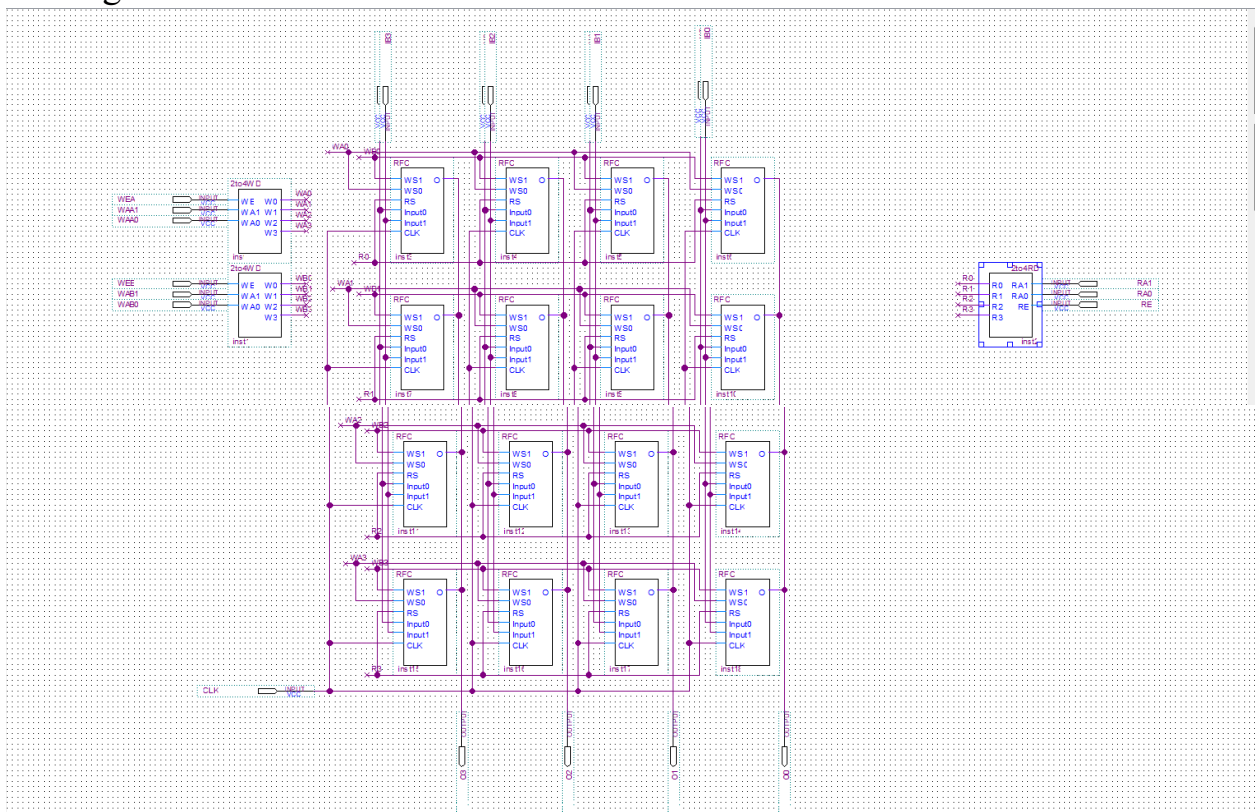
2.2. Register-File Cell (RFC)



2.3. 2-to-4 Read Decoder Block

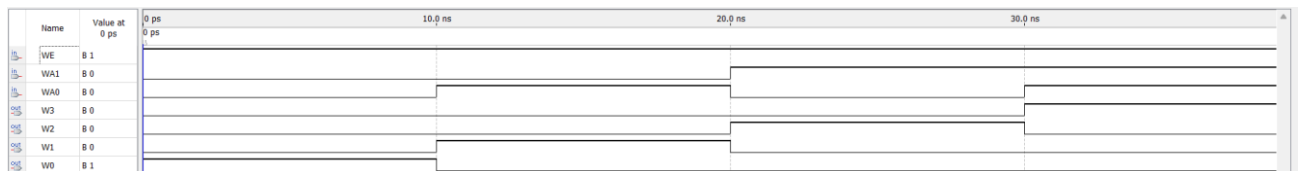


2.4. Register-File 2 Write 1 Read

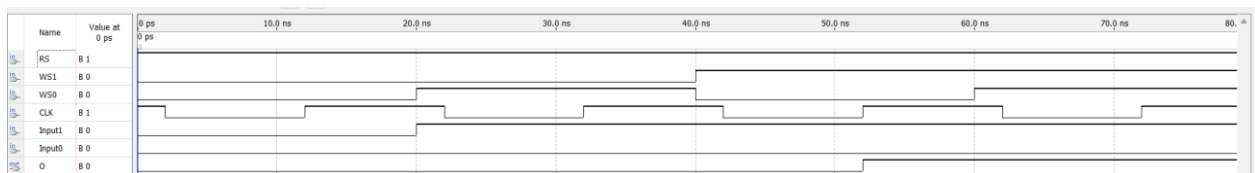


3. Waveform

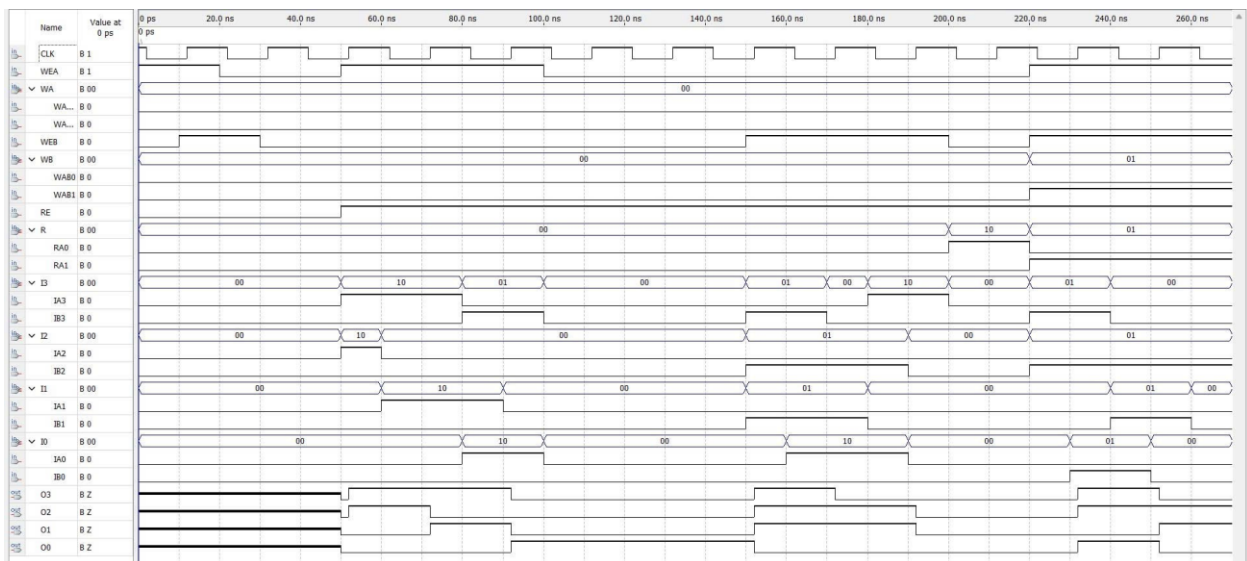
3.1. WD



3.2. RFC



3.3. Mạch chính RF 1 Read 2 Write



- 0 – 50ns: RE = 0 => O = Z
- 50 – 100ns: WEA = 1 & WEB = 0 => RFC chọn giá trị từ WA
 - o WA0 = WA1 = 0 => Input IAx nạp vào RFC hàng 0
 - o RE = 1; RA0 = RA1 = 0 => Output đọc giá trị từ RFC hàng 0
- 100 – 150ns: WEA = WEB = 0 => Giữ nguyên giá trị
- 150 – 200ns: WEA = 0 & WEB = 1 => RFC chọn giá trị từ WB
 - o WB0 = WB1 = 0 => Input IBx nạp vào RFC hàng 0
 - o RE = 1; RA0 = RA1 = 0 => Output đọc giá trị từ RFC hàng 0
- 200 – 220ns: WEA = WEB = 0 => Giữ nguyên giá trị
- 220 – 270ns: WEA=WEB=1 => Ưu tiên ghi giá trị từ WB từ RFC hàng 3
 - o WB0 = 0; WB1 = 1 => Input IBx nạp vào RFC hàng 1
 - o RE = 1; RA0 = 0; RA1 = 1 => Output đọc giá trị từ RFC hàng 1

D. Thiết kế ram 32x32

1. Memory_cell

Present state		Next state
RS	WE	Q
0	X	Z
1	0	Q
1	1	Input

Trong đó:

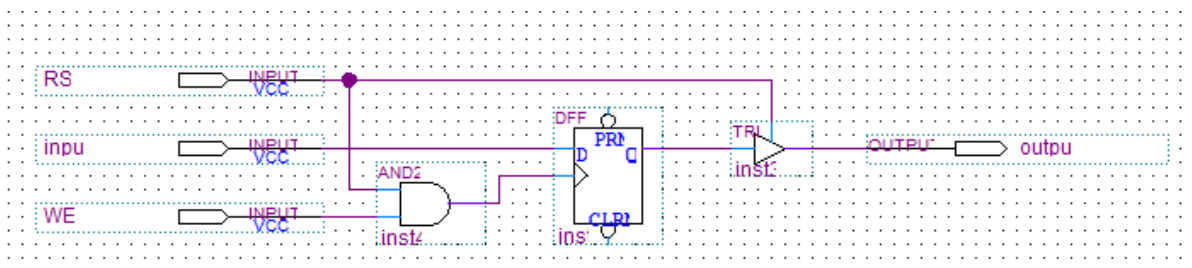
- RS (row select): 0 -> cell không được chọn;
1 -> cell được chọn
- WE (Write enable): 0 -> Giữ nguyên trạng thái trước đó
1 -> Sử dụng input làm đầu ra

Input Dff:

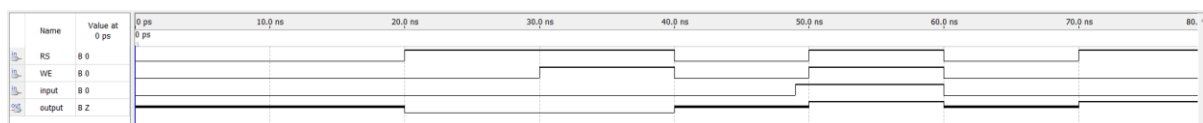
$D = \text{Input}$

$C = RS * WE$

Thiết kế mạch:



Chạy mô phỏng waveform:



- 0ns – 20ns: RS = 0 => cell không được kích hoạt nên có trạng thái Z
- 20ns – 30ns: RS = 1, WE = 0 => Giữ nguyên trạng thái trước đó, ở đây khởi tạo là 0
- 30ns – 40ns: RS = 1, WE = 1, input = 0 => output = 0 vì cell được kích hoạt khi nhận input là 0
- 40ns – 50ns: RS = 0 => cell không được kích hoạt nên có trạng thái Z

- 50ns – 60ns: RS = 1, WE = 1, input = 1 => output = 1 vì cell được kích cạnh lên khi nhận input là 1

- 70ns – 80ns: RS = 1, WE = 0 => Không viết nên cell vẫn lưu trữ output trước đó là 1

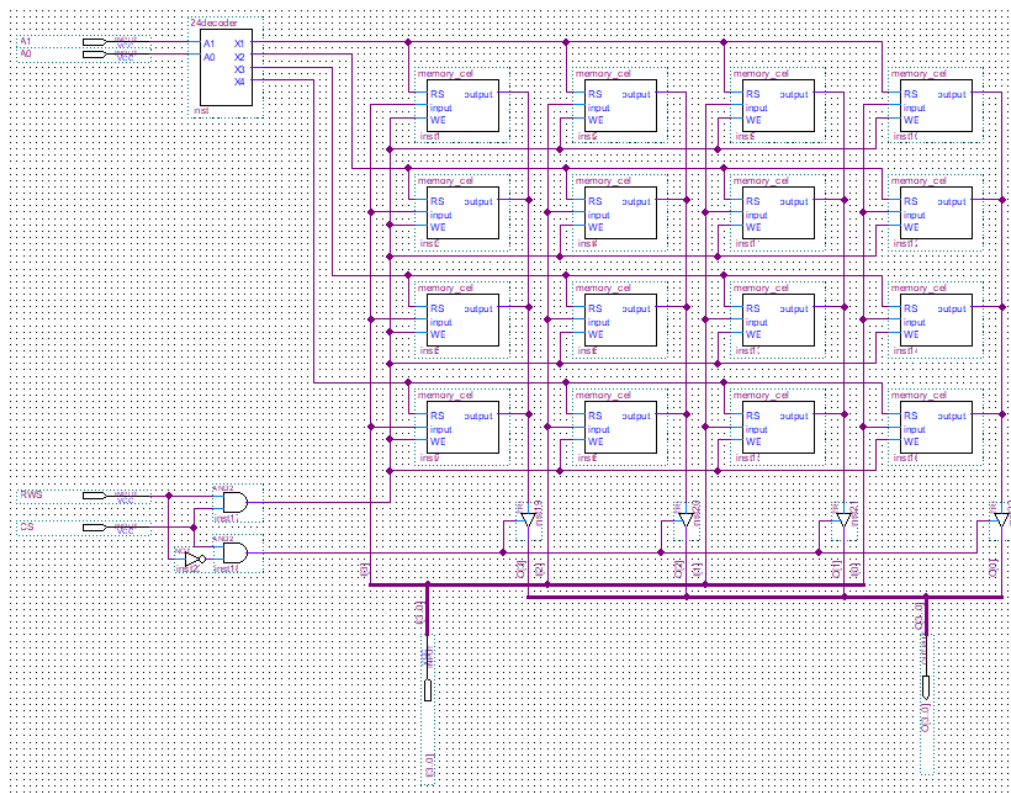
2. 4x4ram

Ý tưởng thiết kế: thiết kế 4x4 ram bằng cách ghép 16 memory cell lại và mã hóa chọn cell theo dòng bằng 2 to 4 decoder

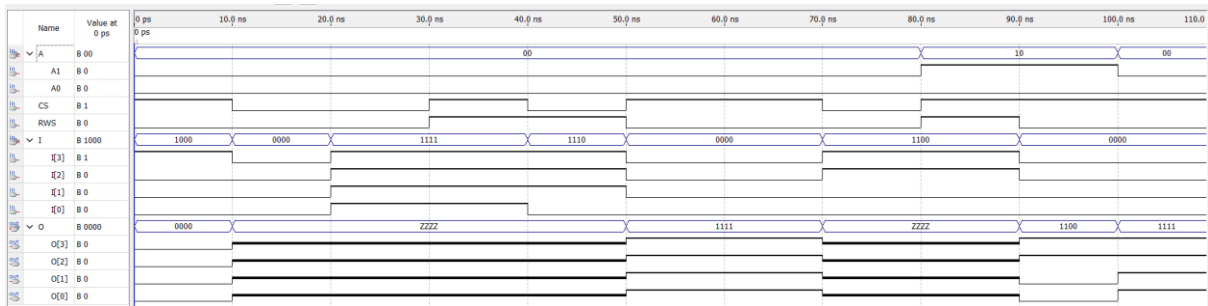
Mô tả chức năng:

- Input:
 - A1A0: 2 bit mã hóa để chọn cell theo dòng
 - RWS: bit lựa chọn chế độ đọc hoặc ghi cho các cell nhớ
 - RWS = 1: Chế độ ghi
 - RWS = 0: Chế độ đọc
 - CS: bit kích hoạt cell nhớ
 - I[3..0]: 4 bit input cần lưu trữ
- Output
 - O[3..0]: 4 bit output của các cell nhớ

Thiết kế mạch:



Chạy mô phỏng waveform:



0ns – 10ns: CS = 1, RWS = 0 => các cell hàng 0 được kích hoạt và cho ra output là 0000 vì giá trị khởi tạo của các cell là 0

30ns: CS = 1, RWS = 1 => các cell hàng 0 được kích hoạt và được nạp giá trị 1111

40ns – 50ns: CS = 0, RWS = 1 => các cell hàng 0 ở chế độ ghi nhưng không được kích hoạt

50ns – 70ns: CS = 1, RWS = 0 => các cell hàng 0 ở chế độ đọc và xuất ra input đã được ghi trước đó là 1111

80ns : CS = 1, RWS = 1 => các cell hàng 3 được kích hoạt và được nạp giá trị 1100

90ns – 100ns: CS = 1, RWS = 0 => các cell hàng 3 ở chế độ đọc và xuất ra input đã được ghi là 1100

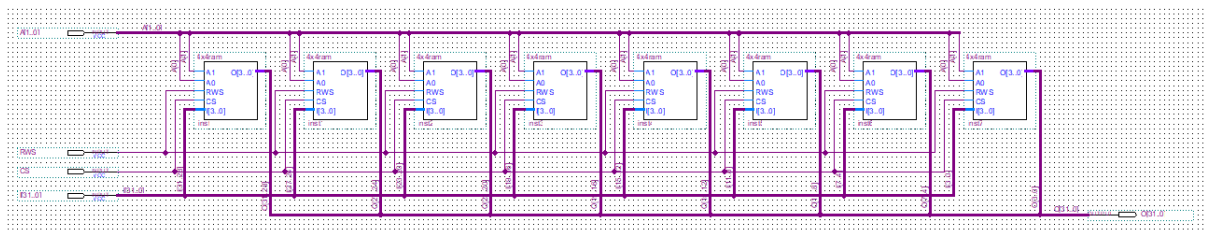
100ns – 110ns: CS = 1, RWS = 0 => các cell hàng 0 ở chế độ đọc và xuất ra input đã được ghi là 1111

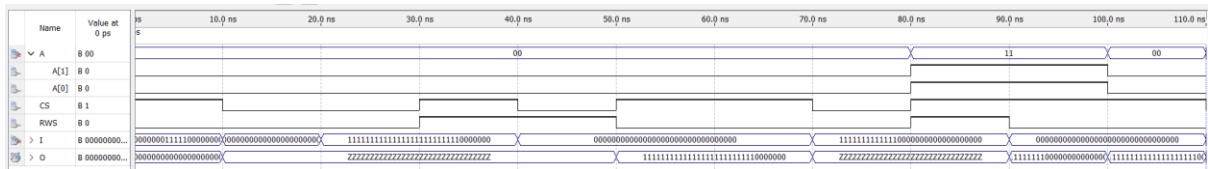
3. 4x32ram

Mạch 4x32 ram được thiết kế bằng cách ghép 8 mạch 4x4ram với nhau, các input như A[1..0], RWS, CS tương tự 4x4ram

Input[31..0], Output[31..0] là các input và output của các cell nhớ

Thiết kế mạch:



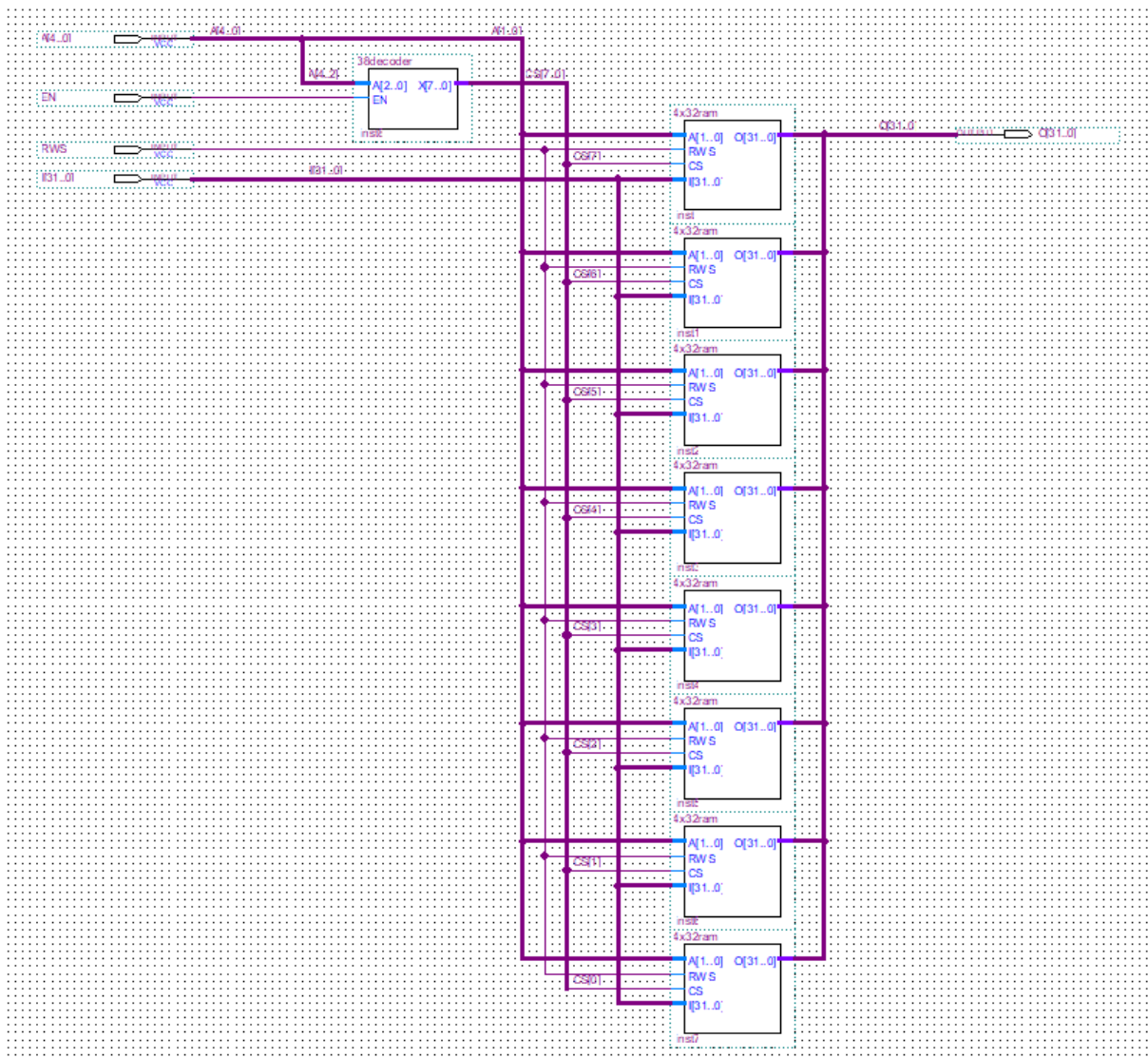


Giải thích tương tự như 4x4ram

4. 32x32ram

Ý tưởng thiết kế: Ghép 8 mạch 4x32ram với nhau

Thiết kế mạch:

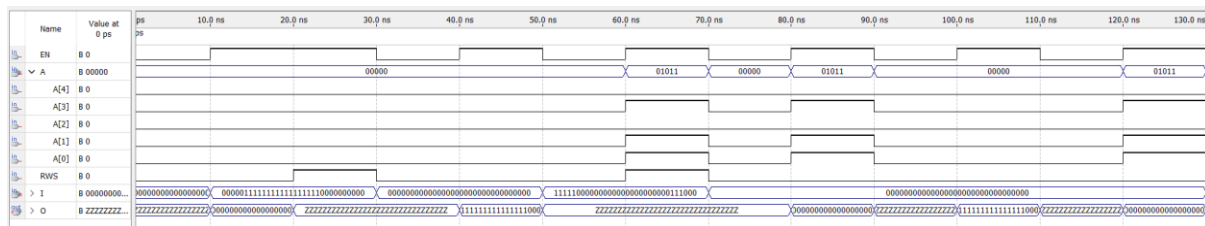


Giải thích thiết kế:

- Input:
 - A[4..0]: 5 bit để chọn dòng thao tác với cell ram, trong đó
 - A[4..2]: 3 bit đi qua 3 to 8 decoder để chọn 1 trong 8 mạch 4x32ram

- A[1..0]: 2 bit để chọn 1 trong 4 dòng của mạch 4x32ram
- EN: bit dùng để kích hoạt 32x32ram
 - EN = 1: chọn 1 trong 8 mạch 4x32ram
 - EN = 0: không chọn mạch 4x32ram nào
- RWS: bit chọn chế độ đọc ghi cho mạch
 - RWS = 1: Chế độ ghi
 - RWS = 0: Chế độ đọc
- I[31..0]: 32 bit input cần lưu trữ
- Output: O[31..0]: 32 bit output của các cell đã được lưu trữ

5. Waveform



0ns- 10ns: EN = 0 => Mạch không được kích hoạt nên có output là Z

10ns – 20ns: EN = 1, A[4..0] = 00000, RWS = 0 => Đọc giá trị tại các cell nhớ dòng thứ 1 của mạch 4x32 thứ 1, giá trị output là 32 bit 0

20ns – 30ns: EN = 1, A[4..0] = 00000, RWS = 1 => Ghi giá trị như waveform vào các cell nhớ dòng thứ 1 của mạch 4x32 thứ 1

40ns – 50ns: EN = 1, A[4..0] = 00000, RWS = 0 => Đọc giá trị tại các cell nhớ dòng thứ 1 của mạch 4x32 thứ 1, giá trị output như waveform

60ns – 70ns: EN = 1, A[4..0] = 01011, RWS = 1 => Ghi giá trị như waveform vào các cell nhớ dòng thứ 4 của mạch 4x32 thứ 3

80ns – 90ns: EN = 1, A[4..0] = 01011, RWS = 0 => Đọc giá trị tại các cell nhớ dòng thứ 4 của mạch 4x32 thứ 3, giá trị output như waveform

100ns – 110ns: Đọc giá trị tương tự 40ns – 50ns

120ns – 130ns: Đọc giá trị tương tự 80ns – 90ns

E. Thiết kế Push-down Stack (LIFO)

1. Cơ sở lý thuyết

Xác định input và output:

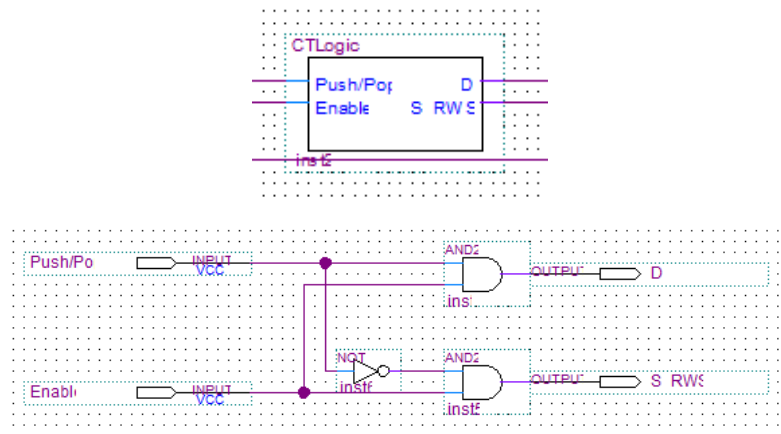
- Cần 4 input:
 - Push/pop: thực hiện chức năng Push hoặc Pop
 - Enable: cho phép các khối hoạt động
 - CLK: điều khiển xung clock
 - Reset: đưa về trạng thái ban đầu

Enable	Reset	Push/Pop	Operation
0	X	X	No change
1	1	X	Reset
1	0	0	Push
1	0	1	Pop

- Output:
 - 32 bit từ RAM32x32
 - 2 cờ báo: Full & Empty

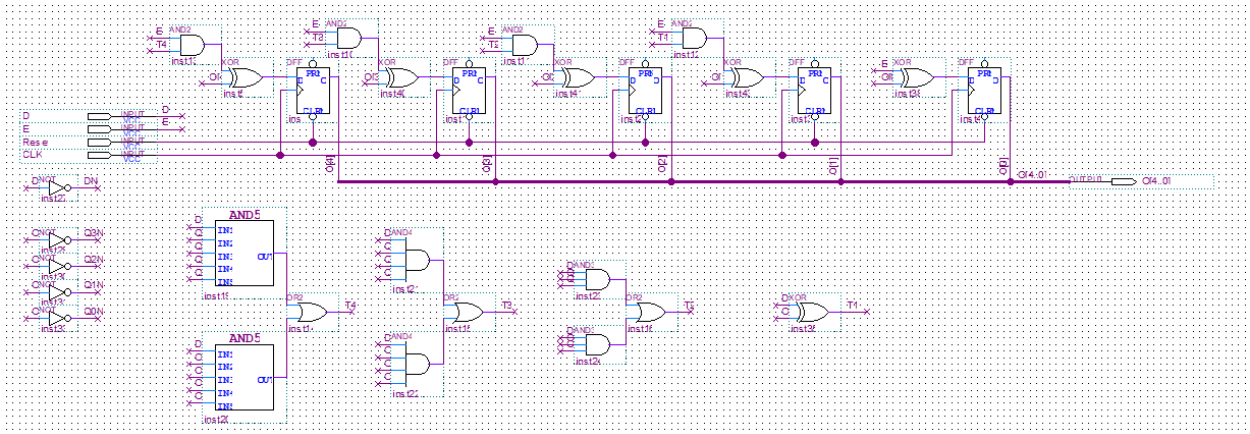
2. Thiết kế các khối cần thiết cho mạch

a. CTLogic:



- Chức năng: Xử lý 2 input là Enable và Push/Pop để cho ra output là input của khối Top, Top – 1 và mux21

b. Top:



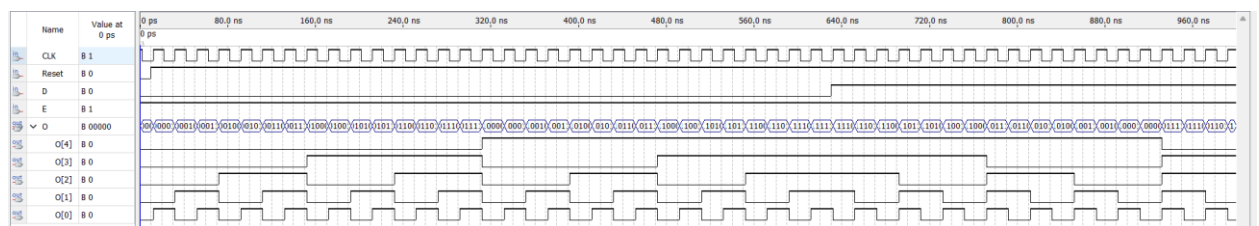
- Chức năng: Quản lý và truy cập phần tử ở đỉnh của Stack
 - o Push: đếm lên [0; 31]
 - o Pop: đếm xuống
- Vì giá trị phần tử chỉ đếm đến 31 → Cần 5 DFF
- Input DFF:
 - o D, E: điều khiển input D
 - o Reset: đưa về trạng thái 00000₂
 - o CLK: điều khiển xung clock

$$T'_n = (D' \cdot \text{Carry up}_n) + (D \cdot \text{Carry Down}_n)$$

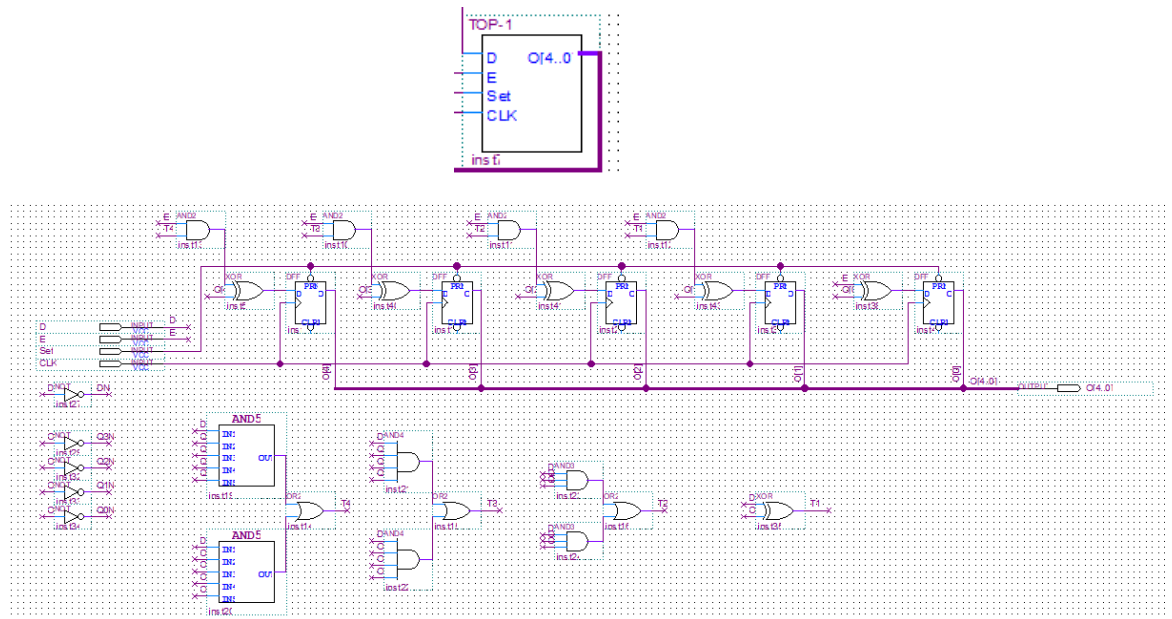
- o Carry up_n: điều kiện để lật khi đếm lên
- o Carry down_n: điều kiện để lật khi đếm xuống

$$D_n = Q_n \oplus (E \cdot T'_n)$$

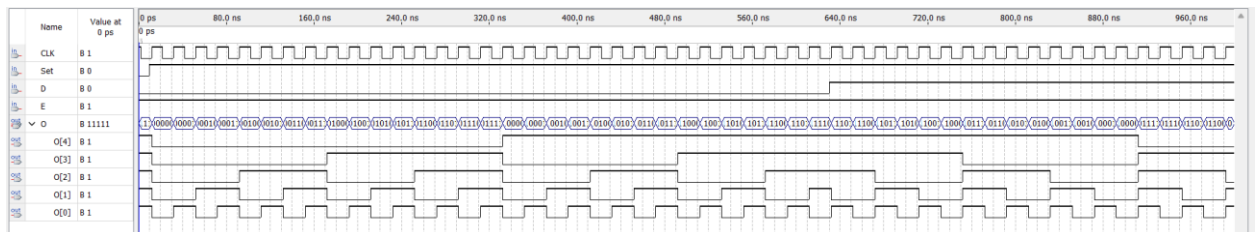
- Output 5 DFF:
 - o Dùng cho input của MUX21 để chọn input 5 bit cho RAM32x32
 - o Dùng cho 2 cờ báo Full & Empty
- Waveform:



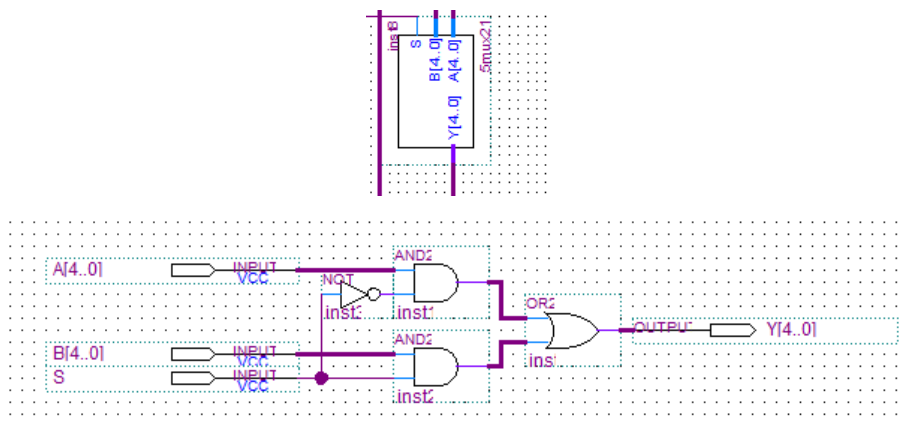
c. Top – 1



- Chức năng: Quản lý và truy cập dữ liệu phần tử ở đỉnh – 1 của Stack
- Input DFF: Giống Top nhưng thay chân Reset thành chân Set để đưa về trạng thái 1111₂
- Output DFF:
 - o Dùng cho input của MUX21 để chọn input 5 bit cho RAM32x32
- Waveform



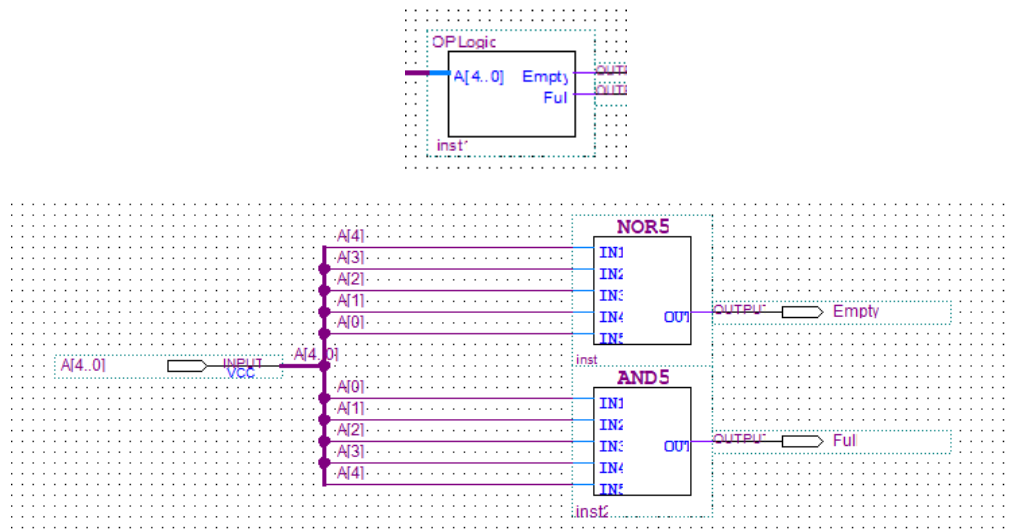
d. Mux21



- Chức năng: chọn input 5 bit cho RAM 32x32 từ output Top & Top – 1

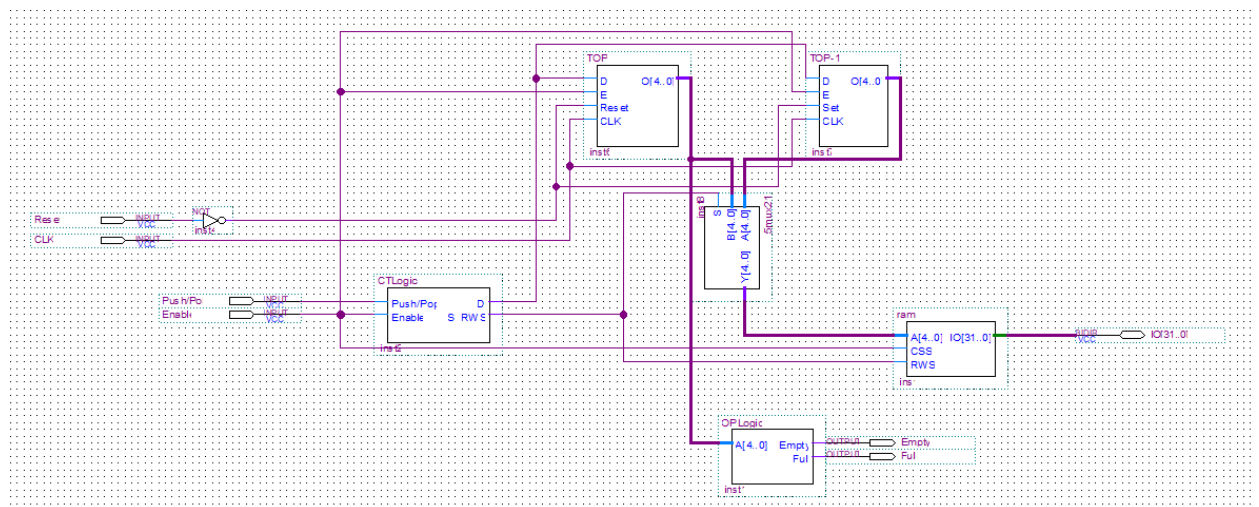
$$Y = S'.A + S.B$$

e. OPLogic

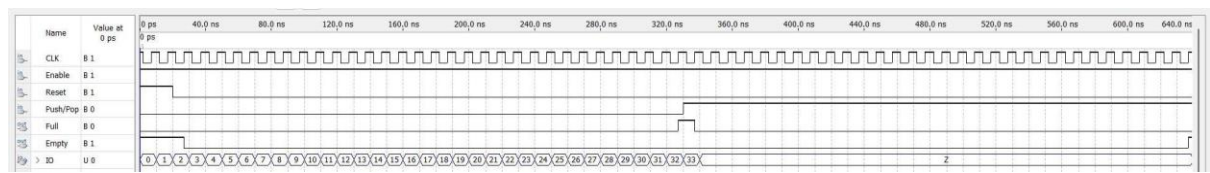


- Chức năng: xử lý 5 bit Output của Top để báo cờ Full hoặc Empty

3. Thiết kế mạch Push-down Stack



4. Waveform



- 0 – 60ns: Mạch chưa có phần tử (chưa Enable; Reset) → Empty = 1
- 60 – 72: Empty = 1 vì CLK chưa tích cực mức lên
- 672ns: Top đã đếm tới 31 → Full = 1 khi CLK tích cực mức lên
- 1292ns: Top đã đếm về 0 → Empty = 1 khi CLK tích cực mức lên

F. Thiết kế queue (FIFO)

1. Cơ sở lý thuyết

Yêu cầu: Thiết kế mạch queue sử dụng ram 32x32

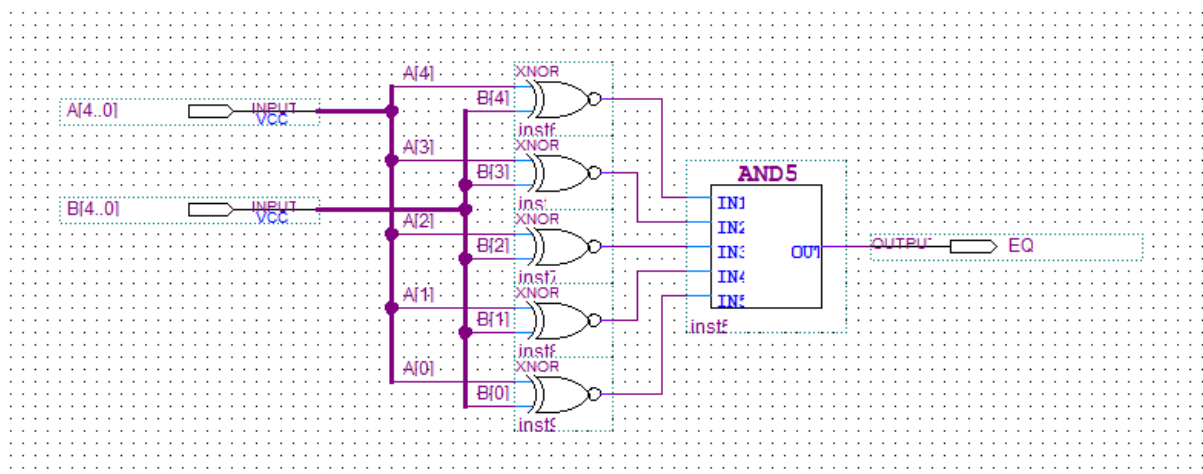
Input:

- I[31..0]: dữ liệu input
- Read/Write: Chọn chế độ đọc hoặc ghi
- En: chân kích hoạt queue
- Reset : reset queue
- Clk : xung clock

Output : O[31..0] : cho ra output tùy lựa chọn đọc ghi

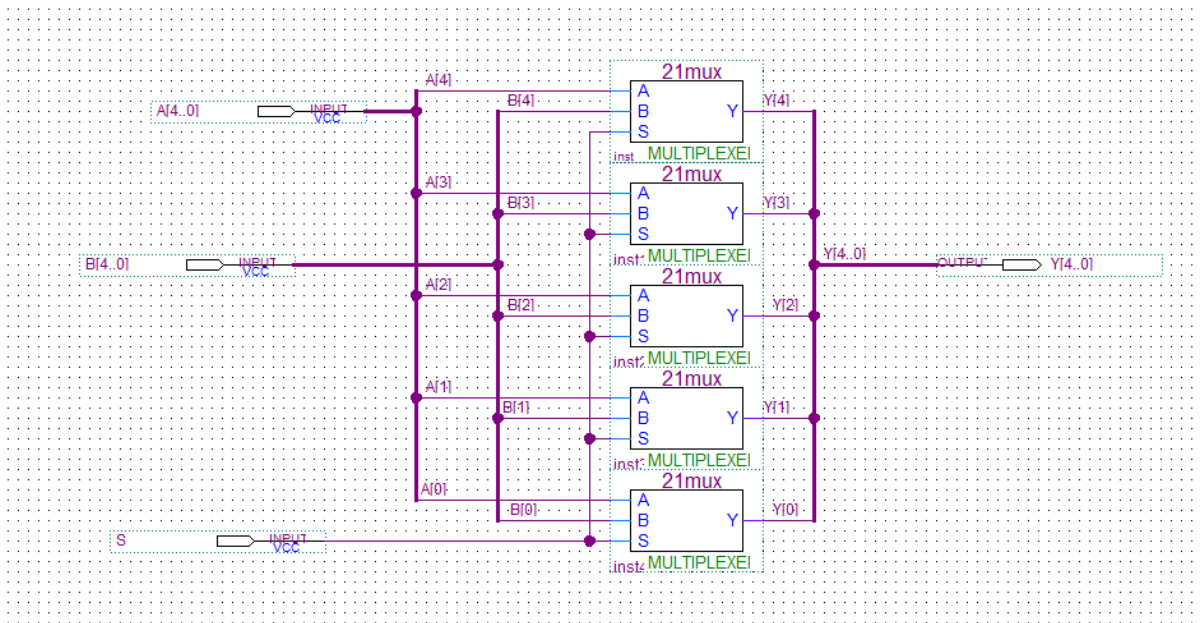
2. Thiết kế thành phần

a. Comparator



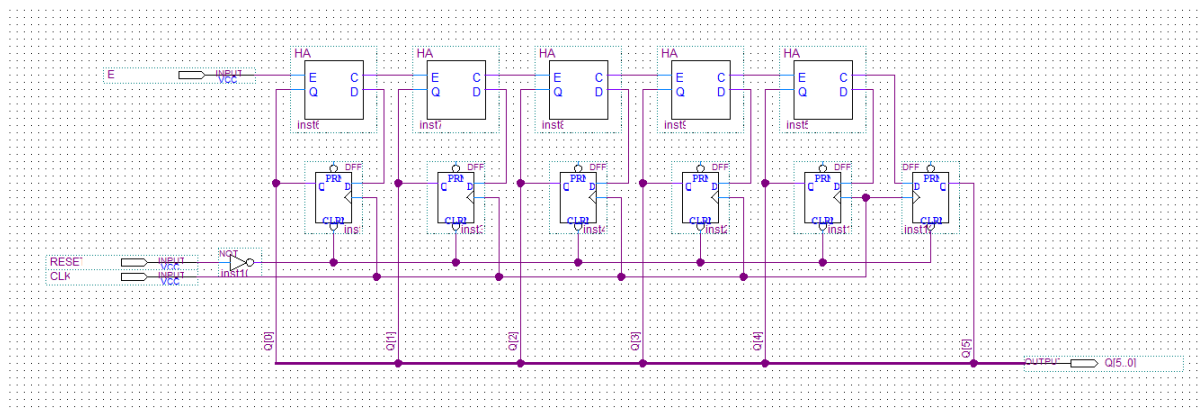
Mạch comparator có công dụng so sánh 5 bit địa chỉ của 2 con trỏ front và back nhằm xác định xem chúng có bằng nhau hay không

b. Selector



Selector giúp chọn địa chỉ cho chế độ đọc hoặc ghi

c. Counter

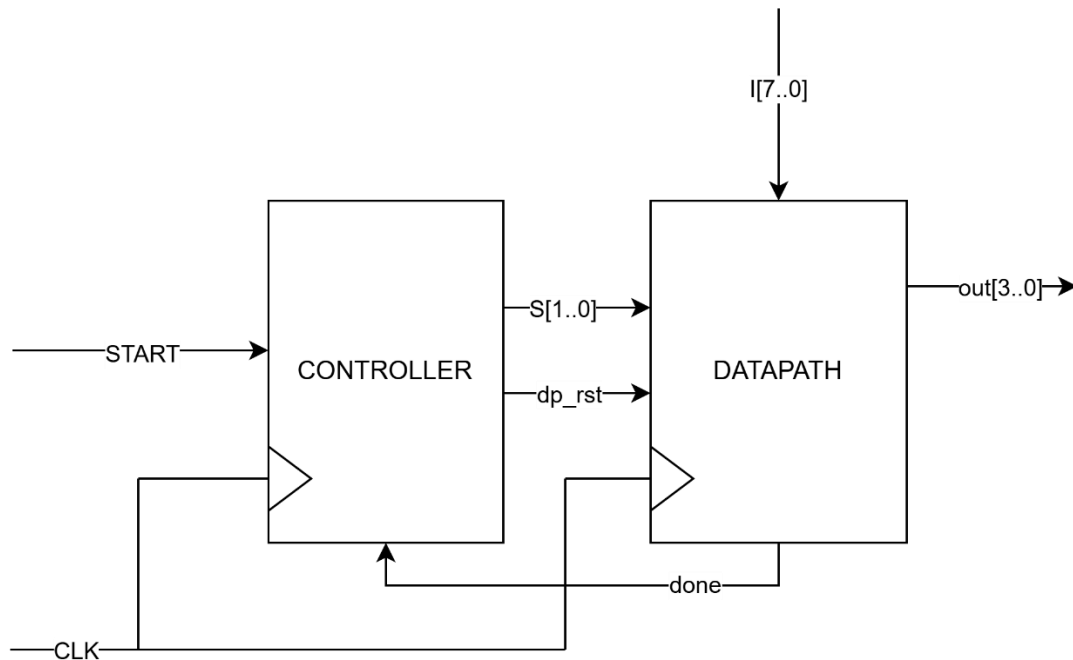


Counter giúp quản lý địa chỉ để đọc (front) và ghi (back)

G. Thiết kế bộ đếm số lượng bit 1

1. Cơ sở lý thuyết

Bộ đếm bao gồm mạch datapath thực hiện đếm và controller để điều khiển datapath



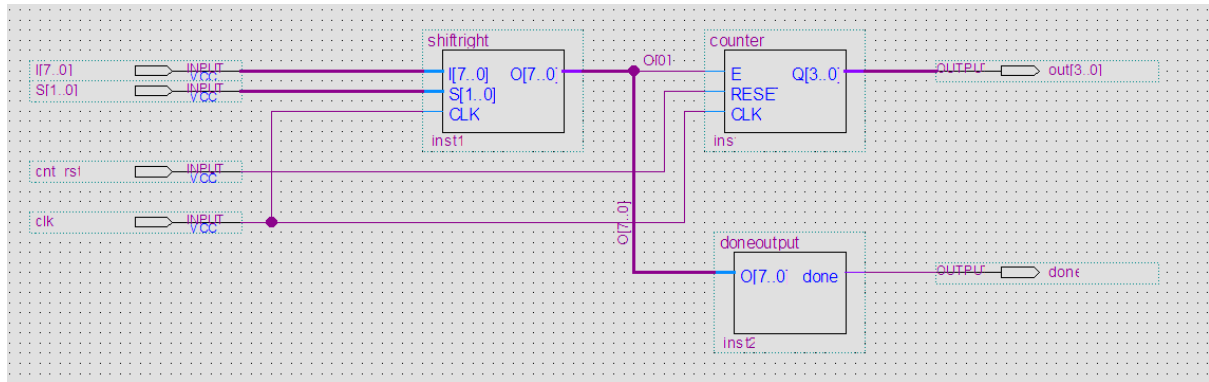
2. Thiết kế datapath

Datapath bao gồm 2 khối là shift-right và counter với

- Input:
 - I[7..0]: input data
 - S[1..0]: điều khiển khối shift-right
 - S = 00: giữ nguyên
 - S = 01: load input
 - S = 11: dịch phải 1 bit
 - Reset: reset counter
 - Clk: xung clock
- Output:
 - Done: kiểm tra đã đếm xong
 - Out[3..0]: số lượng bit 1

Thiết kế:

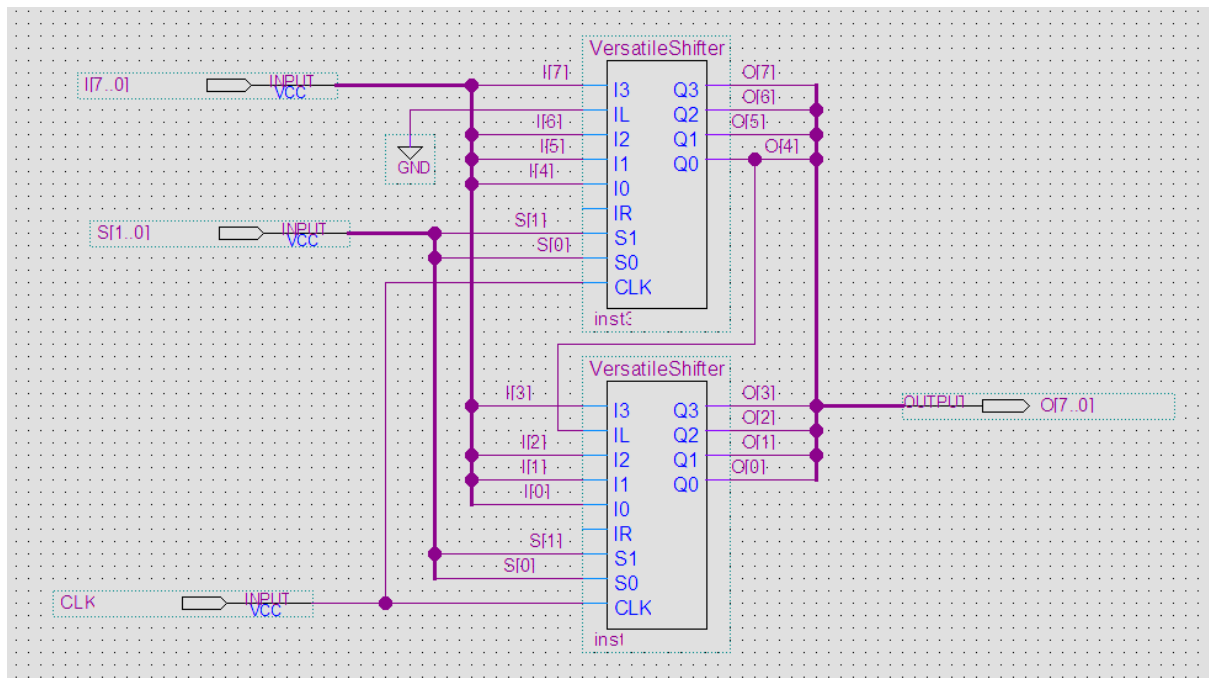
- Mạch tổng quan:



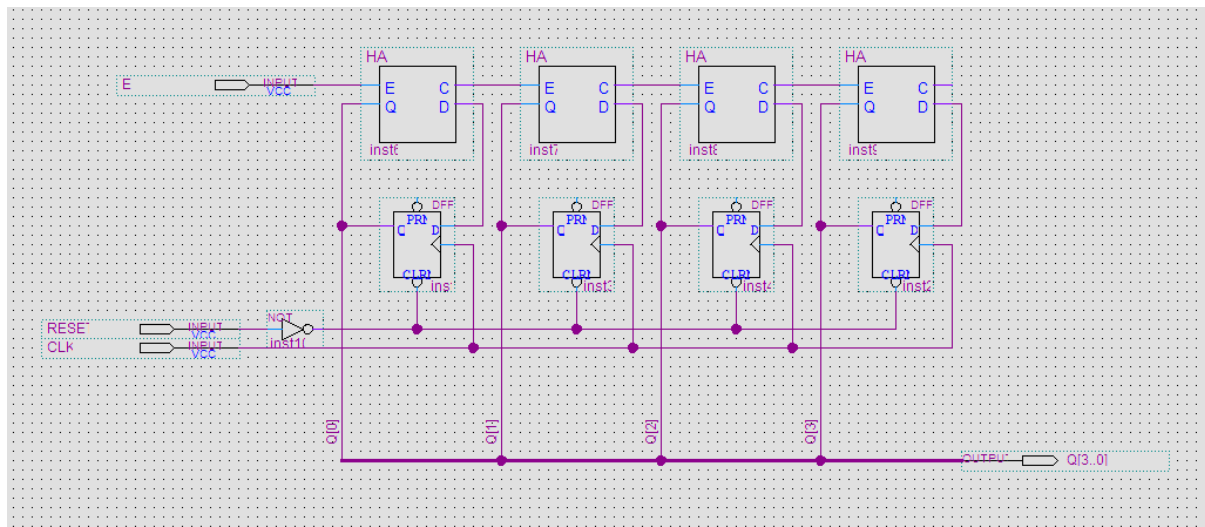
Tín hiệu done được kích hoạt khi Output của khối shift-right là 0

- Mạch shift right:

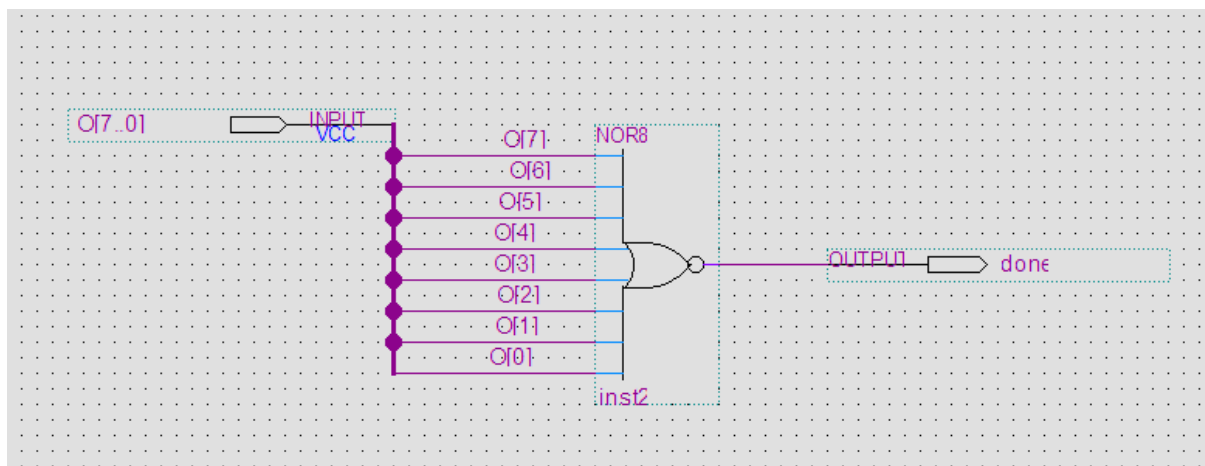
Được nối từ 2 mạch SRwPL đã được thiết kế từ bài trước, sử dụng chung chân S[1..0] làm tín hiệu điều khiển



- Mạch counter đếm lên có reset:



- Mạch báo done:



3. Thiết kế controller

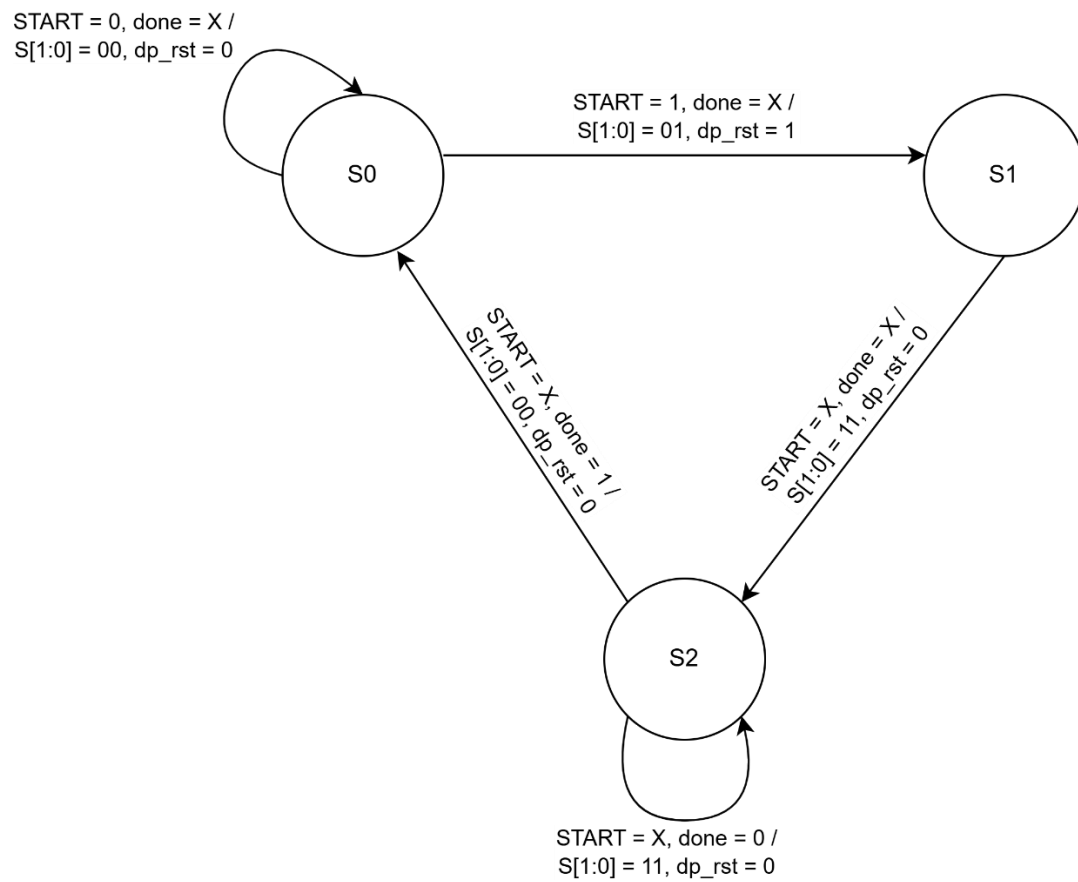
Controller sẽ bao gồm:

- Input:
 - START: tín hiệu khởi động controller
 - done: lấy tín hiệu done từ datapath
 - clk: xung clock
- Output:
 - S[1..0]: điều khiển khối shiftright
 - dp_rst: reset counter của datapath

Ta có 3 trạng thái cho controller :

- S0: Nghỉ
- S1: Load input
- S2: Xử lí đếm

Sơ đồ chuyển trạng thái:



Bảng chuyển trạng thái:

TTHT	INPUT		TTKT	OUTPUT		
	START	done		S[1]	S[0]	dp_rst
S0	0	X	S0	0	0	0
S0	1	X	S1	0	1	1
S1	X	X	S2	1	1	0
S2	X	0	S2	1	1	0
S2	X	1	S0	0	0	0

Mã hóa trạng thái:

S0: 00 (Q1'Q0')

S1: 01 (Q1'Q0)

S2: 10 (Q1Q0')

Vì có 3 trạng thái nên ta sử dụng 2 D-flipflop

Biểu thức input cho các D-flipflop:

$$D0 = Q0+ = \sim Q1 \& \sim Q0 \& START$$

$$D1 = Q1+ = (\sim Q1 \& Q0) \mid (Q1 \& \sim Q0 \& \sim done)$$

Biểu thức output của controller:

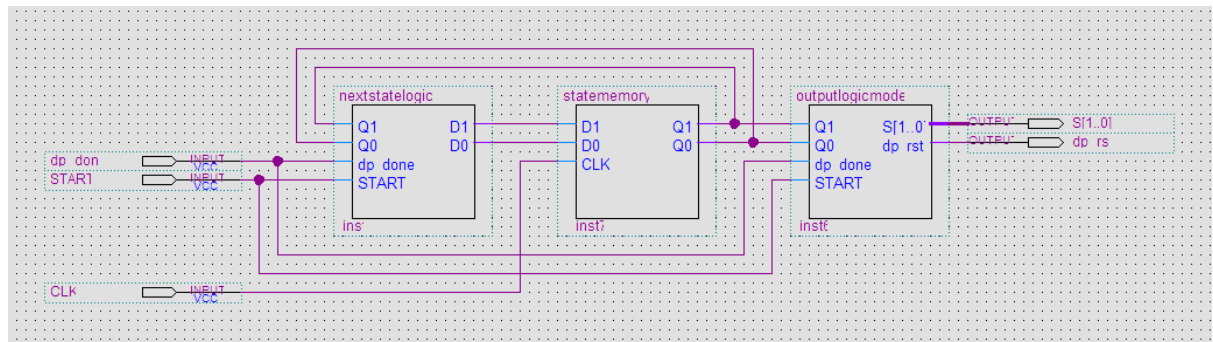
$$S[1] = (\sim Q1 \& Q0) \mid (Q1 \& \sim Q0 \& \sim done)$$

$$S[0] = (\sim Q1 \& \sim Q0 \& START) \mid (\sim Q1 \& Q0) \mid (Q1 \& \sim Q0 \& \sim done)$$

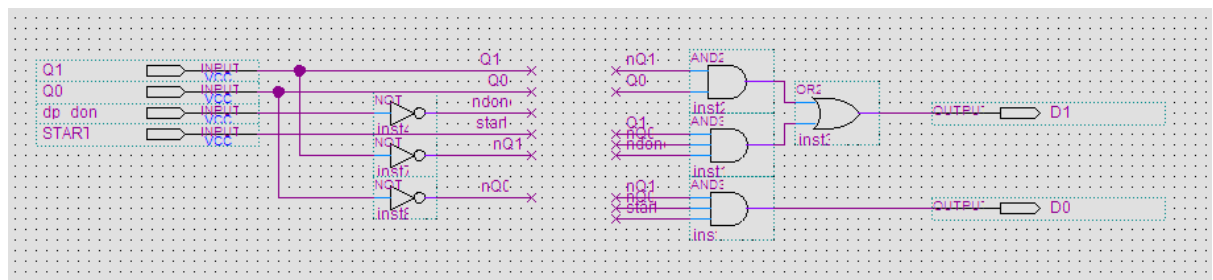
$$dp_rst = \sim Q1 \& \sim Q0 \& START$$

Thiết kế:

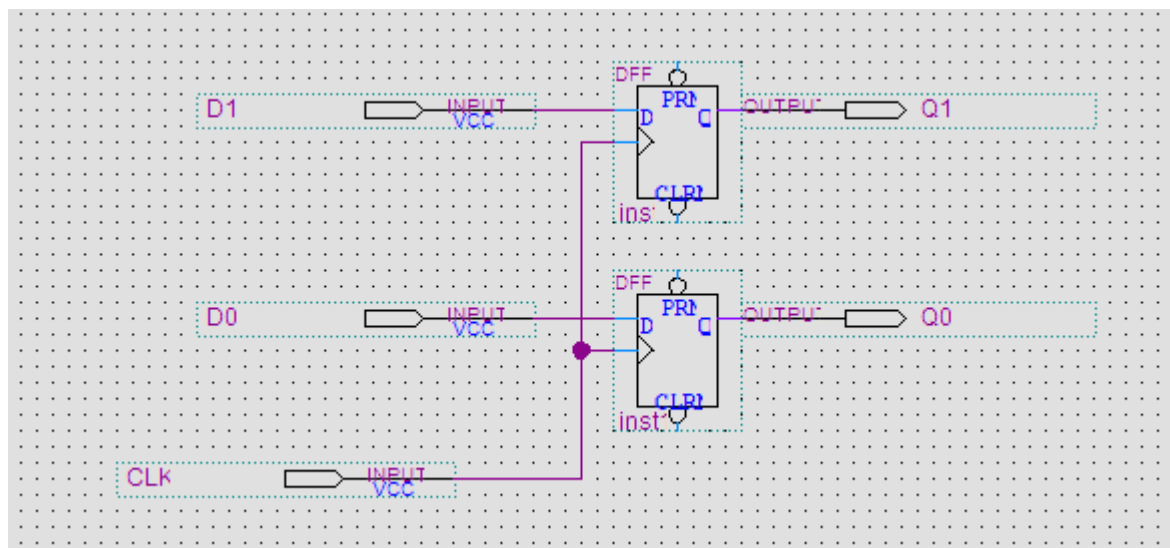
- Mạch tổng quan:



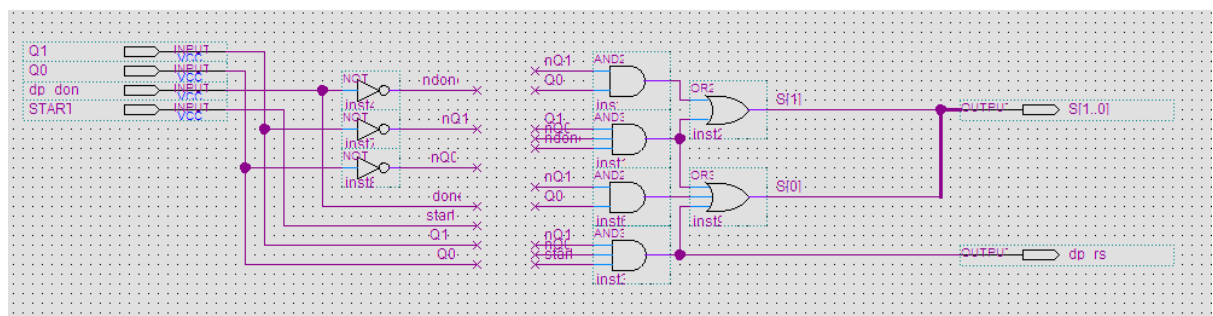
- Mạch next state logic:



- Mạch state memory:

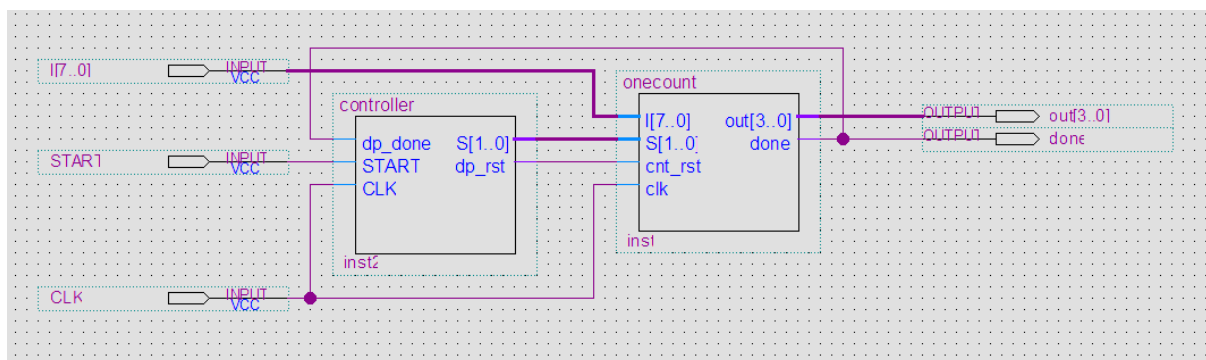


- Mạch output logic mode:

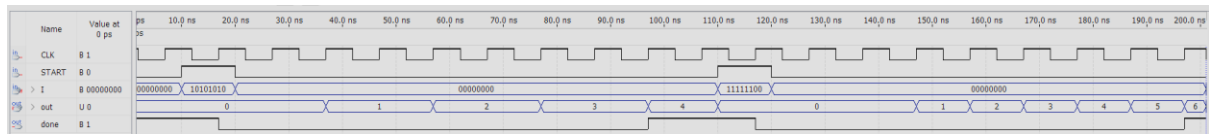


4. Mạch hoàn chỉnh

Ta kết hợp controller và datapath thành mạch đếm bit 1 hoàn chỉnh:



5. Waveform



0ns – 10ns: START = 0 -> mạch không được load và đếm và done = 1

10ns – 20ns: START = 1 -> mạch load input, tại 17ns có xung cạnh lên thì reset done = 0

20ns – 100ns: máy bắt đầu đếm và đến khi đã đếm hết số bit 1 thì bật done = 1 ở 97ns

110ns – 120ns: START = 1 -> mạch load input và reset output về 0, tại 117ns có xung cạnh lên thì reset done = 0

120ns – 200ns: máy bắt đầu đếm và khi đã đếm hết số bit 1 thì bật done = 1 ở 197ns

H. Thiết kế bộ tính tổng từ 1 đến 8

1. Ý tưởng thiết kế

Thiết kế 2 thành phần Controller và Datapath để điều khiển mạch tính tổng

1.1. Datapath

- Các khối cần thiết:

- Counter Block (4 bit): đếm từ 1 đến 8 và có chức năng reset về 1
- Done Check (DC) Block: kiểm tra nếu đã đến 8 thì hoàn thành
- Vì tổng từ 1 đến 8 bằng 36 $\rightarrow$ 6 bit nhị phân nên ta cần bộ cộng và bộ lưu trữ 6 bit:
 - Register (6 bit): lưu kết quả cộng từ Adder Block
 - Adder Block (6 bit): đưa output của Counter Block về 6 bit, cộng giá trị hiện tại từ Counter và Register, sau đó lưu kết quả vào Register

- Input và Output cho từng khối:

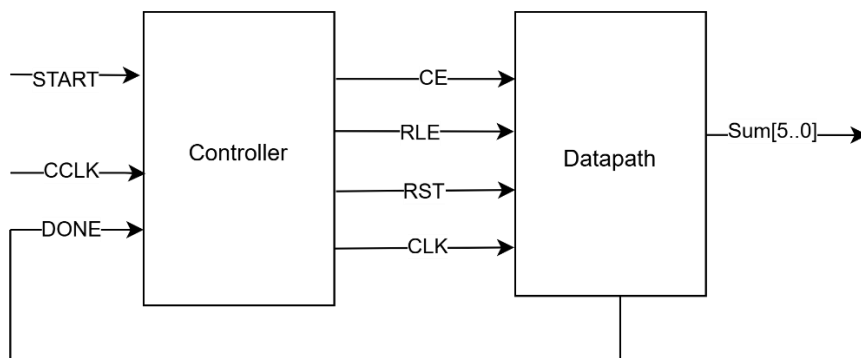
- Counter Block:
 - Input:
 - Counter Enable (CE): cho phép đếm
 - Counter Reset (CRe): đưa về trạng thái 0001_2
 - Xung Clock (CLK)
 - Output:
 - Giá trị nhị phân 4 bit – $O[3..0]$
- DC Block:
 - Input:
 - $O[3..0]$ từ Counter Block
 - Output:
 - 1 bit DONE

- Register Block:
 - Input:
 - S[5..0]: giá trị 6 bit từ Adder Block
 - Register Load Enable (RLE): cho phép ghi
 - Register Clear (R_CLR): đưa về trạng thái 000000₂
 - Xung Clock (CLK)
 - Output:
 - Sum[5..0]: Lưu trữ kết quả của Adder Block
- Adder Block:
 - Input:
 - A[3..0]: giá trị output 4 bit từ Counter Block
 - B[5..0]: giá trị output 6 bit từ Register Block
 - Output:
 - O[5..0]: Tổng của A và B, được lưu vào Register
- Input và Output của Datapath:
 - Input:
 - Counter Enable (CE): kích hoạt Counter Block
 - Register Load Enable (RLE): cho phép Register Block ghi giá trị
 - Clear (CLR): Cho CRe (Counter Block) và R_CLR (Register Block)
 - Xung Clock (CLK): điều khiển xung clock cho Counter Block và Register Block
 - Output:
 - Tín hiệu 1 bit DONE: kiểm tra hoàn thành
 - Sum[5..0]: Tổng 6 bit

1.2. Controller

- Điều khiển tín hiệu Input của Datapath
- Input và Output của Controller:
 - Input:
 - START: cho phép bắt đầu tính toán
 - DONE: tín hiệu kiểm tra hoàn thành
 - CCLK: tín hiệu xung clock
 - Output: (là các Input của Datapath)
 - CE
 - RLE
 - RST
 - CLK

1.3. Mạch tổng



2. Thiết kế Datapath

2.1.1. Thiết kế các khối trong Datapath

a. Counter Block

- Vì Block hoạt động đến số cao nhất là 8 nên dùng 4 DFF

Input		Current State				Next State			
CE	Reset	Q3	Q2	Q1	Q0	Q3+	Q2+	Q1+	Q0+
0	X	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0	1
	0	0	0	0	1	0	0	1	0
		0	0	1	0	0	0	1	1
		0	0	1	1	0	1	0	0
		0	1	0	0	0	1	0	1
		0	1	0	1	0	1	1	0
		0	1	1	0	0	1	1	1
		0	1	1	1	1	0	0	0
		1	0	0	0	1	0	0	1

- Input DFF:

○ D3:

Q3Q2/Q1Q0	00	01	11	10
00	X	0	0	0
01	0	0	1	0
11	X	X	X	X
10	1	X	X	X

$$D3 = CE'.Q3 + CE.(Q3 + Q2Q1Q0)$$

○ D2:

Q3Q2/Q1Q0	00	01	11	10
00	X	0	1	0
01	1	1	0	1
11	X	X	X	X
10	0	X	X	X

$$D2 = CE'.Q2 + CE . (Q2'Q1Q0 + Q2Q1' + Q2Q0')$$

○ D1:

Q3Q2/Q1Q0	00	01	11	10
00	X	1	0	1
01	0	1	0	1
11	X	X	X	X
10	0	X	X	X

$$D1 = CE'.Q1 + CE . (Q1'Q0 + Q1Q0')$$

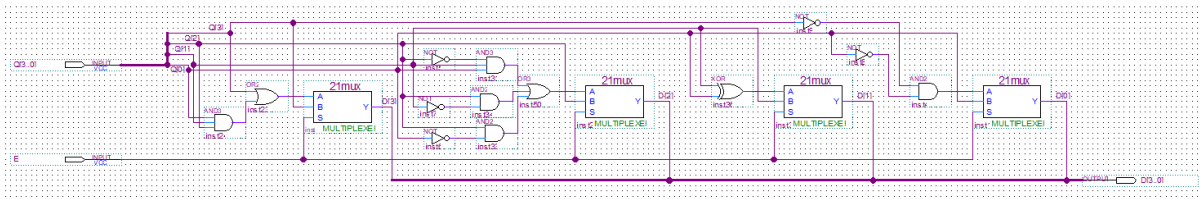
○ D0:

Q3Q2/Q1Q0	00	01	11	10
00	X	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	X	X	X

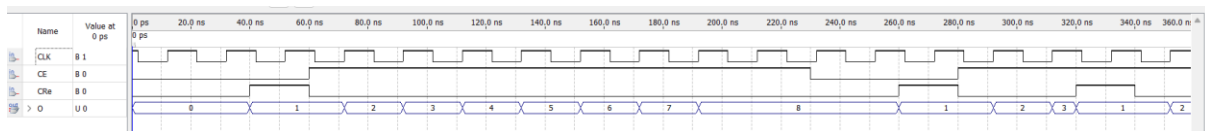
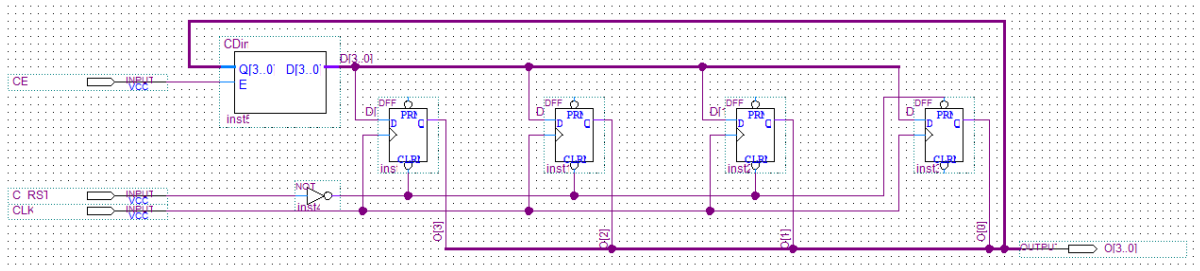
$$D0 = CE'.Q0 + CE . Q0'$$

- Mô phỏng:

- Đóng gói tín hiệu điều khiển Input thành 1 khối (CDin):



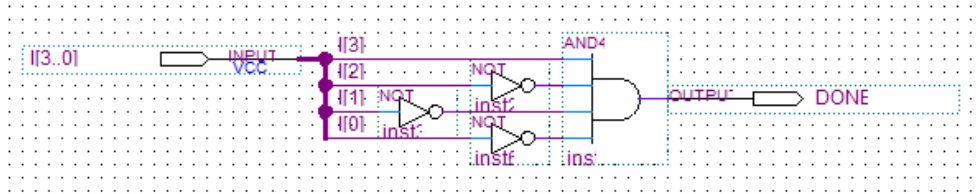
- Counter Block:



- 0 – 40ns: CE = C_RST = 0 → giá trị giữ nguyên
- 40 – 60ns: CE = 0, C_RST = 1 → Reset giá trị về 1
- 72 – 192ns: CE = 1, C_RST = 0 → Đếm đến 8
- 192 – 260ns: C_RST = 0 → Giữ nguyên giá trị 8
- 260 – 280ns: C_RST = 1 → Reset giá trị về 1
- 280 – 320ns: CE = 1, C_RST = 0 → Đếm lên
- 320 – 340ns: C_RST = 1 → Reset giá trị về 1
- 340 – 360ns: CE = 1, C_RST = 0 → Đếm lên

b. DC Block

- Kiểm tra nếu đầu vào là 8 thì DONE = 1
 - $DONE = I[3].I[2]'.I[1]'.I[0]'$
- Mô phỏng:

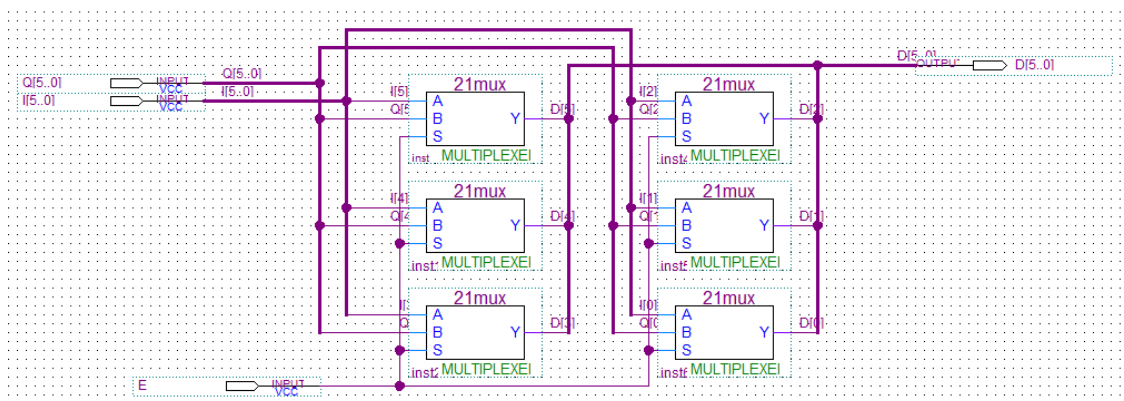


c. Register Block 6bit (Register6)

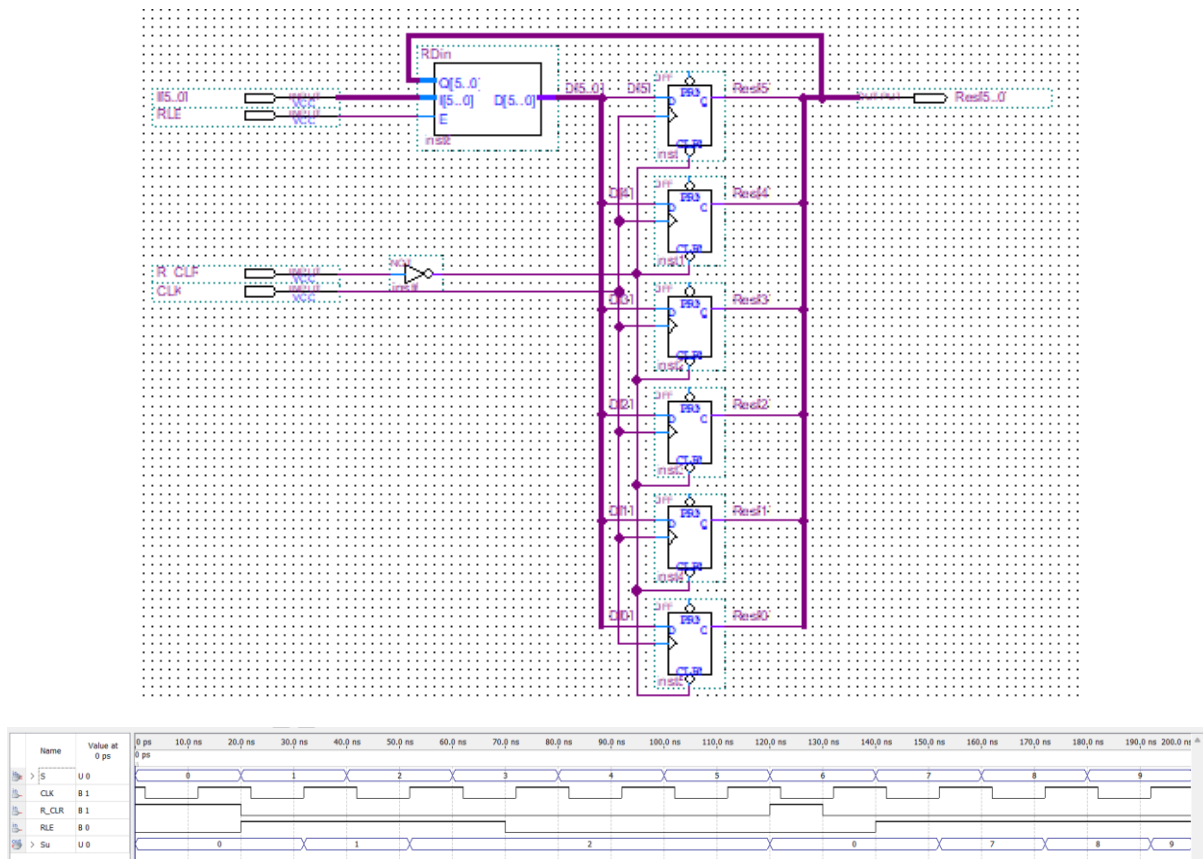
- Lưu trữ giá trị 6bit, ghi giá trị mới khi RLE = 1 và Reset về 0 khi R_CLR = 1

Input		Curr State	Next State
RLE	R_CLR	Q[5..0]	Q[5..0] +
X	1	0	0
0	0	Q[5..0]	Q[5..0]
1	0	Q[5..0]	I[5..0]

- Input DFF:
 - $D_n = RLE' \cdot Q_n + RLE \cdot I_n$
- Mô phỏng:
 - Đóng gói tín hiệu điều khiển input DFF thành 1 khối (RDin):



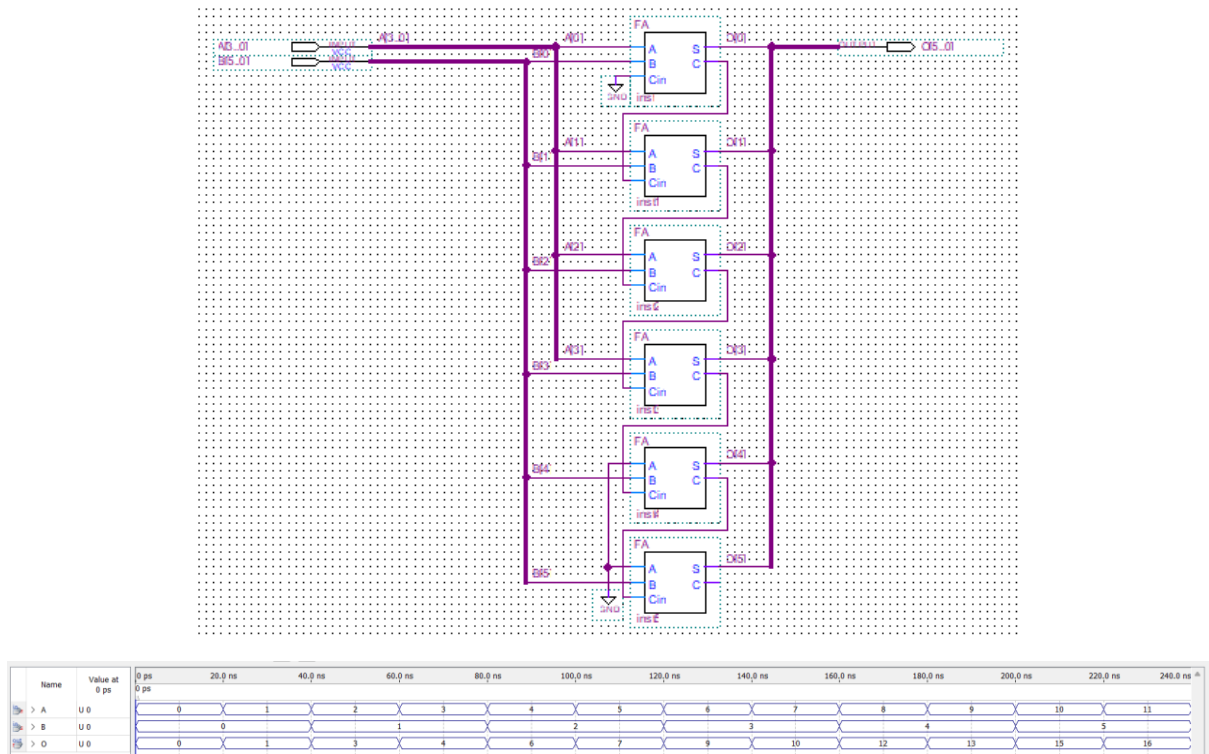
- Register Block:



- 0 – 20ns: R_CLR = 1 → Reset về 0
- 20ns – 70ns: R_CLR = 0, RLE = 1 → Ghi giá trị khi CLK tích cực mức lên
- 70 – 120ns: R_CLR = 0, RLE = 0 → Lưu trữ giá trị
- 120 – 130ns: R_CLR = 1 → Reset giá trị đang lưu trữ về 0
- 140 – 200ns: R_CLR = 0, RLE = 1 → Ghi giá trị khi CLK tích cực mức lên

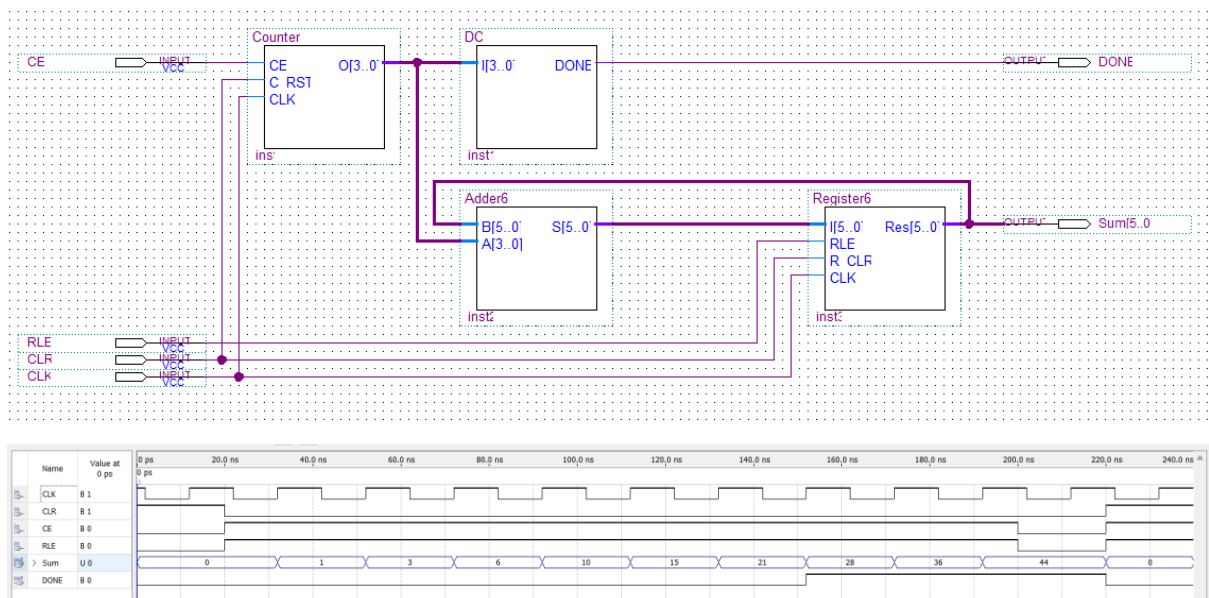
d. Adder Block 6bit (Adder6)

- Tính tổng của output Counter Block và giá trị trong Register Block
 - Vì output của Register Block là 6 bit → đưa output Counter Block về 6 bit với 2 bit cao là 00
 - Bộ cộng 6 bit nên sử dụng 6 FA để xử lý, với bit nhớ đầu là 0
- Mô phỏng:



2.1.2. Thiết kế Datapath

- Kết nối các Block theo Input và Output đã đề ra trong Phần 1.1, ta có:



- 0 – 20ns: CLR = 1, CE = 0, RLE = 0 → Reset giá trị về 0

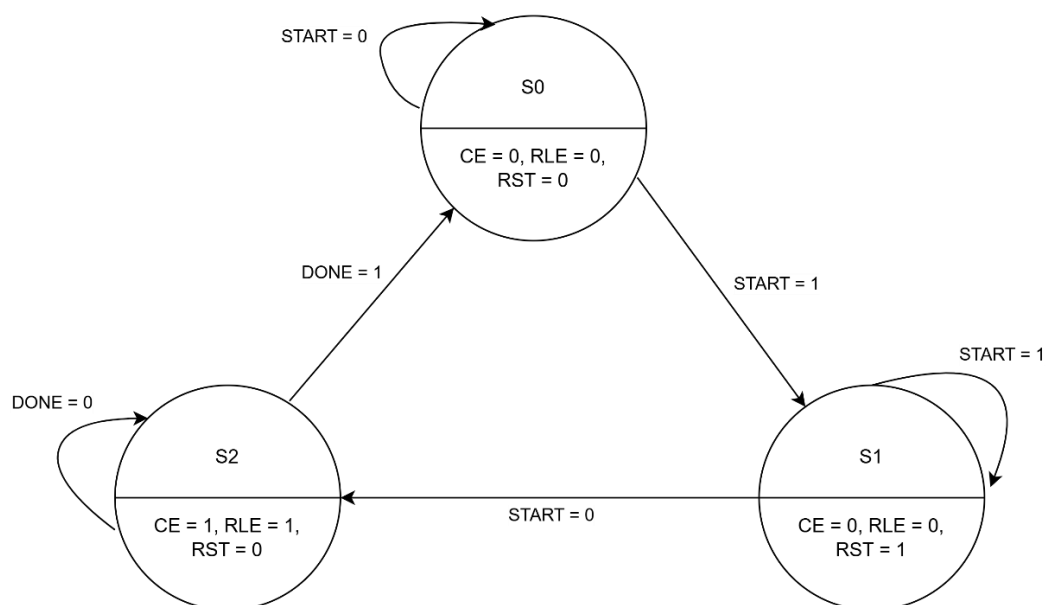
- 20 – 200ns: $CLR = 0$, $CE = 1$, $RLE = 1 \rightarrow$ Datapath hoạt động tại chế độ RUN, giá trị tổng được cập nhật mỗi khi CLK tích cực mức lên
 - o Tại 152ns: Counter Block = 8 $\rightarrow$ DONE = 1, nhưng đến xung clock sau thì DONE mới cập nhật tín hiệu điều khiển tại Controller.
- 200 – 220ns: $CLR = CE = RLE = 0 \rightarrow$ Datapath giữ trạng thái hiện tại
- 220 – 240ns: $CLR = 1 \rightarrow$ Reset giá trị về 0

3. Thiết kế Controller

- Mã hóa trạng thái:

Trạng thái	Mã hóa	Mô tả	Chức năng
S0	00	Nghỉ	Chờ tín hiệu Start để bắt đầu
S1	01	Bắt đầu	Thực hiện Reset các khối trong Datapath
S2	10	Chạy	Thực hiện chức năng của Datapath cho tới khi DONE = 1

- FSM:



- Thiết kế:

○ Input DFF:

$$\blacksquare D1 = (Q1'.Q0.START') + (Q1.Q0'.DONE')$$

$$\blacksquare D0 = Q1'.START$$

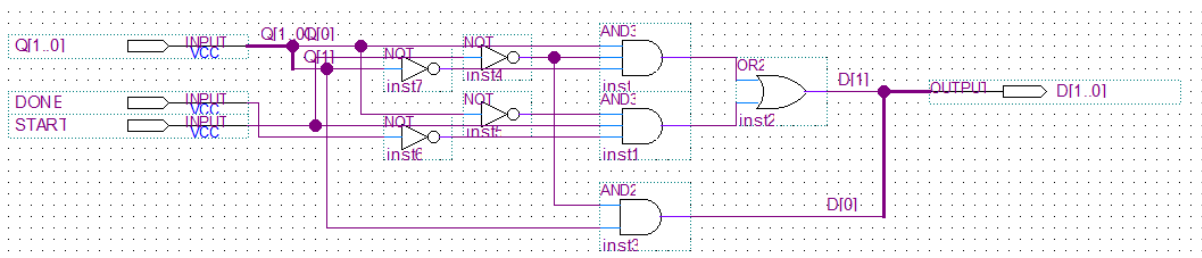
○ Output của Controller:

$$\blacksquare CE = (Q1'.Q0.START') + (Q1.Q0'.DONE')$$

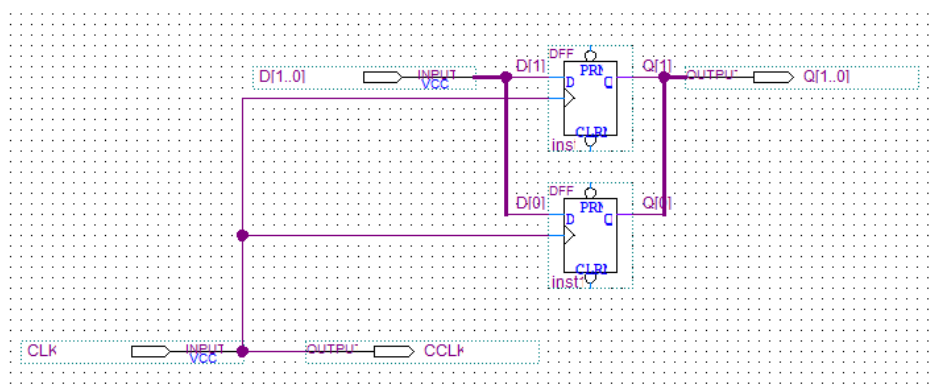
$$\blacksquare RLE = (Q1'.Q0.START') + (Q1.Q0'.DONE')$$

$$\blacksquare RST = Q1'.START$$

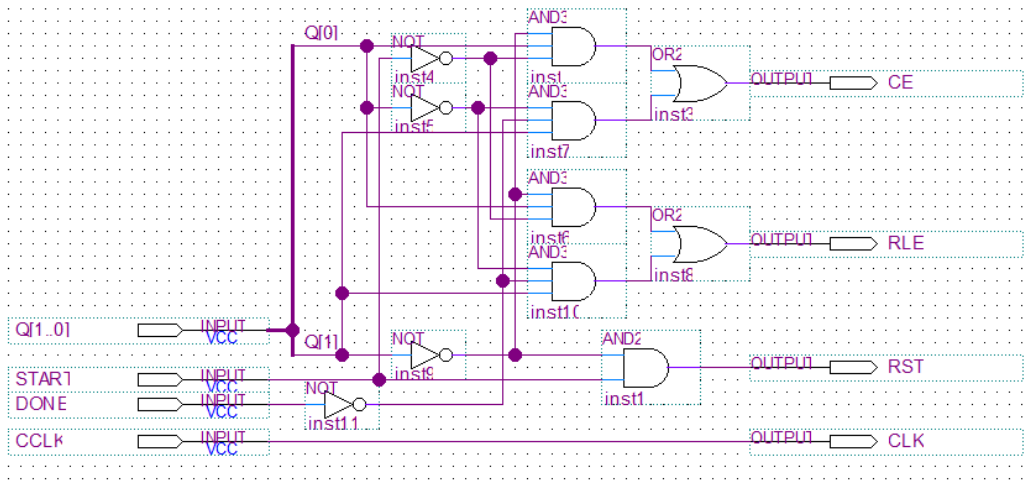
a. Thiết kế khối Next-State Logic (Comb):



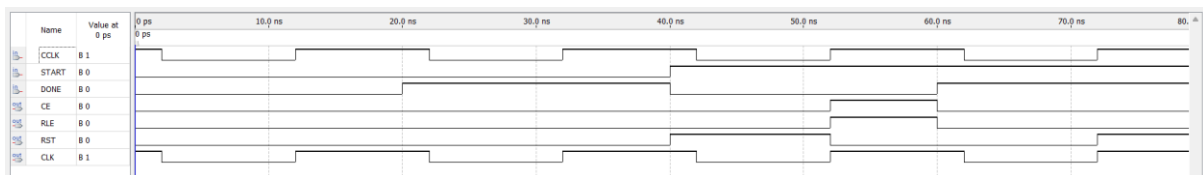
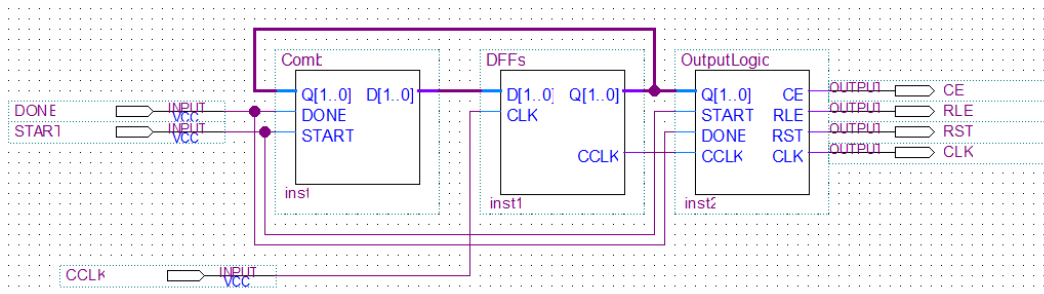
b. Thiết kế khối DFFs:



c. Thiết kế khối Output Logic:

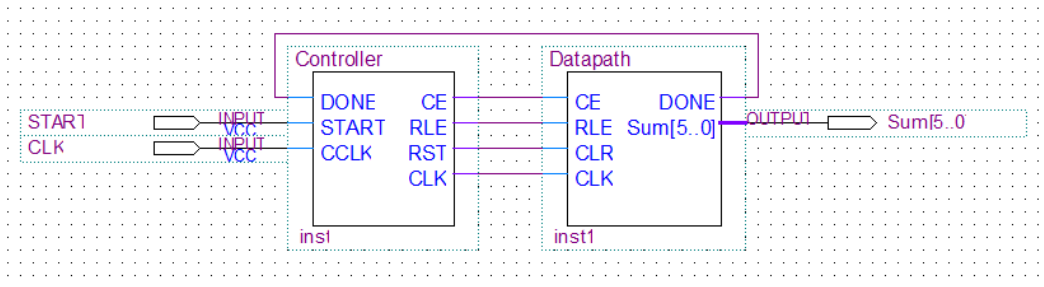


- Mô phỏng:

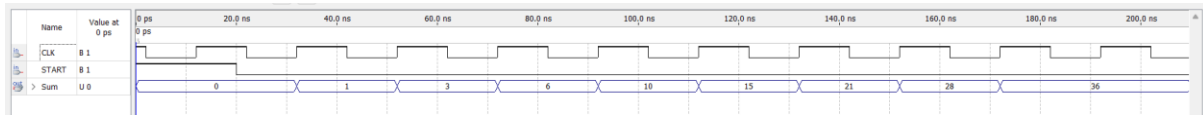


- 0 – 40ns: S0 → CE = RST = RLE = 0
- 40 – 52ns: S0 → S1 nên CE = RLE = 0, RST = 1
- 52 – 60ns: S1 → S2 nên CE = RLE = 1, RST = 0
- 60 – 72ns: S2 → S0 nên CE = RST = RLE = 0
- 72 – 80ns: S0 → S1 nên CE = RLE = 0, RST = 1

4. Thiết kế mạch tính tổng từ 1 đến 8 (Sum8)



5. Waveform



- 0 – 20ns: START = 1 → mạch đang ở trạng thái S1 (reset)
- 20 – 210ns: mạch ở trạng thái S2 (run) và tính tổng = $36 = \sum_{i=1}^8 i$

I. Thiết kế khối tính căn bậc 2 của tổng bình phương 2 số 8 bit có dấu

1. Cơ sở lý thuyết

- Input: 2 input 8 bit có dấu

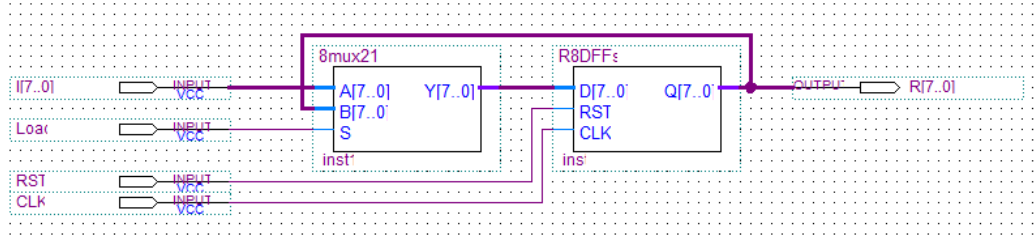
$$\sqrt{a^2 + b^2} = \max((0.875x + 0.5y), x) = \max((\frac{7}{8}x + \frac{1}{2}y), x)$$

Với: $x = \max(a, b)$, $y = \min(a, b)$

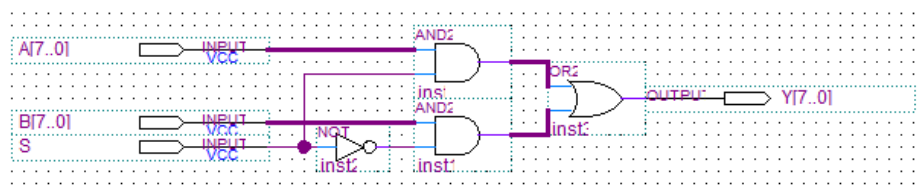
- B1: tính trị tuyệt đối của x và y
- B2: Tìm max, min
- B3: Thay đổi phép chia thành phép dịch phải bit
 - $\frac{1}{2}$: ta cần dịch phải 1 bit
 - $\frac{7}{8} = 1 - \frac{1}{8}$: dịch phải 3 bit rồi lấy số ban đầu trừ đó giá trị đã dịch
- B4: Tính tổng của 2 giá trị thương
- B5: Tìm max giữa thương và Input lớn nhất
- Các khối cần thiết cho Datapath:
 - 3 khối Register 8 bit
 - 3 khối Selector để chọn Input cho 3 Register
 - Các khối chức năng:
 - Trị tuyệt đối (AVU)
 - Min (MinUnit)
 - Max (MaxUnit)
 - Cộng 8 bit
 - Trừ 8 bit
 - Dịch phải 1 bit & 3 bit
- Output: dữ liệu 8 bit

2. Thiết kế mạch Datapath

2.1. Thiết kế các Register 8 bit



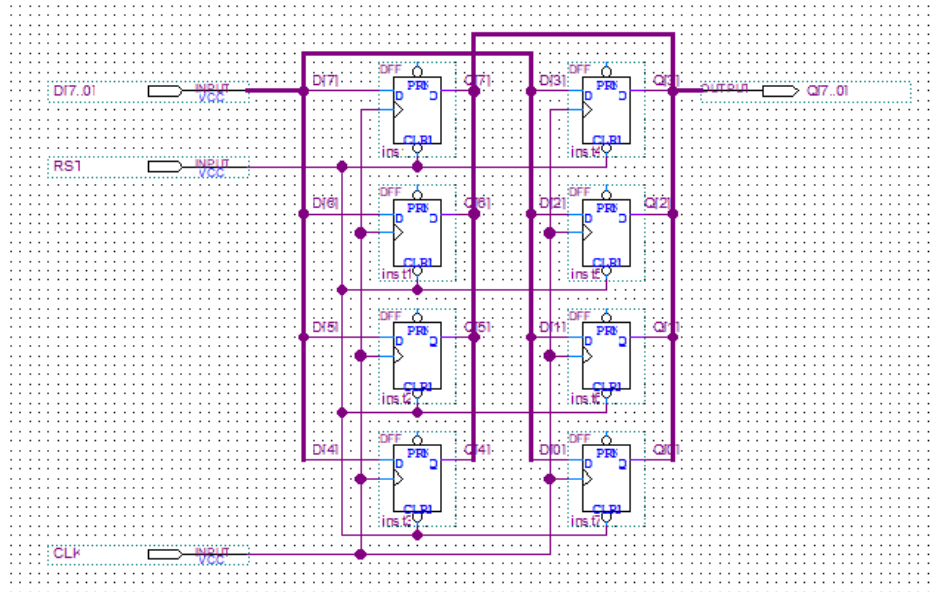
- Mạch có chức năng:
 - Nạp: khi Load = 1, RST = 0
 - Giữ: khi Load = 0, RST = 0
 - Reset: khi RST = 1
 - Giá trị thay đổi khi tín hiệu CLK tích cực mức lên
- Các khối có trong Register:
 - (1) Khối 8mux21:



▪ Chọn giá trị đầu ra dựa vào S

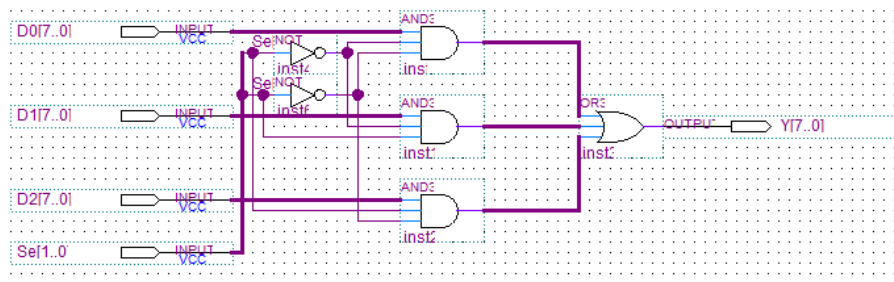
- $S = 0 \rightarrow Y[7..0] = B[7..0]$
- $S = 1 \rightarrow Y[7..0] = A[7..0]$

- Khối R8DFFs:



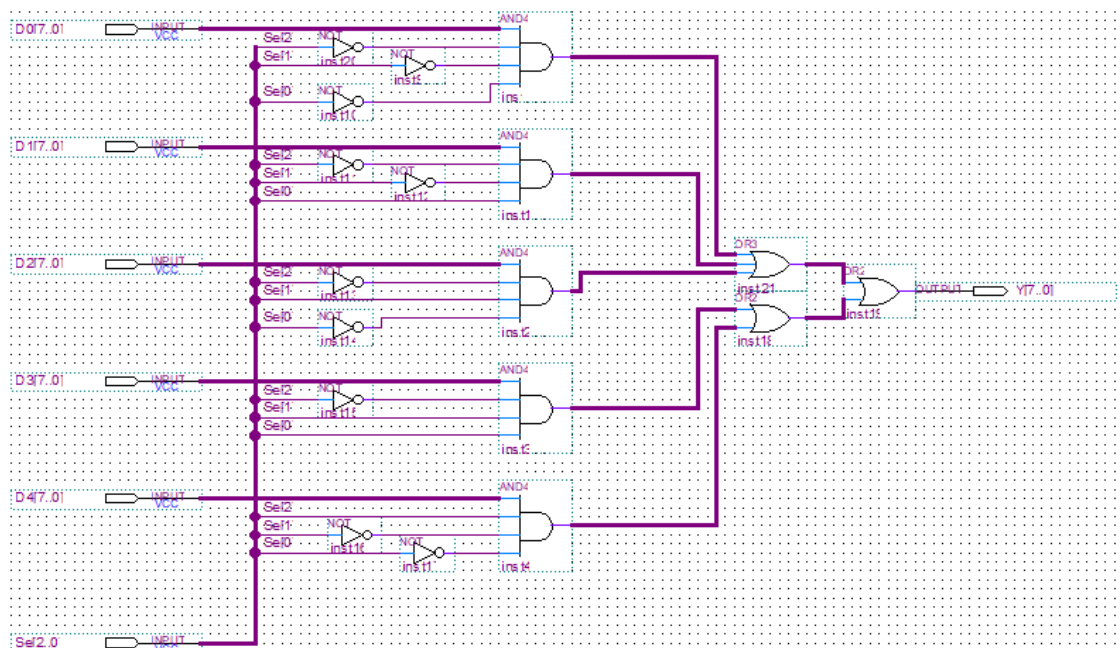
2.2. Thiết kế các Selector

a. Khối 8mux31



Chọn giá trị đầu ra dựa trên giá trị $Se[1..0] \in \{00, 01, 10\}$

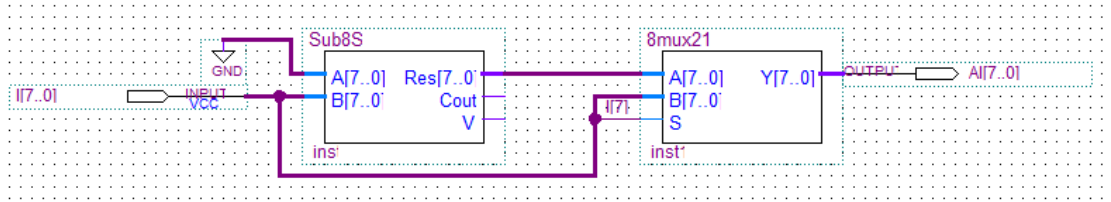
b. Khối 8mux51



Chọn giá trị đầu ra dựa trên giá trị $Se[2..0] \in \{000, 001, 010, 011, 100\}$

c. Khối 8mux21

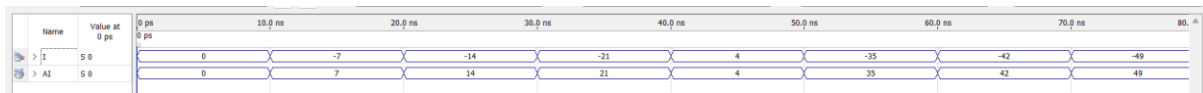
2.3. Thiết kế khối AVU



- Mạch có chức năng: tìm ra giá trị dương (trị tuyệt đối) của giá trị đầu vào $I[7..0]$

- o Với $I[7]$ (sign bit) = 1 $\rightarrow I[7..0]$ là giá trị âm $\rightarrow$ Output $A[7..0] = 0 - I[7..0]$
- o Với $I[7] = 0 \rightarrow I[7..0]$ là giá trị dương $\rightarrow$ Output $A[7..0] = I[7..0]$

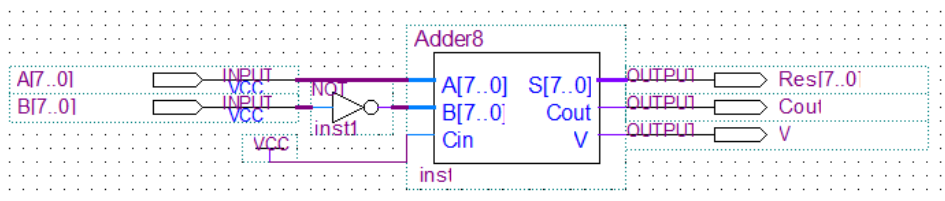
- Waveform:



- o 0 – 10ns: $|0| = 0$
- o 10 – 40ns: Chuyển các giá trị âm thành giá trị dương
- o 40 – 50ns: Giữ nguyên giá trị dương
- o 50 – 80ns: Chuyển các giá trị âm thành giá trị dương

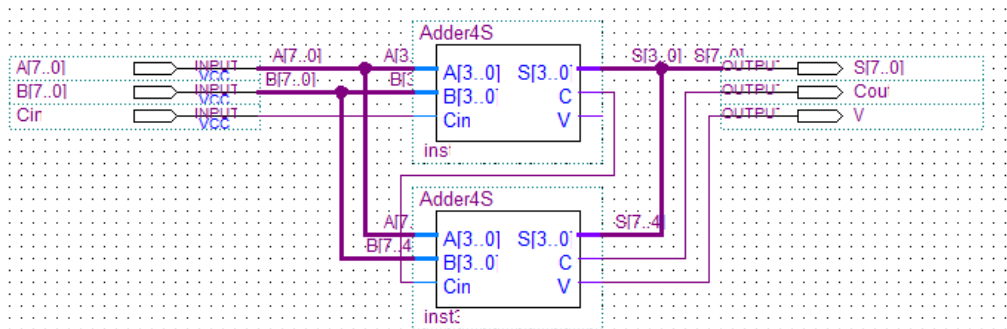
- Các khối trong AVU

- o Khối Sub8S: tính hiệu của 2 giá trị 8 bit có dấu

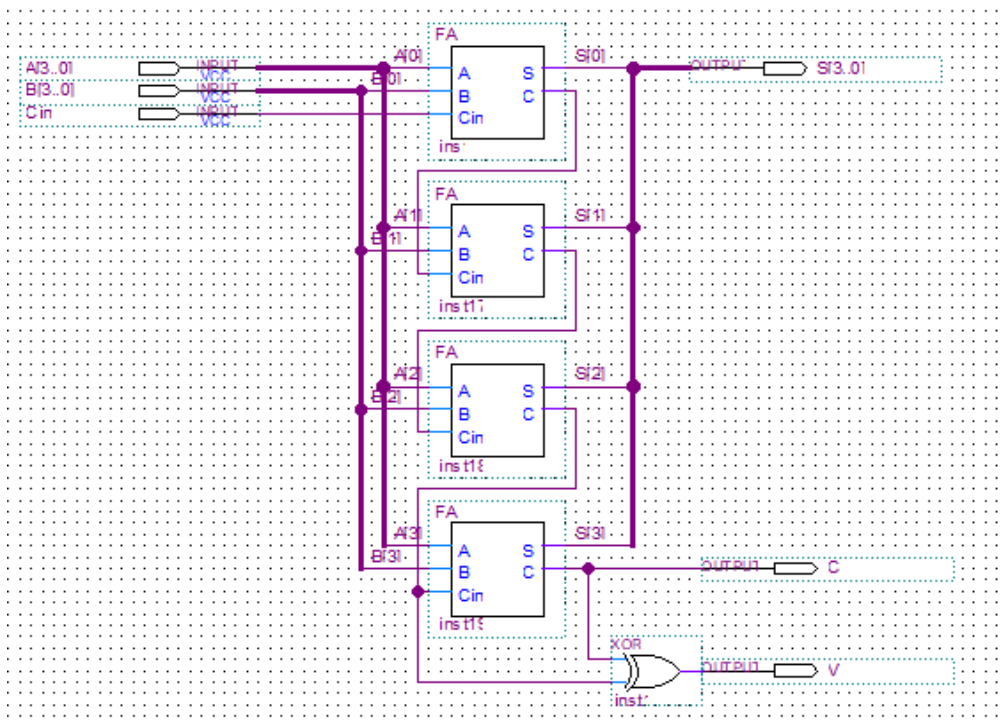


- $A - B = A + B' + 1$

- o Output: $Res[7..0]$ là hiệu của $A - B$
- o (3) Khối Adder8: tính tổng của 2 giá trị 8 bit có dấu từ 2 khối tính tổng 2 giá trị 4 bit có dấu



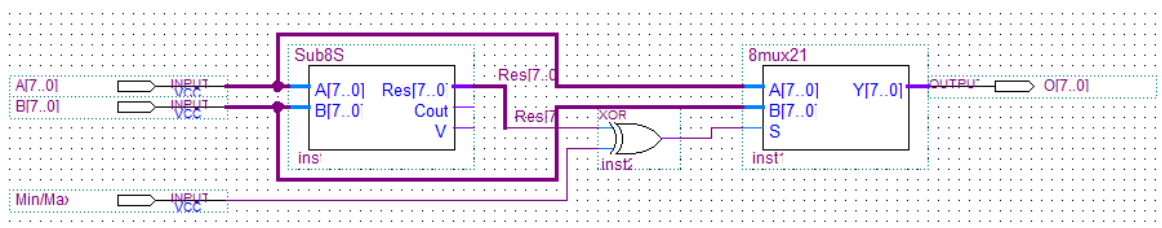
Khối Adder4S: tính tổng 2 giá trị 4 bit có dấu



Khối 8mux21

2.4. Thiết kế khối MinUnit và MaxUnit

a) Thiết kế khối Min/MaxUnit

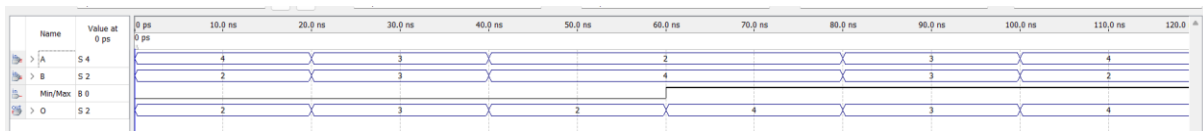


- Tính hiệu của $A - B$, sau đó lấy sign bit để so sánh với tín hiệu Min/Max để chọn ra giá trị Min hoặc Max
 - o $\text{Min/Max} = 0 \rightarrow$ tìm Min

- $A > B \rightarrow$ sign bit của hiệu là 0 XOR 0 = Min/Max $\rightarrow$ 0 $\rightarrow$ chọn B là Output
- $A < B \rightarrow$ sign bit của hiệu là 1 XOR 0 = Min/Max $\rightarrow$ 1 $\rightarrow$ chọn A là Output

○ Min/Max = 1 $\rightarrow$ tìm Max (hoạt động ngược lại với Min)

- Waveform:

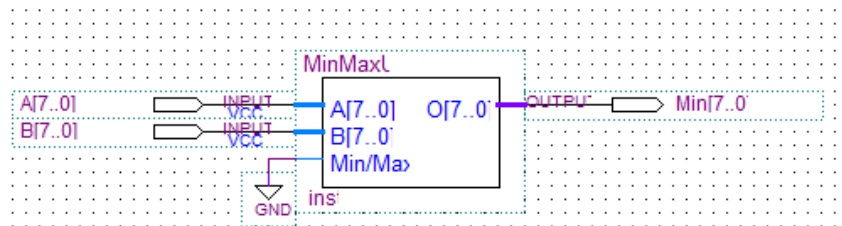


- 0 – 60ns: Min/Max = 0 $\rightarrow$ O = Min(A,B)
- 60 – 120ns: Min/Max = 1 $\rightarrow$ O = Max(A,B)

- Các khối có trong Min/MaxUnit:

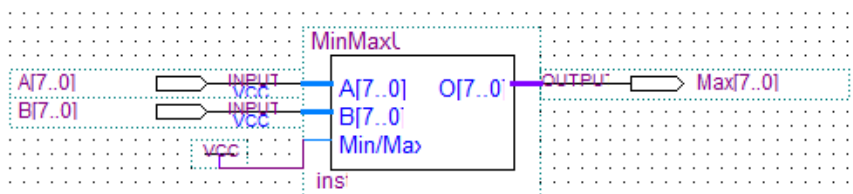
- Sub8S
- 8mux21

b) Thiết kế khối MinUnit



- Đóng gói MinMaxUnit và cho tín hiệu Min/Max = 0 để tìm Min

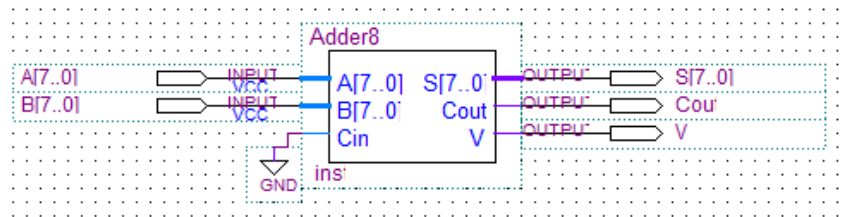
c) Thiết kế khối MaxUnit



- Đóng gói MinMaxUnit và cho tín hiệu Min/Max = 1 để tìm Max

2.5. Thiết kế khối Cộng & Trừ 8 bit có dấu

a) Thiết kế khối Adder8S



Mạch có chức năng: tính tổng của 2 giá trị 8 bit có dấu

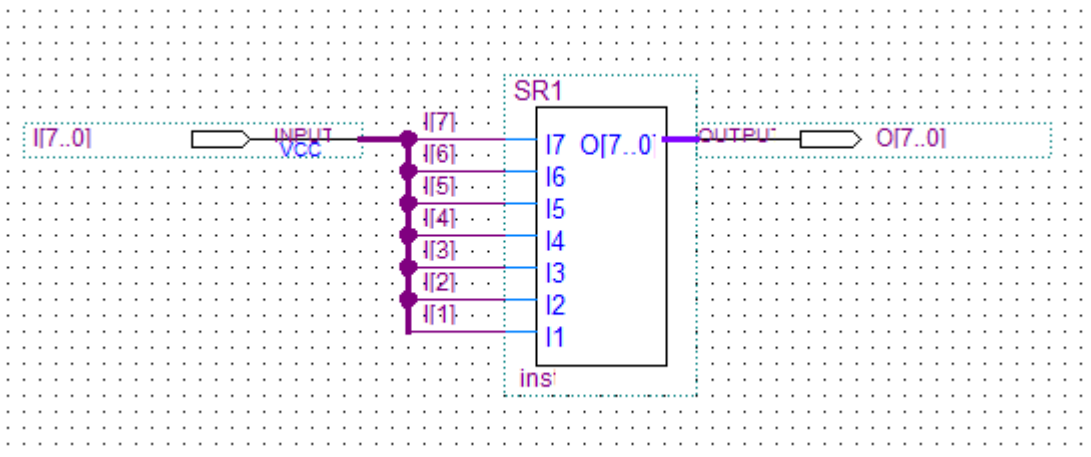
Output: S[7..0] là tổng của 2 giá trị 8 bit

b) Thiết kế khối Sub8S

Đã mô tả thiết kế như trên

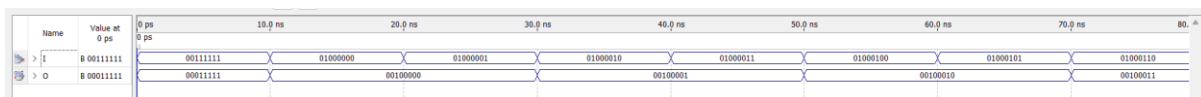
2.6. Thiết kế khối dịch phải 1 bit & 3 bit

a) Thiết kế khối 8ShiftRight1



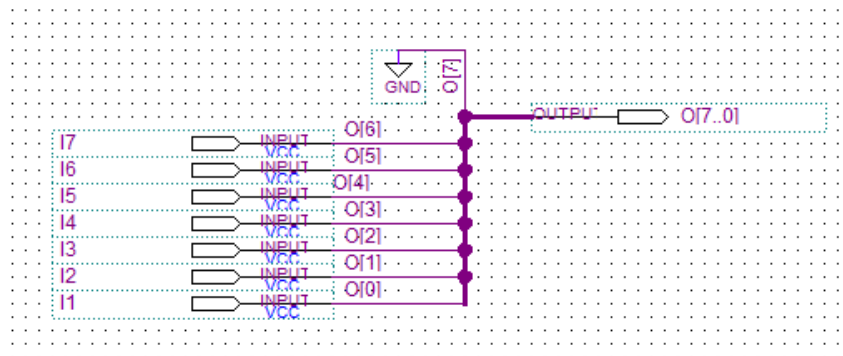
Mạch có chức năng: dịch phải 1 bit giá trị 8 bit

Waveform:

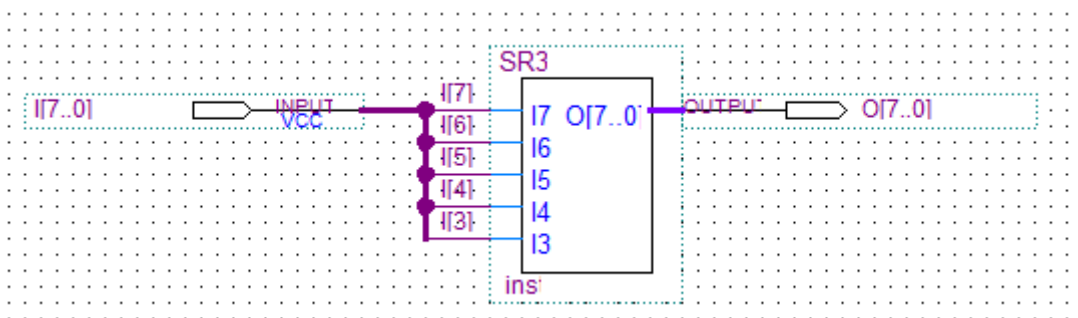


O là kết quả dịch phải 1 bit của I

Các khối có trong mạch 8ShiftRigh1: Khối SR1

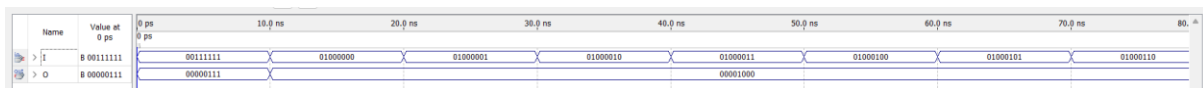


b) Thiết kế khối 8ShiftRight3



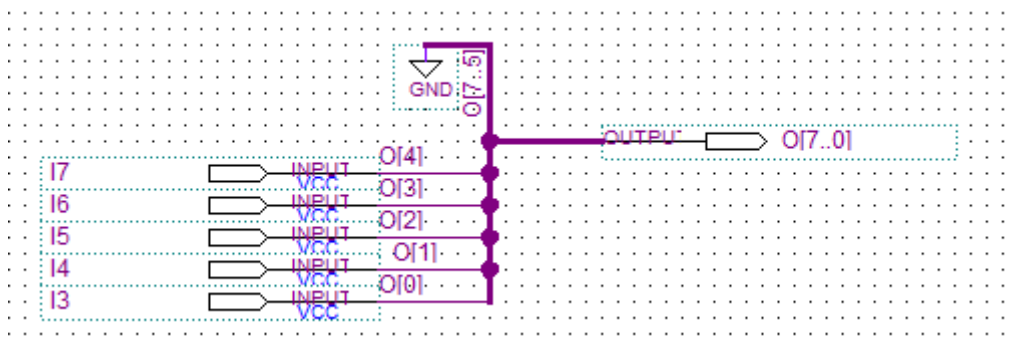
Với chức năng tương tự như 8SR1, nhưng thay vì dịch 1 bit thì dịch 3 bit

Waveform:

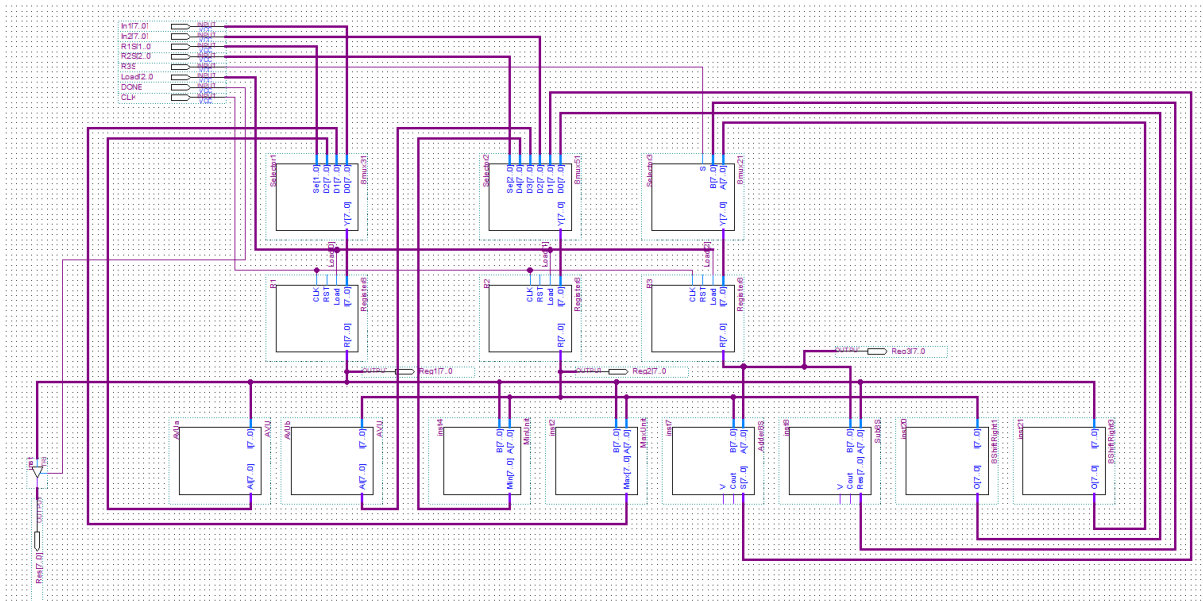


O là giá trị dịch phải 3 bit của I

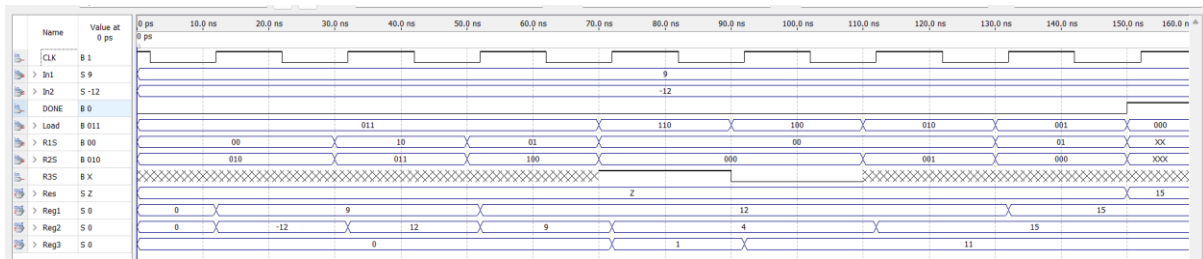
Các khối có trong 8ShiftRight3: khối SR3



2.7. Khối datapath SRA8



- Waveform:



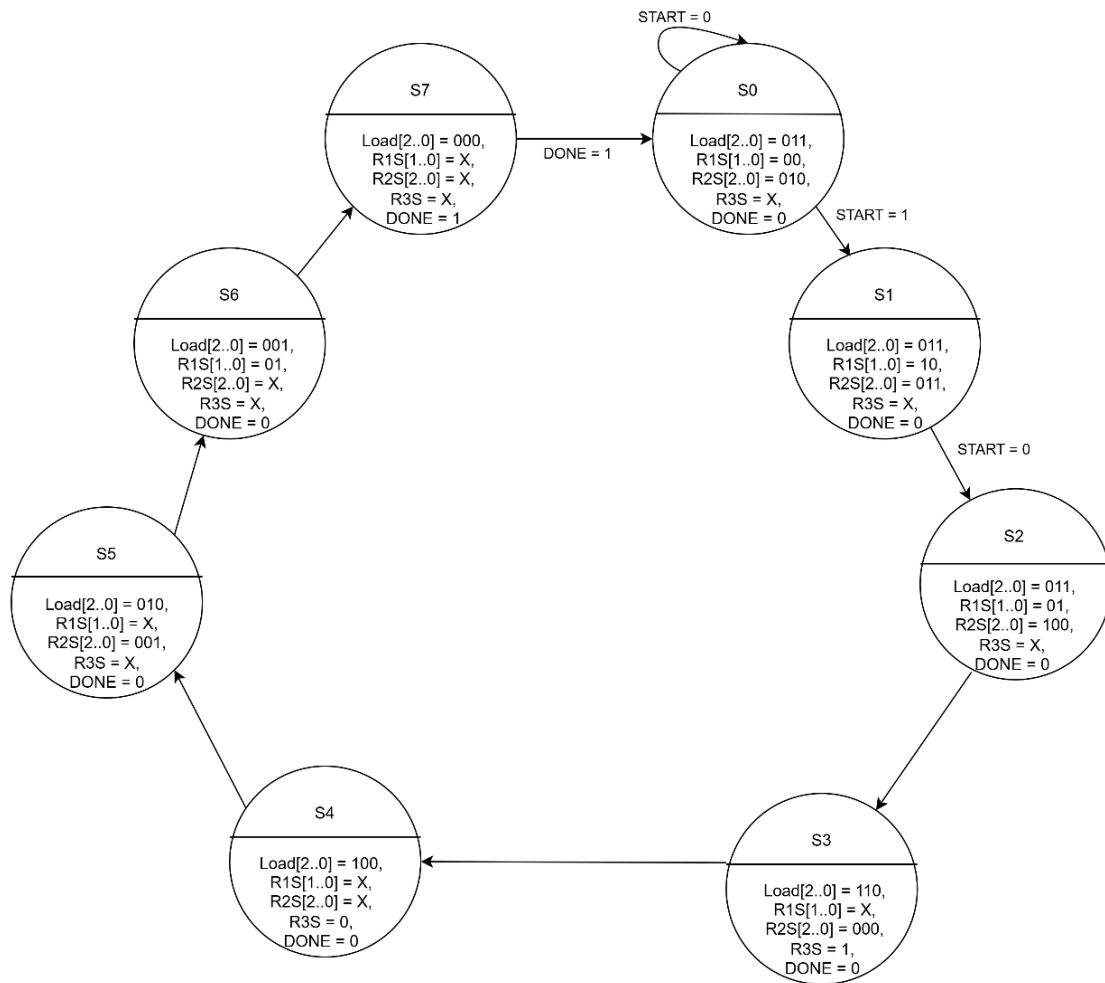
Khoảng thời gian	Chức năng	Register đọc / ghi	Output
0 – 32ns	$a = \text{In1} = 9$ $b = \text{In2} = -12$	0 / R1 & R2	Z
32 – 52ns	$T1 = a = 9$ $T2 = b = 12$	R1 & R2 / R1 & R2	
52 – 72ns	$X = \max(t1, t2) = 12$ $Y = \min(t1, t2) = 9$	R1 & R2 / R1 & R2	
72 – 92ns	$T3 = x \gg 3 = 1$ $T4 = y \gg 1 = 4$	R1 & R2 / R2 & R3	
92 – 112ns	$T5 = x - t3 = 11$	R1 & R3 / R3	
112 – 132ns	$T6 = t4 + t5 = 15$	R2 & R3 / R2	
132 – 152ns	$T7 = \max(t6, x) = 15$	R2 & R1 / R1	15
152 – 160ns	Output = $t7 = 15$	R1 / 0	

3. Thiết kế mạch Controller

3.1. Mã hóa trạng thái:

- S0 - 000: $a = \text{In1}$, $b = \text{In2}$; chờ START
- S1 - 001: $t1 = |a|$, $t2 = |b|$
- S2 - 011: $x = \max(t1, t2)$, $y = \min(t1, t2)$
- S3 - 010: $t3 = x \gg 3$, $t4 = y \gg 1$
- S4 - 110: $t5 = x - t3$
- S5 - 111: $t6 = t4 + t5$
- S6 - 101: $t7 = \max(t6, x)$
- S7 - 100: $\text{Output} = t7$
- Input:
 - START
 - DONE
- Output: điều khiển các tín hiệu đầu vào của Datapath
 - Load[2..0]
 - R1S[1..0]
 - R2S[2..0]
 - R3S
 - DONE

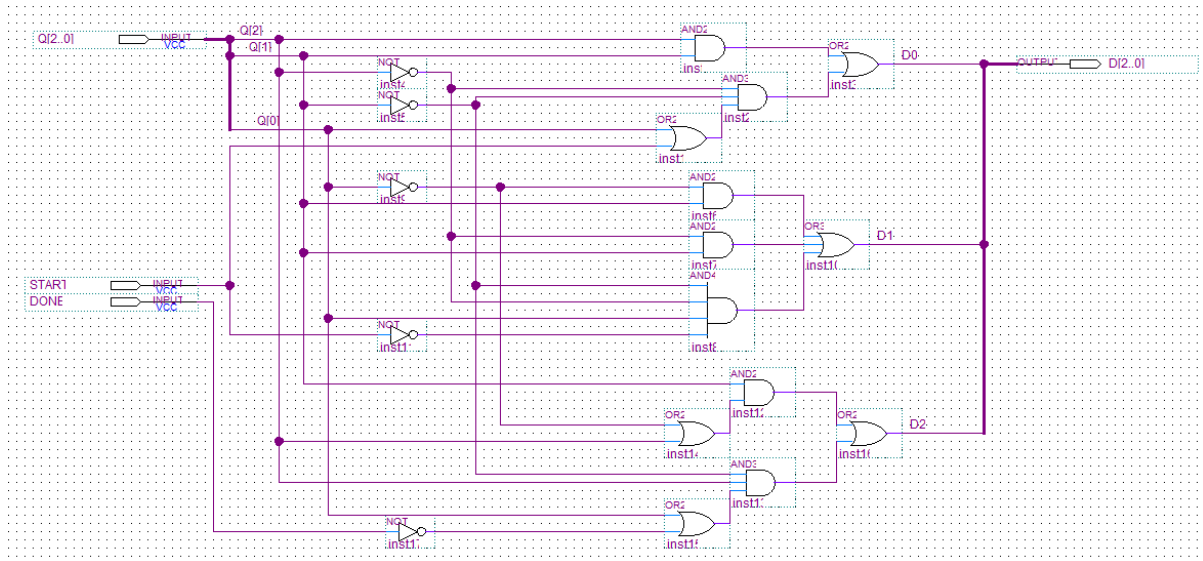
3.2. Sơ đồ chuyển trạng thái:



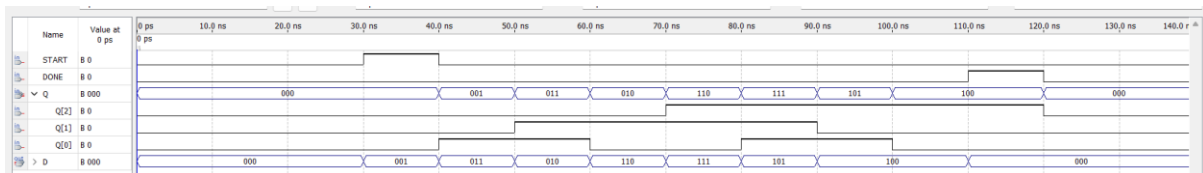
3.3. Thiết kế khối NextStateLogic

- Input cho các DFFs:

- $D0 = Q2.Q1 + Q2'.Q1'.(Q0 + START)$
- $D1 = Q1.Q0' + Q2'.Q1 + Q2'.Q1'.Q0.START'$
- $D2 = Q1.(Q2 + Q0') + Q2.Q1'.(Q0 + DONE')$

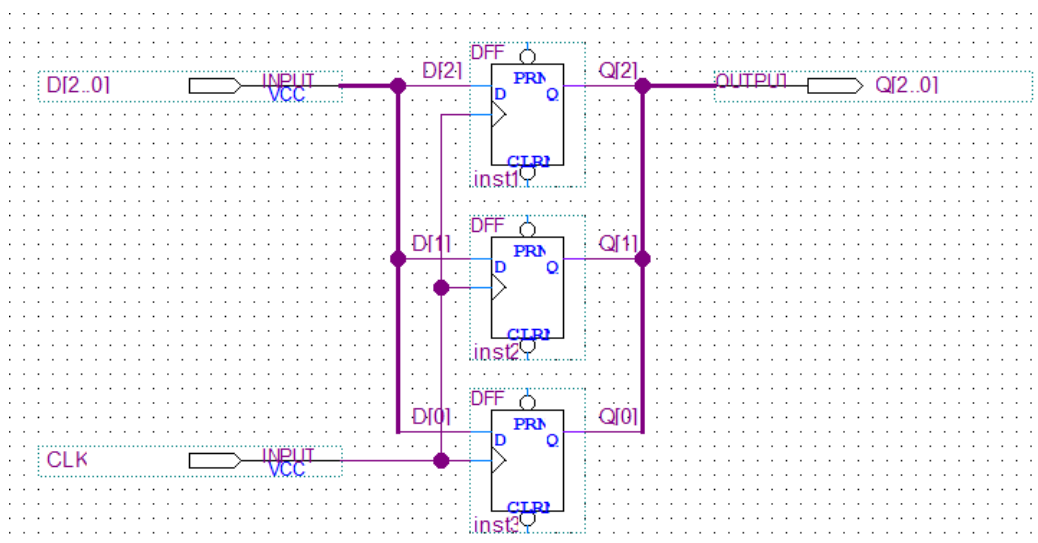


- Waveform:

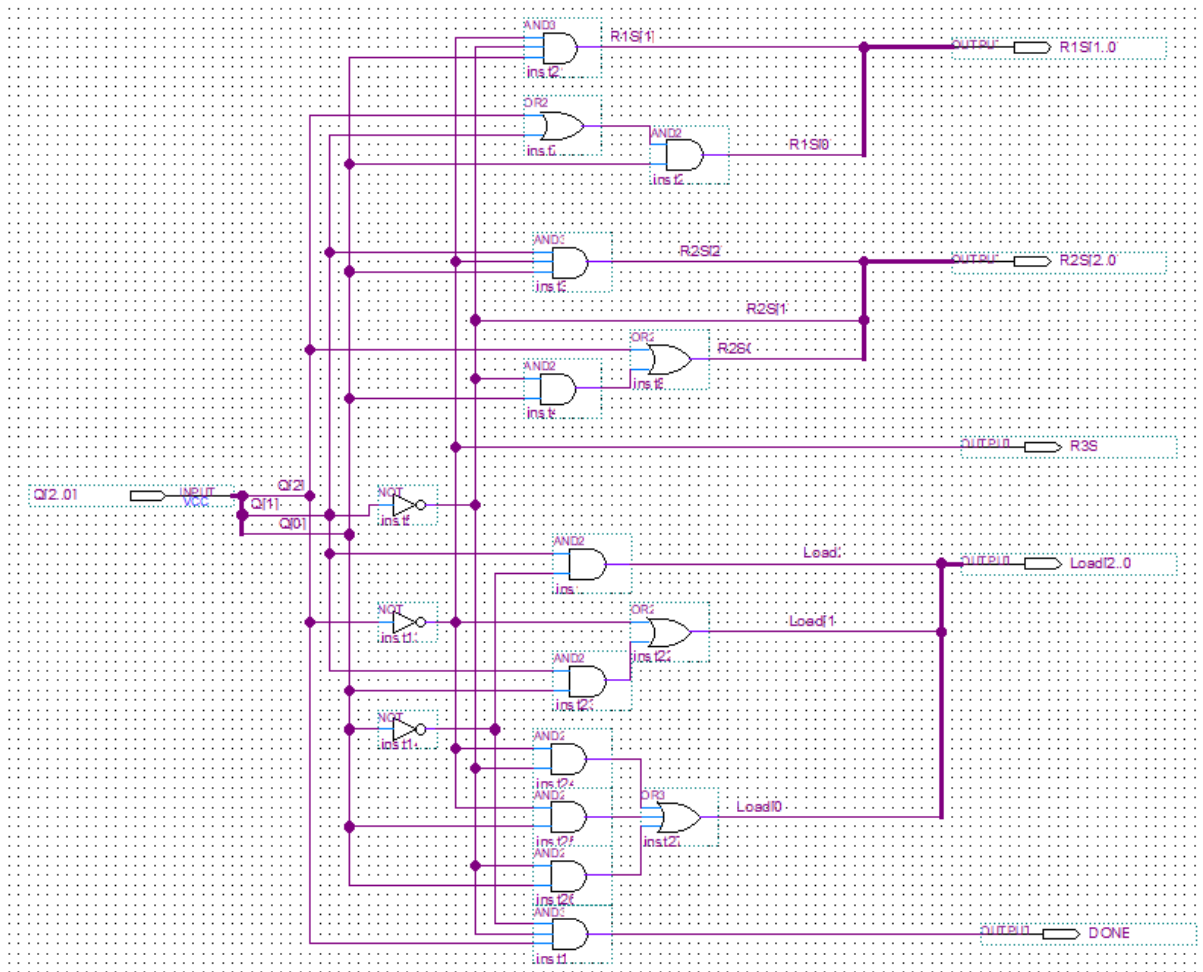


- 0 – 30ns: S0 chờ START
- 30ns: START = 1: S0 → S1
- 30 – 100ns: S1 → S2 → S3 → S4 → S5 → S6 → S7
- 100 – 110ns: DONE = 0: S7 → S7
- 110 – 120ns: DONE = 1: S7 → S0

3.4. Thiết kế khối DFFs:



3.5. Thiết kế khối OutputLogic:



- Biểu thức logic của các tín hiệu điều khiển

○ R1S[1..0]:

- $R1S[1] = Q2' \cdot Q1' \cdot Q0$
- $R1S[0] = Q0 \cdot (Q2 + Q1)$

○ R2S[2..0]:

- $R2S[2] = Q2' \cdot Q1 \cdot Q0$
- $R2S[1] = Q1'$
- $R2S[0] = Q2 + Q1' \cdot Q0$

○ $R3S = Q2'$

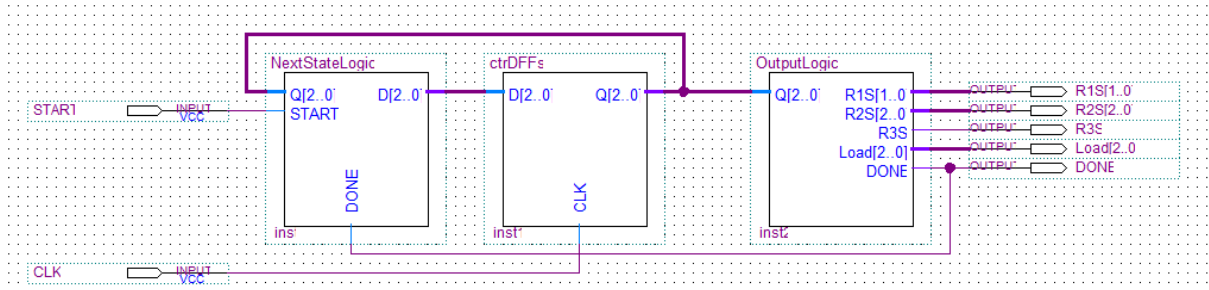
○ Load[2..0]:

- $Load[2] = Q1 \cdot Q0'$
- $Load[1] = Q2' + Q1 \cdot Q0$

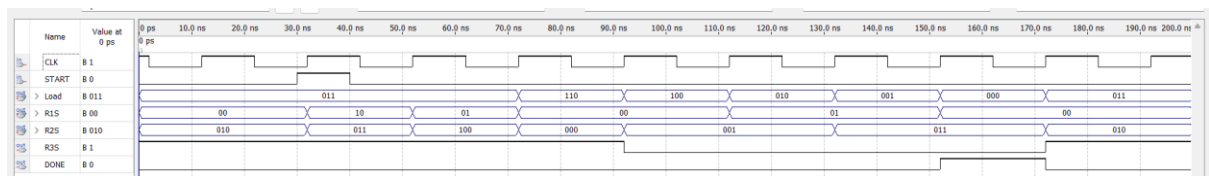
- $\text{Load}[0] = Q2'.Q1' + Q2'.Q0 + Q1'.Q0$

- $\text{DONE} = Q2.Q1'.Q0'$

3.6. Thiết kế mạch Controller

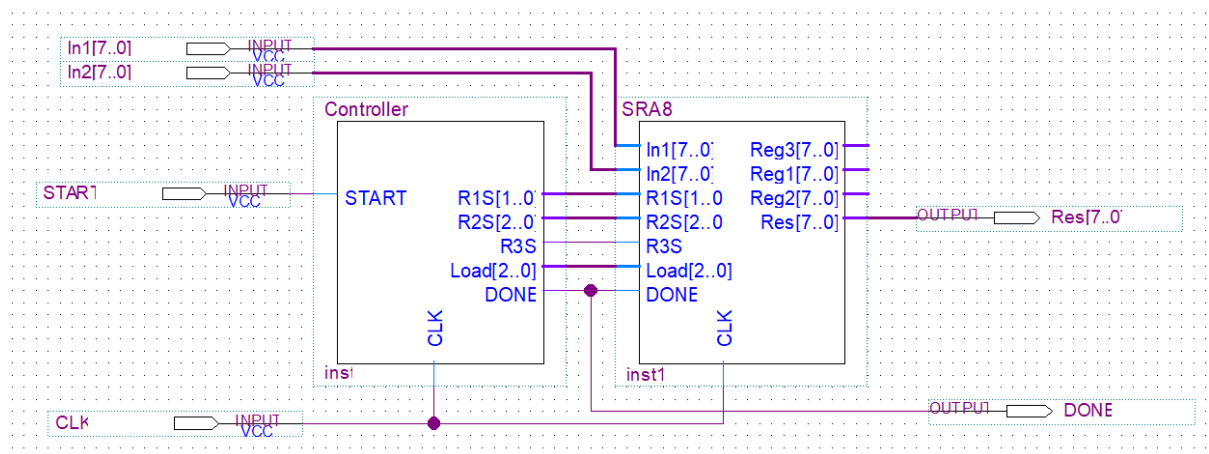


- Waveform:

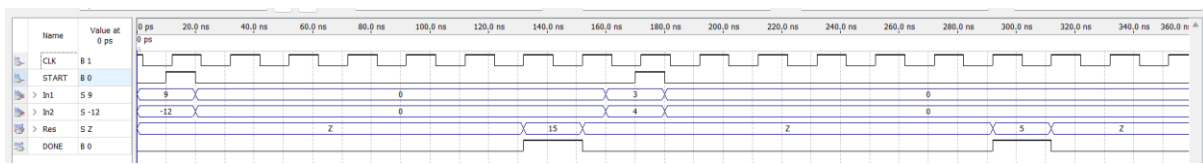


- 0 – 32ns: S0 & START = 0 → giữ S0: truyền tín hiệu nạp Input cho Datapath
- 32ns: START = 1: S0 → S1
- 32 – 172ns: S1 → S2 → S3 → S4 → S5 → S6 → S7: truyền lần lượt tín hiệu điều khiển cho Datapath để thực hiện chức năng đúng theo từng trạng thái
- 172ns: DONE = 1: S7 → S0: cờ báo hoàn tất, quay về trạng thái đầu

4. Thiết kế mạch tổng



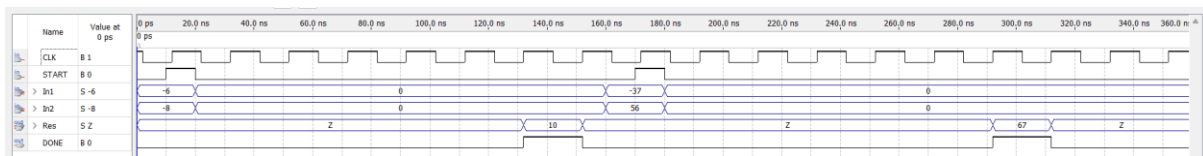
5. Waveform



○ Thực hiện tính toán $\sqrt{9^2 + (-12)^2} = 15$ và $\sqrt{3^2 + 4^2} = 5$

⇒ Kết quả đúng và cờ DONE bật khi hoàn tất tính toán

- Thực hiện tính toán với cặp toán hạng khác:



○ Thực hiện tính toán $\sqrt{(-6)^2 + (-8)^2} = 10$ và

$\sqrt{(-37)^2 + 56^2} \approx \max((0.875 \times 56 + 0.5 \times 37), 56) \approx 67$

⇒ Kết quả đúng và cờ DONE bật khi hoàn tất tính toán

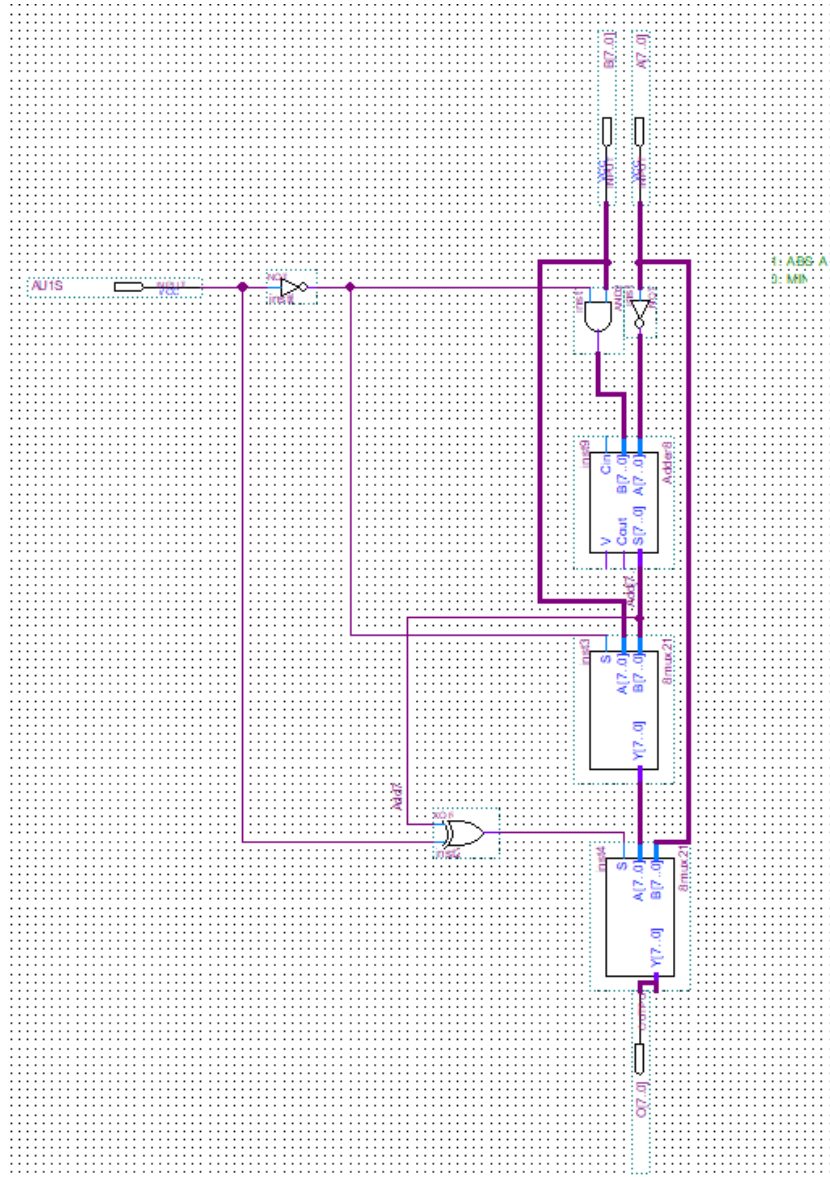
J. Thiết kế sử dụng Function-unit Sharing

1. Cơ sở lý thuyết

- Sử dụng các khối, register, shift right 1 bit & 3 bit đã thiết kế ở Register Sharing
- Thiết kế AU1 có chức năng tính trị tuyệt đối và min (Abs/MinUnit)
 - Input: R1 & R2
 - Output:
 - ABS cho R1
 - Min cho R2
- Thiết kế AU2 có chức năng tính trị tuyệt đối, max, tính tổng & hiệu (Abs/Max/Adder/SubUnit)
 - Input: (R1 hoặc R3) & R2
 - Output:
 - ABS, Adder, Sub cho R2
 - Max cho R1

2. Thiết kế Datapath

1.1. Thiết kế AU1

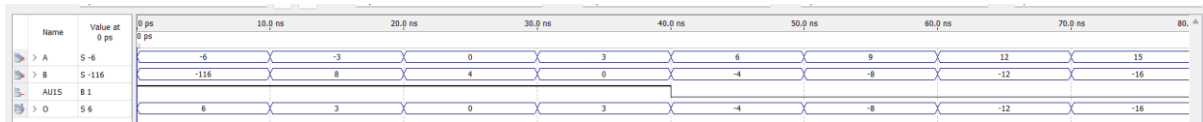


- Thiết kế dựa trên mạch 3 chức năng: Abs/Min/Max

Chức năng	Abs/Min/Max	Abs/Min
Tín hiệu điều khiển	C[1..0]	AU1S
Chọn chức năng	C[1..0] = - 01: Abs - 10: Min - 11: Max	AU1S = - 0: Min - 1: Abx

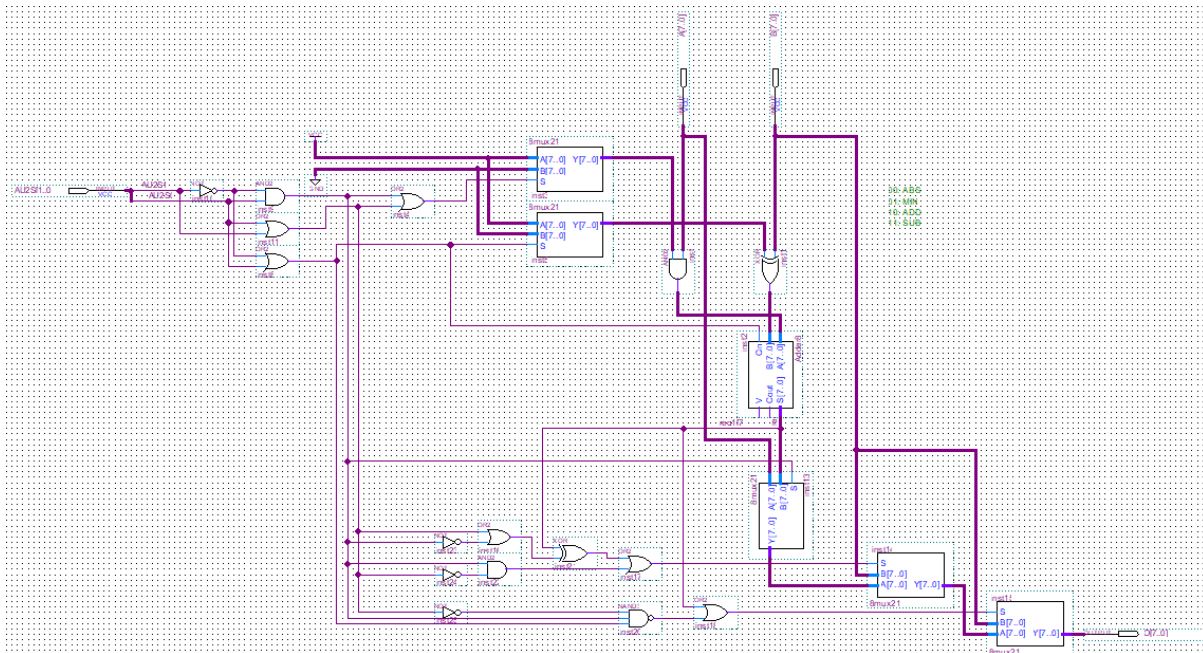
⇒ C1 = AU1S'; C0 = AU1S

- Waveform:



- 0 – 40ns: AU1S = 1 $\rightarrow$ O = |A|
- 40 – 80ns: AU1S = 0 $\rightarrow$ O = min(A, B)

1.2.Thiết kế AU2

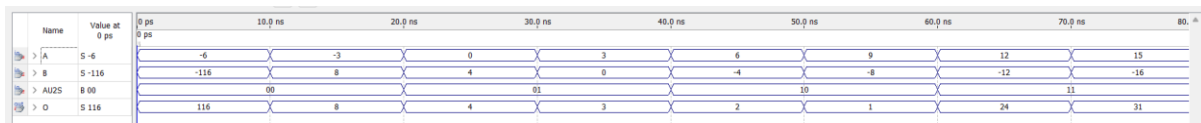


- Thiết kế dựa trên mạch 5 chức năng: Add/Abs/Sub/Min/Max

Chức năng	Add/Abs/Sub/Min/Max	Abs/Max/Add/Sub
Tín hiệu điều khiển	C[2..0]	AU2S[1..0]
Chọn chức năng	C[2..0] = - 001: Add - 100: Abs - 101: Sub - 110: Min - 111: Max	AU2S[1..0] = - 00: Abs - 01: Max - 10: Add - 11: Sub

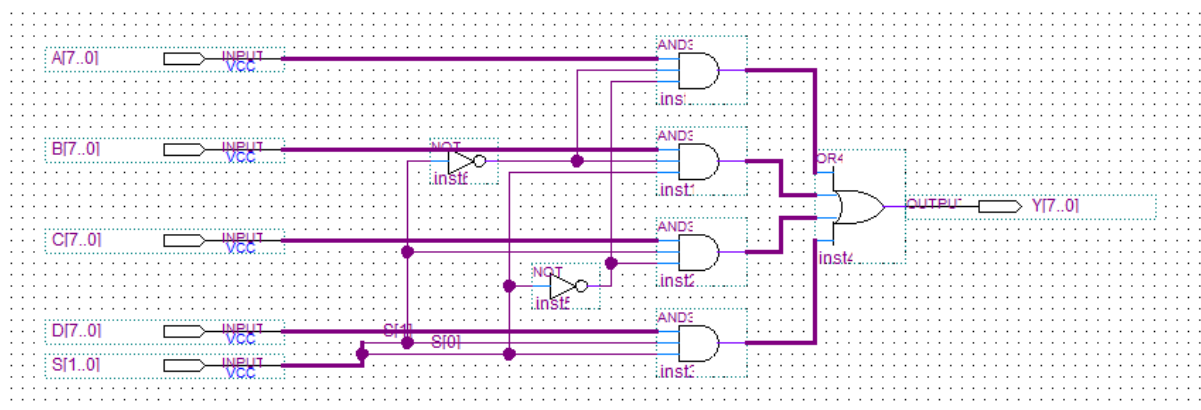
$\Rightarrow C2 = AU2S1' + AU2S0; C1 = AU2S1' \cdot AU2S0; C0 = AU2S1 + AU2S0$

- Waveform:

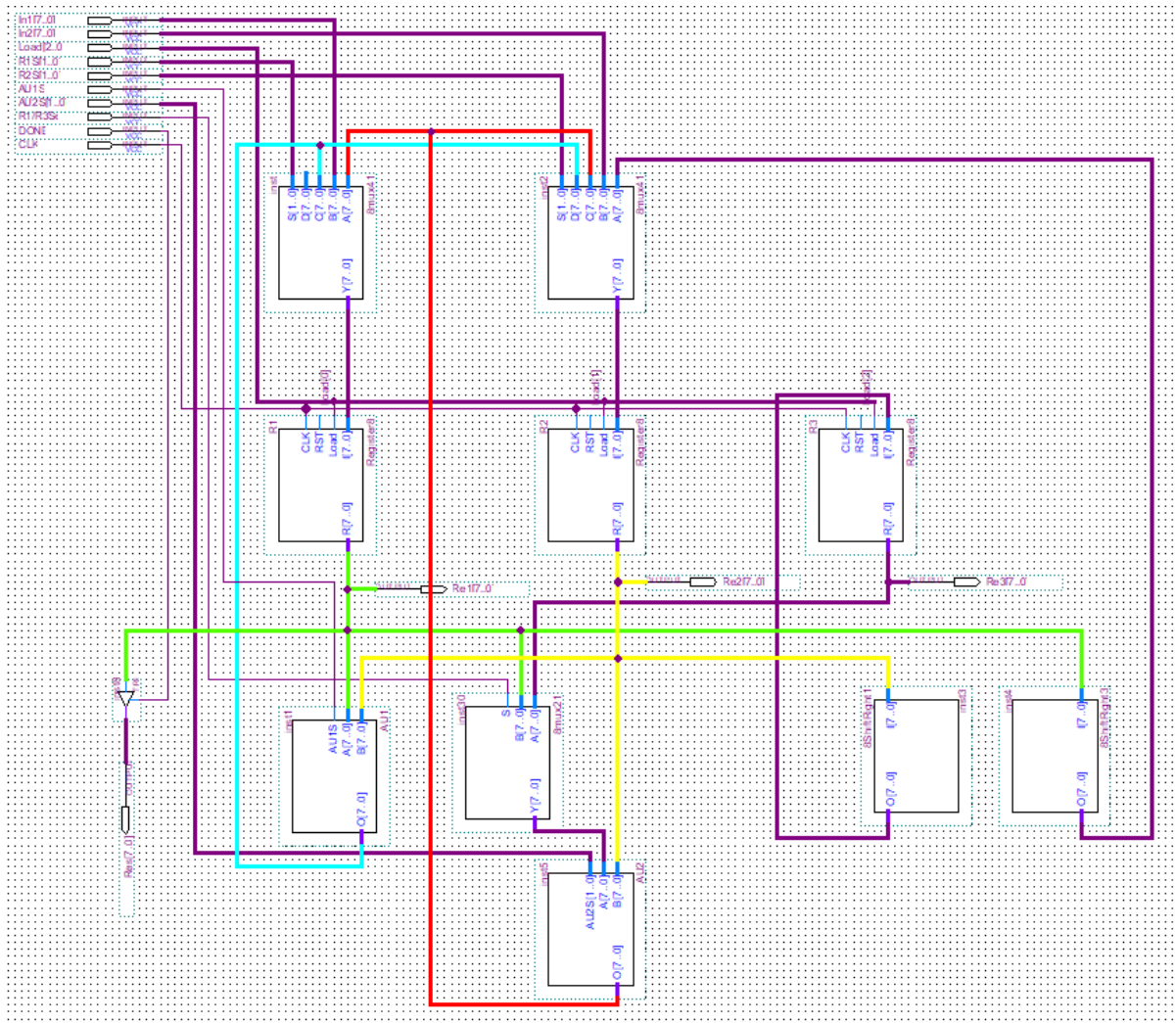


- 0 – 20ns: AU2S[1..0] = 00 → O = |B|
- 20 – 40ns: AU2S[1..0] = 01 → O = max(A, B)
- 40 – 60ns: AU2S[1..0] = 10 → O = A + B
- 60 – 80ns: AU2S[1..0] = 11 → O = A - B

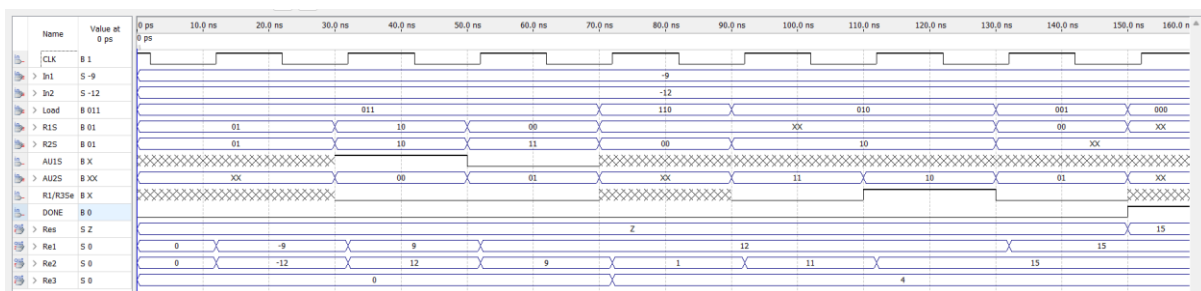
- Thiết kế 8mux41:



1.3. Thiết kế Datapath tổng quan



1.4. Waveform:



Khoảng thời gian	Chức năng	Thanh ghi đọc/ ghi	Output
0 – 32ns	a = In1 b = In2	0 / R1 & R2	Z
32 – 52ns	t1 = a t2 = b	R1(AU1) / R1 R2(AU2) / R2	

52 – 72ns	$x = \max(t1, t2)$ $y = \min(t1, t2)$	R1 & R2 (AU2) / R2 R1 & R2 (AU1) / R1	
72 – 92ns	$t3 = x \gg 3$ $t4 = y \gg 1$	R1 (SH2) / R2 R2 (SH1) / R3	
92 – 112ns	$t5 = x - t3$	R1 & R2 (AU2) / R2	
112 – 132ns	$t6 = t4 + t5$	R3 & R2 (AU2) / R2	
132 – 152ns	$t7 = \max(t6, x)$	R1 & R2 (AU2) / R1	
152 – 160ns	Output = t7	0/0	15

3. Thiết kế Controller

a. Mã hóa trạng thái:

- S0 – 000: nạp, chờ Start
- S1 – 001: tính trị tuyệt đối
- S2 – 011: tính max, min
- S3 – 010: dịch phải
- S4 – 110: tính hiệu
- S5 – 111: tính tổng
- S6 – 101: tìm max
- S7 – 100: Output = max

b. Input: START & DONE

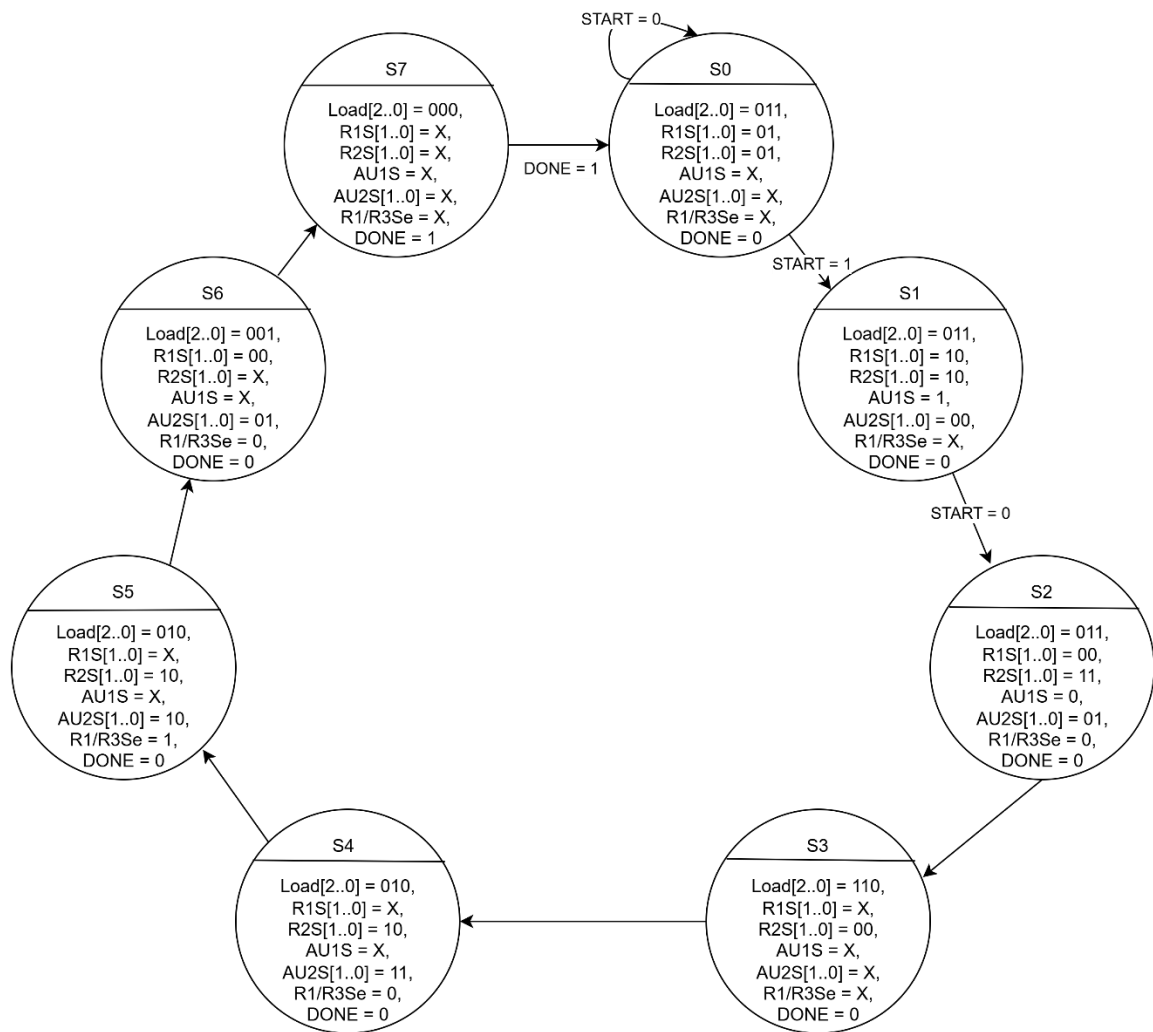
START: tín hiệu bắt đầu

DONE: tín hiệu hoàn thành

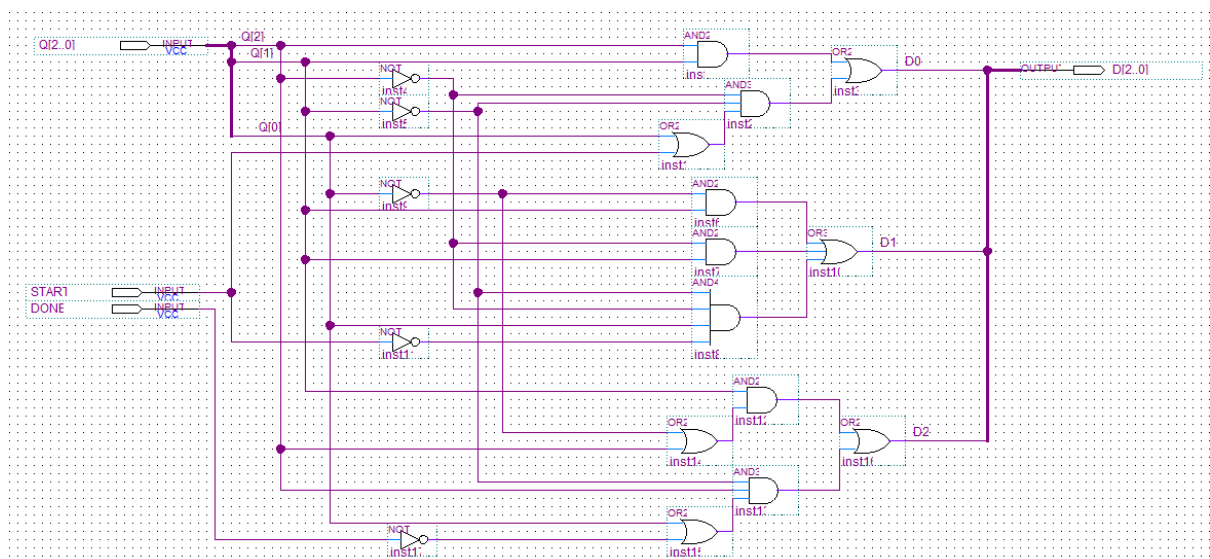
c. Output: các tín hiệu Input của Datapath

- Load[2..0]
- R1S[1..0]
- R2S[1..0]
- AU1S
- AU2S[1..0]
- R1/R3Se
- DONE

d. Sơ đồ chuyển trạng thái:

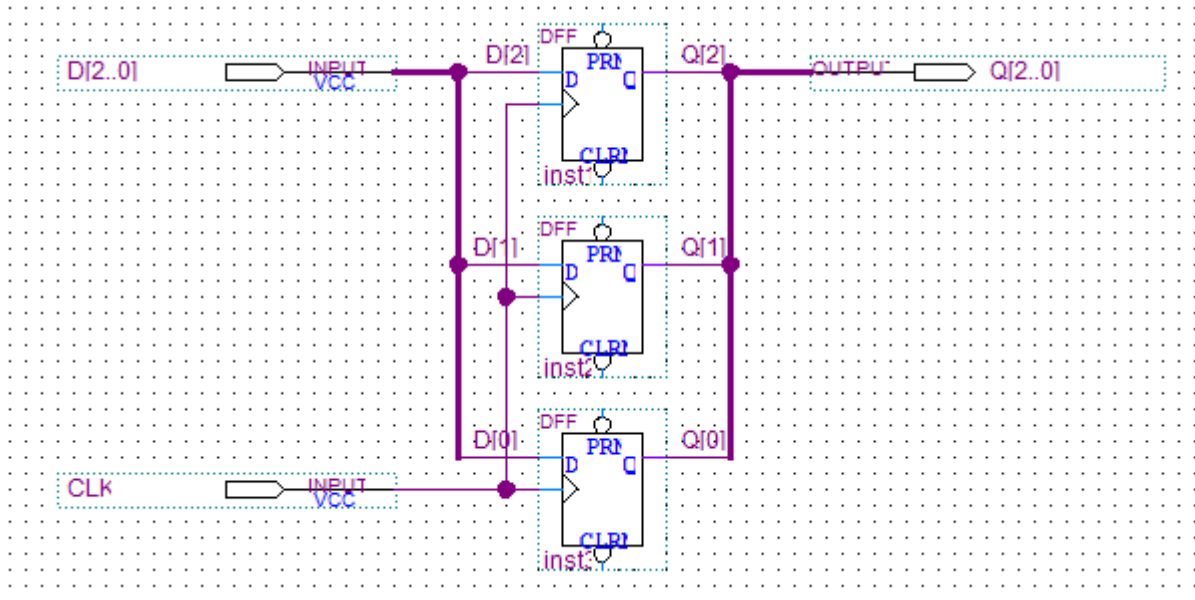


e. Thiết kế khối NextStateLogic



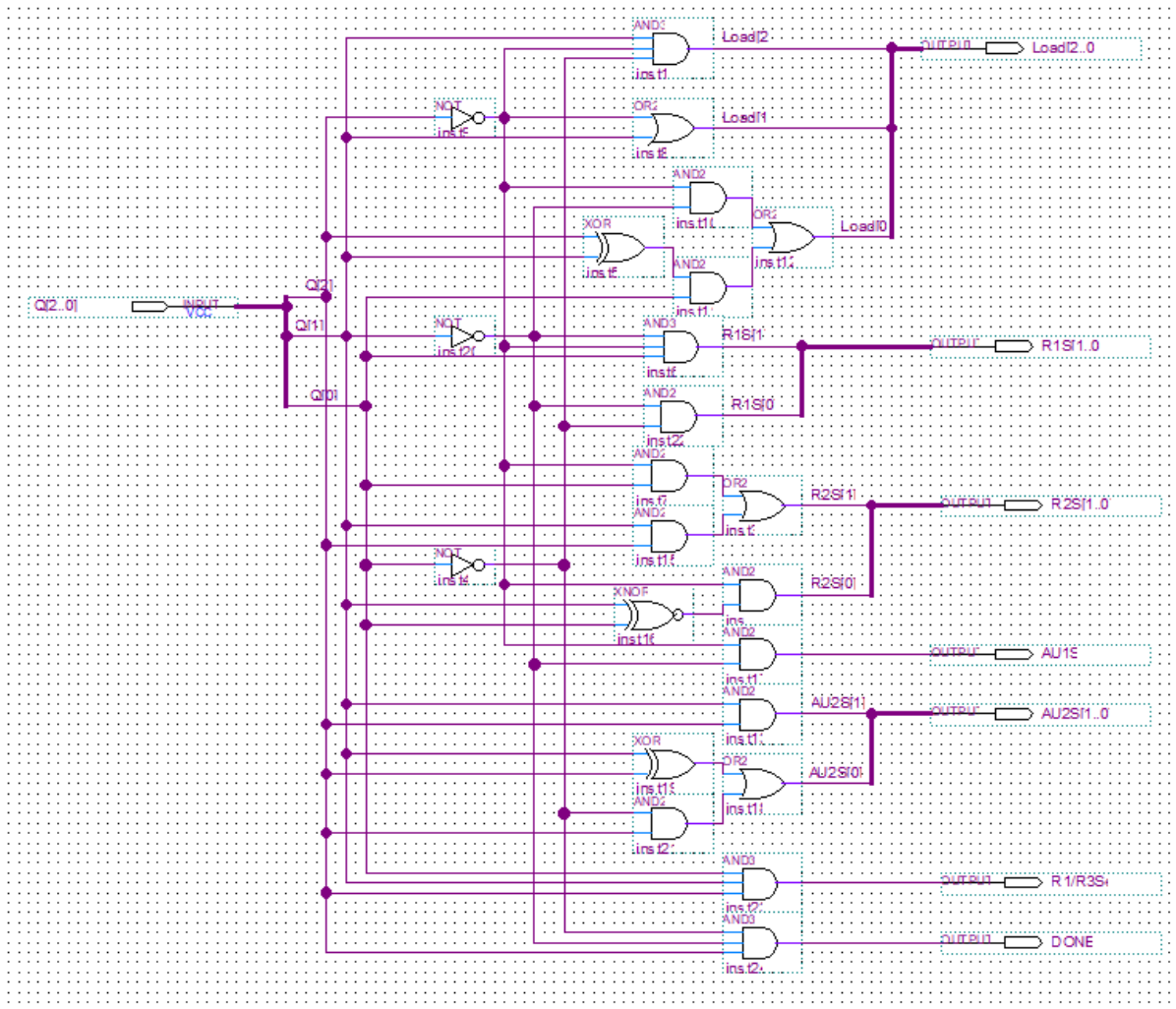
Sử dụng lại khối NextStateLogic từ phần Register Sharing vì có cùng số trạng thái

f. Thiết kế khối DFFs



Sử dụng lại khối DFFs từ phần Register Sharing vì có cùng số trạng thái

g. Thiết kế khối OutputLogic

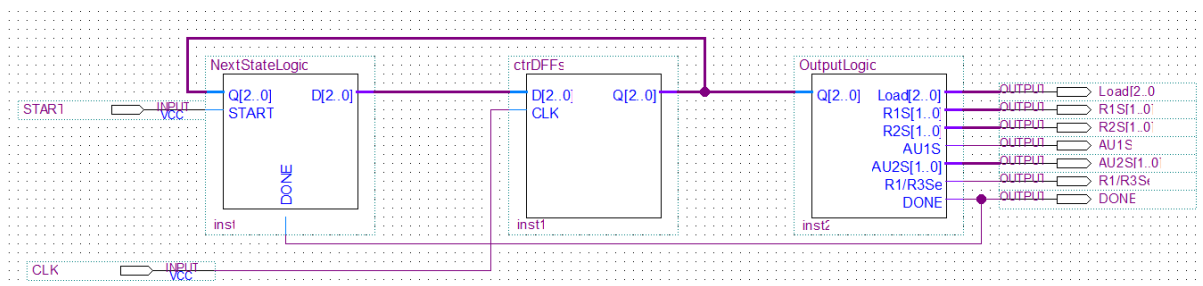


Biểu thức output của controller:

- i. Load[2..0]:
 - Load[2] = $Q2'.Q1.Q0'$
 - Load[1] = $Q2' + Q1'$
 - Load[0] = $Q2'.Q1' + Q0.(Q2 \oplus Q1)$
- ii. R1S[1..0]:
 - R1S[1] = $Q2'.Q1'.Q0$
 - R1S[0] = $Q1'.Q0'$
- iii. R2S[1..0]:
 - R2S[1] = $Q2.Q1 + Q2'.Q0$
 - R2S[0] = $Q2'.(Q1 \oplus Q0)'$

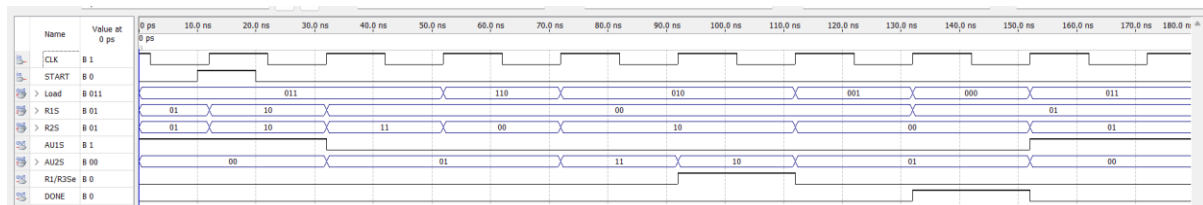
- iv. $AU1S = Q2'.Q1'$
- v. $AU2S[1..0]$:
 - $AU2S[1] = Q2.Q1$
 - $AU2S[0] = Q2 \oplus Q1 + Q2.Q0'$
- vi. $R1/R3Se = Q2.Q1.Q0$
- vii. $DONE = Q2.Q1'.Q0'$

h. Thiết kế Controller



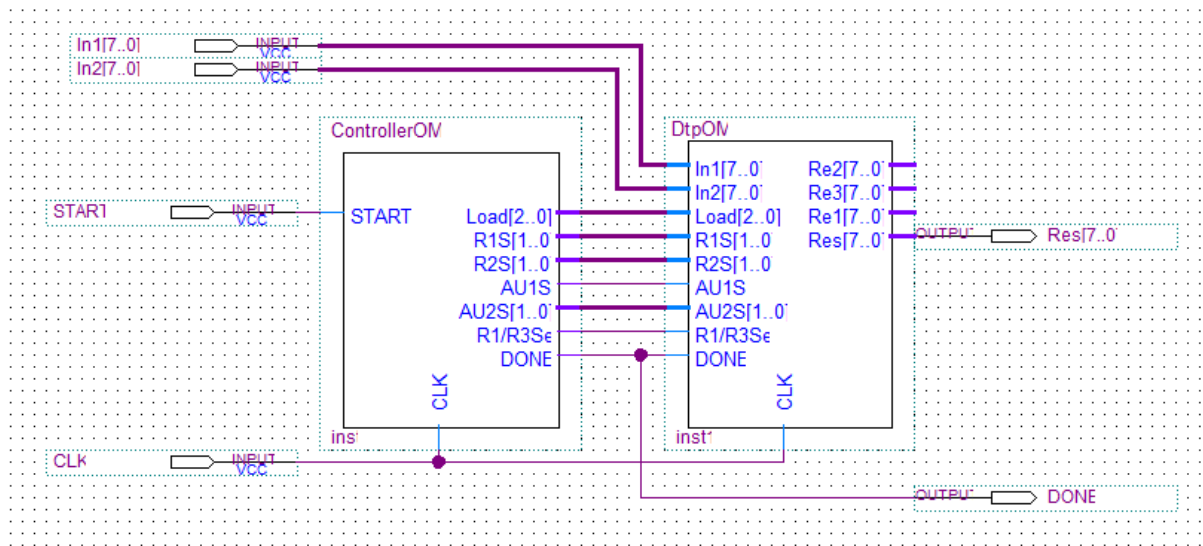
Controller được thiết kế theo Moore

i. Waveform:

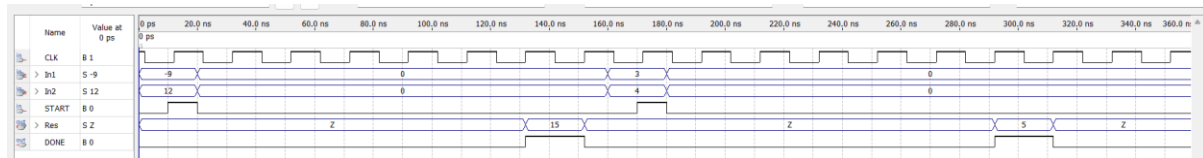


- 0 – 12ns: $S0 \& START = 0 \rightarrow$ giữ $S0$: truyền tín hiệu nạp input cho Datapath
- 12ns: $START = 1, S0 \rightarrow S1$: chuyển qua tín hiệu tính trị tuyệt đối
- 12 – 152ns: $S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6 \rightarrow S7$: lần lượt truyền tín hiệu để thực hiện tuần tự theo từng bước với từng trạng thái
- 152ns: $DONE = 1, S7 \rightarrow S0$: quay về trạng thái nạp Input

4. Thiết kế mạch theo Function-Unit Sharring



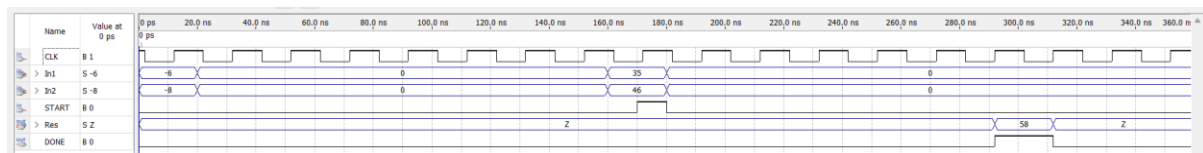
5. Waveform



Thực hiện tính toán: $\sqrt{(-9)^2 + 12^2} = 15$ và $\sqrt{3^2 + 4^2} = 5$

⇒ Kết quả đúng và cờ DONE được bật ngay khi hoàn tất tính toán

Thực hiện với cặp toán hạng khác:

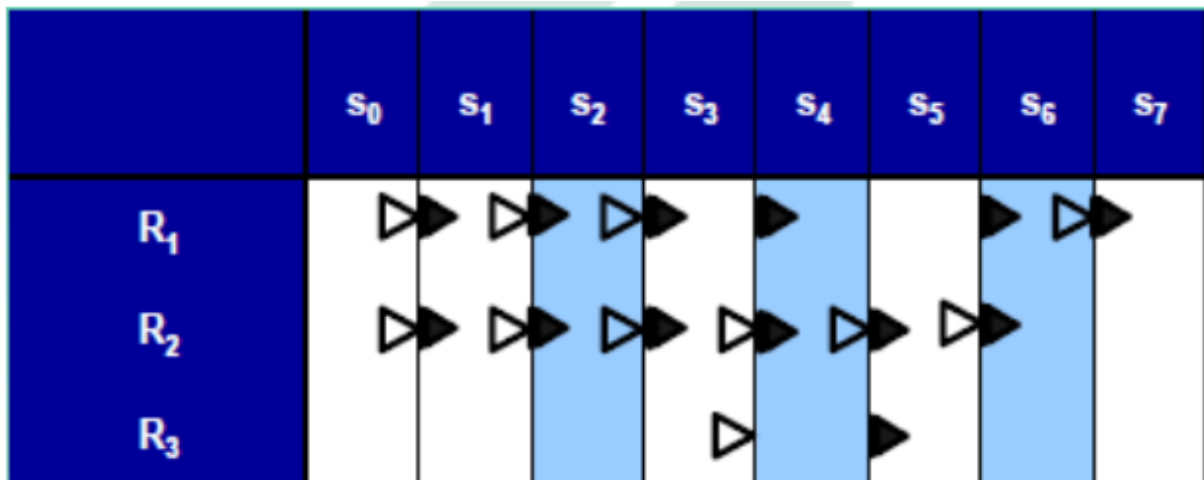


Thực hiện tính toán $\sqrt{(-6)^2 + (-8)^2} = 10$ và $\sqrt{35^2 + 46^2} \approx \max((0.875 \times 46 + 0.5 \times 35), 46) \approx 58$

⇒ Kết quả đúng và cờ DONE được bật khi hoàn tất tính toán

K. Thiết kế sử dụng Bus-sharing

1. Cơ sở lý thuyết



Có thể thấy, R_1 và R_3 không cùng thực hiện chung 1 chức năng tại bất kì thời điểm nào, vậy ta có thể cho R_1 và R_3 dùng chung 1 bus, sử dụng tri-state cho output của R_1 và R_3 để chọn input cho bus

Sử dụng lại các khối Register, Shift Right 1 & 3 từ bài trên

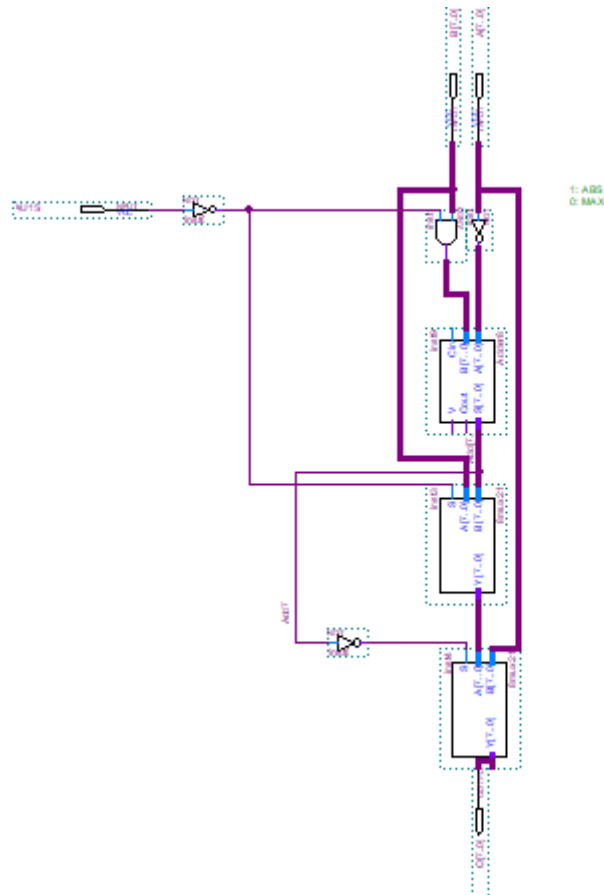
Thiết kế lại AU:

- AU1: Abs & max
- AU2: Abs & min & + & -

2. Thiết kế datapath

Các khối R_1 , R_2 , R_3 và Shift Right 1 & 3 được thiết kế như các bài trên

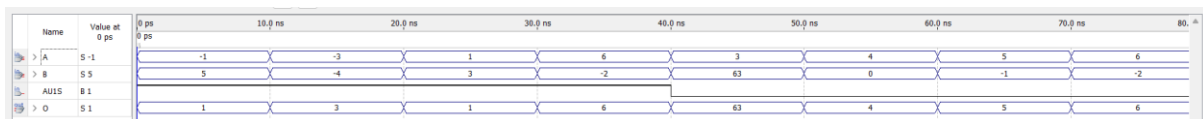
2.1. Thiết kế khối AU1



Với tín hiệu điều khiển AU1S:

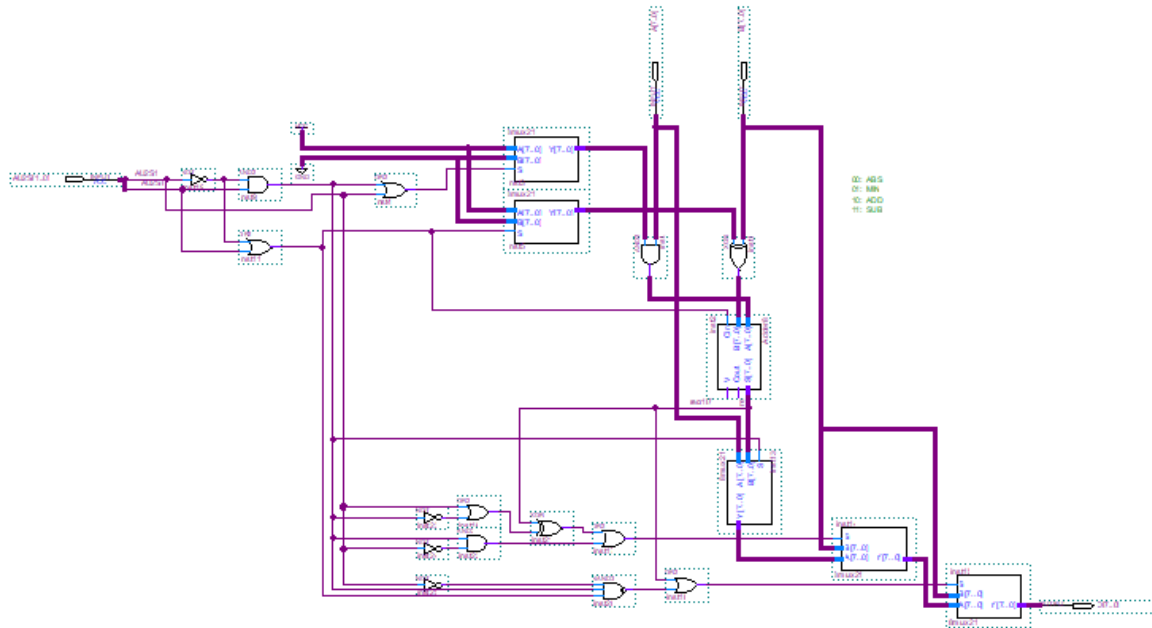
- AU1S = 0: tìm Max của 2 giá trị input
- AU1S = 1: tìm giá trị tuyệt đối (abs) của input A[7..0]

Waveform:



- 0 – 40ns: AU1S = 1: tính $O = |A|$
- 40 – 80ns: AU1S = 0: tính $O = \max(A, B)$

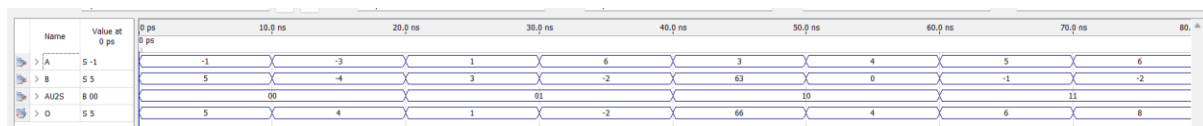
2.2. Thiết kế khối AU2



Với tín hiệu điều khiển AU2S[1..0]:

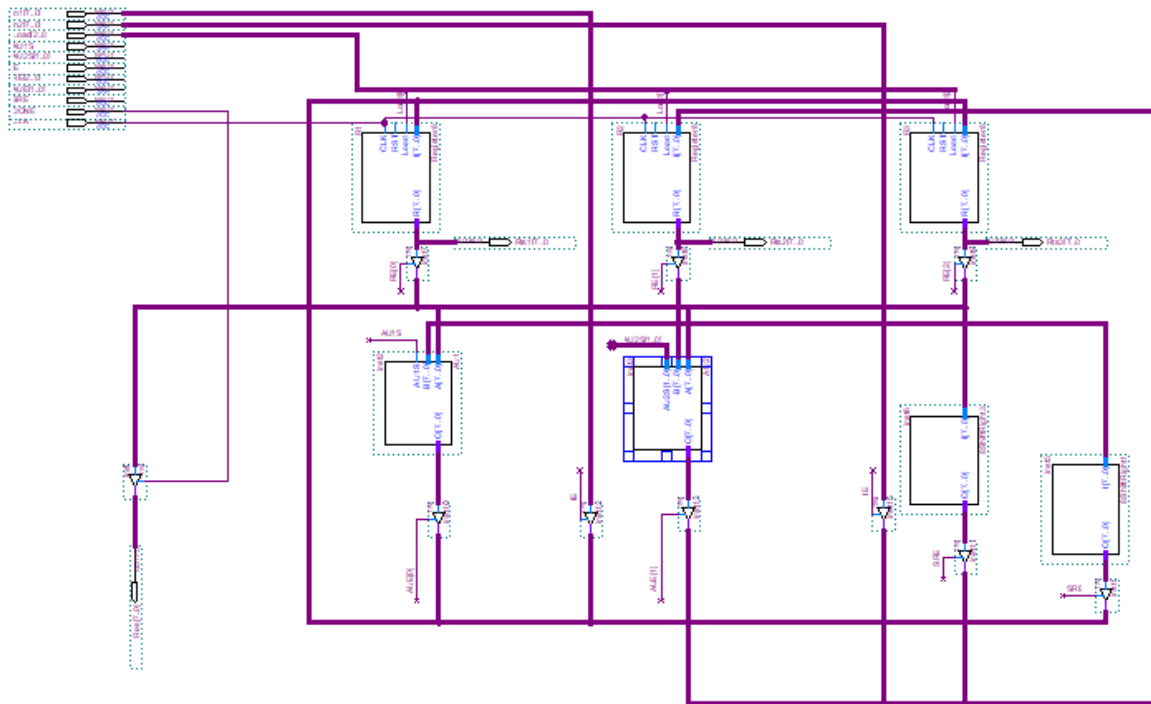
- 00: tính giá trị tuyệt đối (abs) của input B[7..0]
- 01: tính min của 2 input
- 10: tính tổng của 2 input
- 11: tính hiệu của 2 input

Waveform:

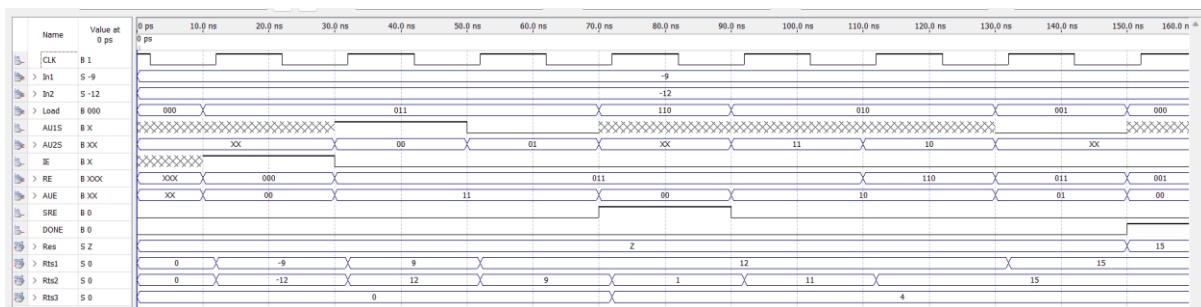


- 0 – 20ns: AU2S[1..0] = 00: tính $O = |B|$
- 20 – 40ns: AU2S[1..0] = 01: tính $O = \min(A, B)$
- 40 – 60ns: AU2S[1..0] = 10: tính $O = A + B$
- 60 – 80ns: AU2S[1..0] = 11: tính $O = A - B$

2.3. Thiết kế Datapath



Waveform:



Khoảng thời gian	Chức năng	Thanh ghi đọc / ghi	Output
12 – 32ns	A = In1 = -9 B = In2 = -12	0 / R1(A) & R2(B)	Z
32 – 52ns	T1 = a T2 = b	R1 & R2 / R1(t1) & R2(t2)	
52 – 72ns	x = max(t1, t2) y = min(t1, t2)	R1 & R2 / R1(x) & R2(y)	
72 – 92ns	T3 = x >> 3 T4 = y >> 1	R1 & R2 / R2(t3) & R3(t4)	

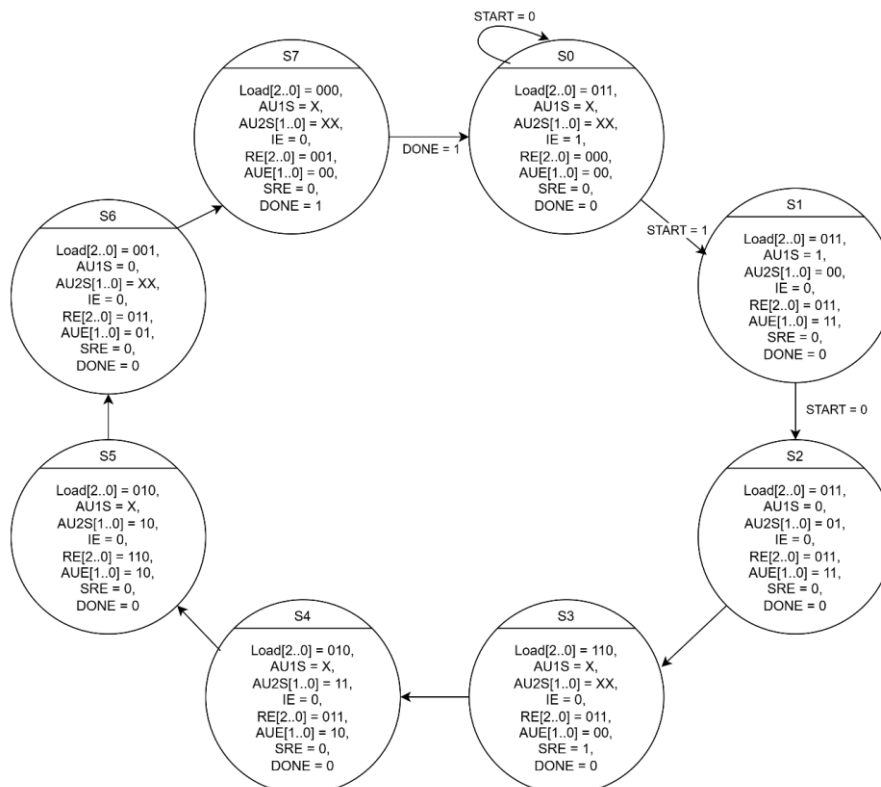
92 – 112ns	$T5 = x - t3$	R1 & R2 / R2(t5)	
112 – 132ns	$T6 = t4 + t5$	R3 & R2 / R2(t6)	
132 – 152ns	$T7 = \max(t6, x)$	R2 & R1 / R1(t7)	
152 – 160ns	Output = t7	0 / R1	15

3. Thiết kế controller

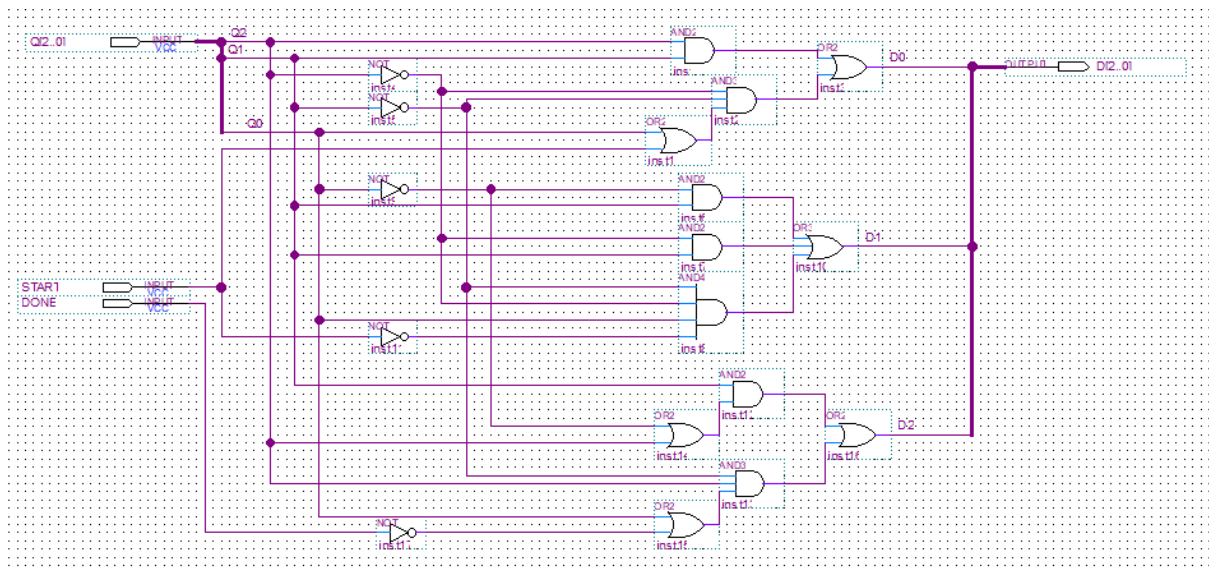
3.1. Mã hóa trạng thái:

- S0 – 000: nạp In1, In2 (a, b)
- S1 – 001: tính trị tuyệt đối (t1, t2)
- S2 – 011: tính max, min (x, y)
- S3 – 010: dịch phải 1 bit & 3 bit (t3, t4)
- S4 – 110: tính hiệu (t5)
- S5 – 111: tính tổng (t6)
- S6 – 101: tính max (t7)
- S7 – 100: Output

3.2. Sơ đồ chuyển trạng thái:



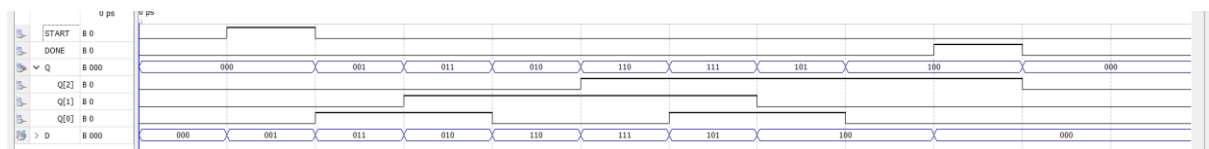
3.3. Thiết kế NextStateLogic



Input các DFF:

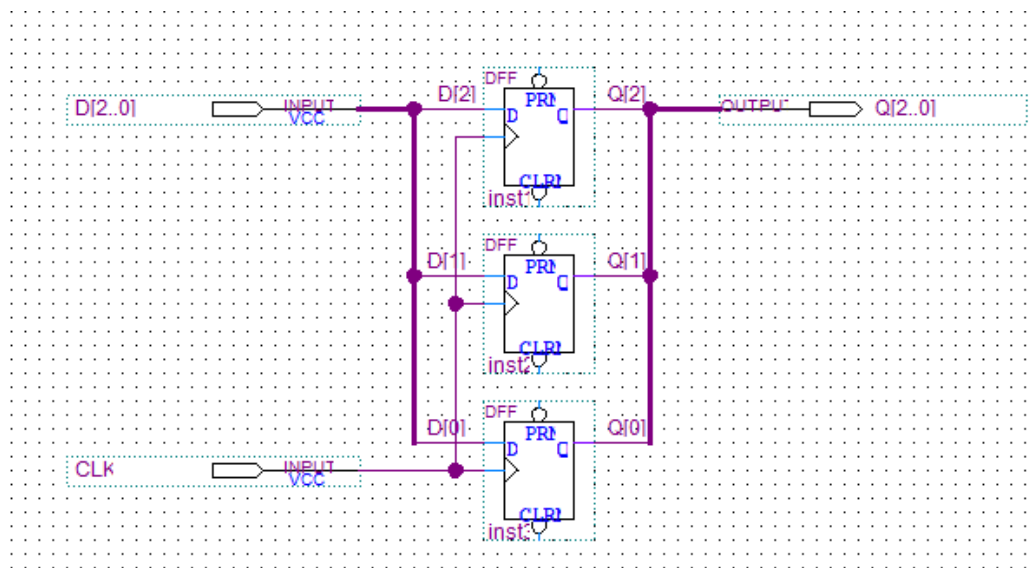
- $D0 = Q2.Q1 + Q2'.Q1'.(Q0 + START)$
- $D1 = Q1.Q0' + Q2'.Q1 + Q2'.Q1'.Q0.START'$
- $D2 = Q1.(Q2.Q0') + Q2.Q1'.(Q0 + DONE')$

Waveform:

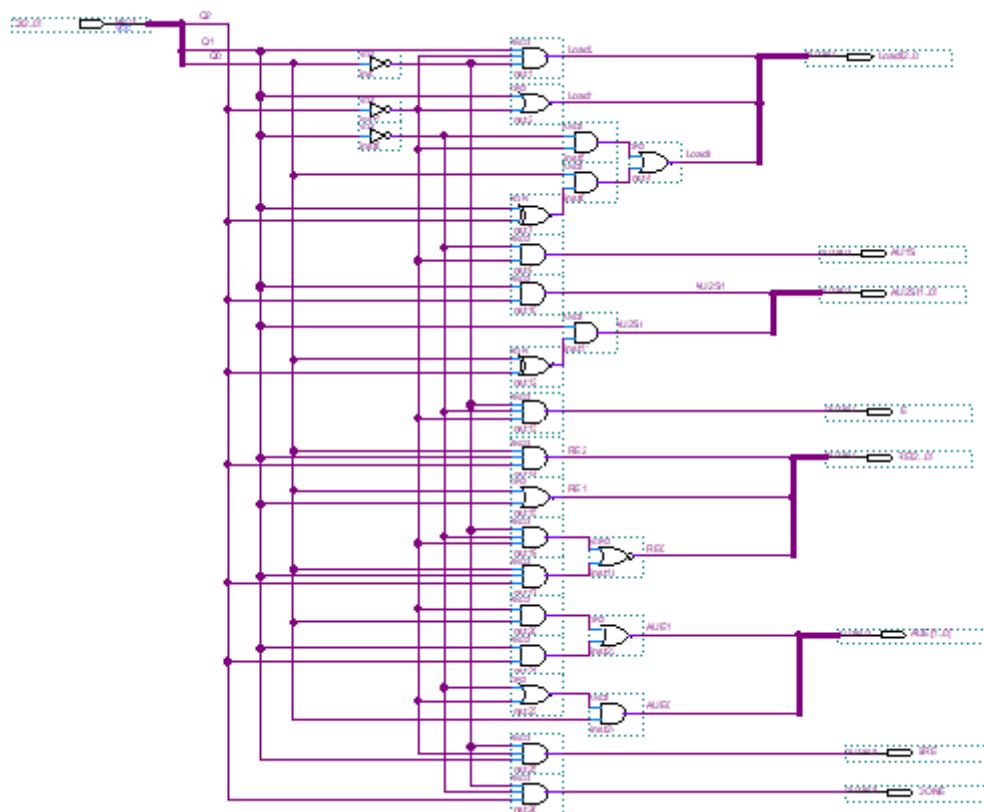


- 0 – 10ns: START = 0: S0 → S0
- 10 – 20ns: START = 1: S0 → S1
- 20 – 80ns: S1 → S2 → S3 → S4 → S5 → S6 → S7
- 80 – 90ns: DONE = 0: S7 → S7
- 90 – 100ns: DONE = 1: S7 → S0

3.4. Thiết kế khối DFFs



3.5. Thiết kế khối OutputLogic



Load[2..0]:

- $\text{Load}[2] = Q2'.Q1.Q0'$
- $\text{Load}[1] = Q2' + Q1$
- $\text{Load}[0] = Q2'.Q1' + Q0.(Q2 \oplus Q1)$

$$AU1S = Q2'.Q1'$$

AU2S[1..0]:

- AU2S[1] = Q2.Q1
- AU2S[0] = Q1.(Q2 $\oplus$ Q0)

$$IE = Q2'.Q1'.Q0'$$

RE[2..0]:

- RE[2] = Q2.Q1.Q0
- RE[1] = Q1 + Q0
- RE[0] = (Q2'.Q1'.Q0' + Q2.Q1.Q0)'

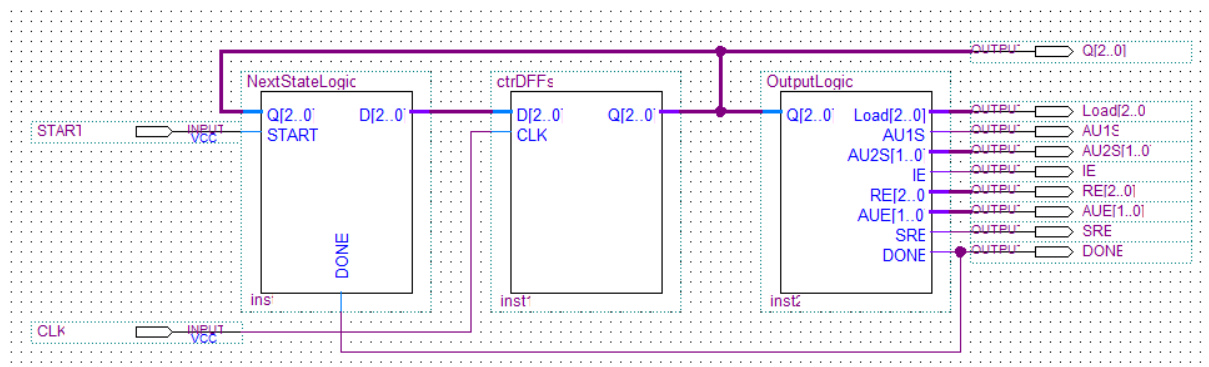
AUE[1..0]:

- AUE[1] = Q2'.Q0 + Q2.Q1
- AUE[0] = Q0.(Q2' + Q1')

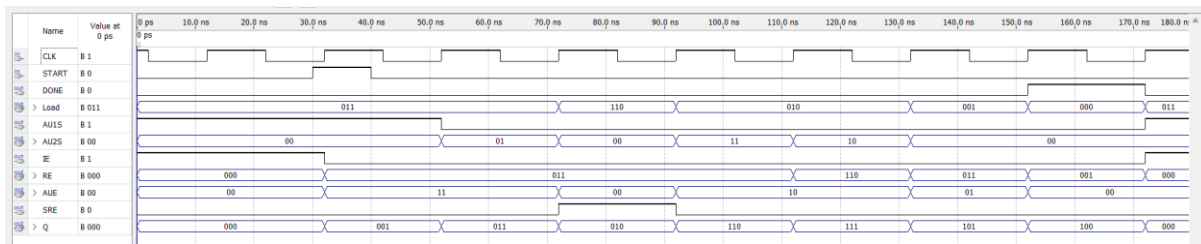
$$SRE = Q2'.Q1.Q0'$$

$$DONE = Q2.Q1'.Q0'$$

3.6. Thiết kế Controller

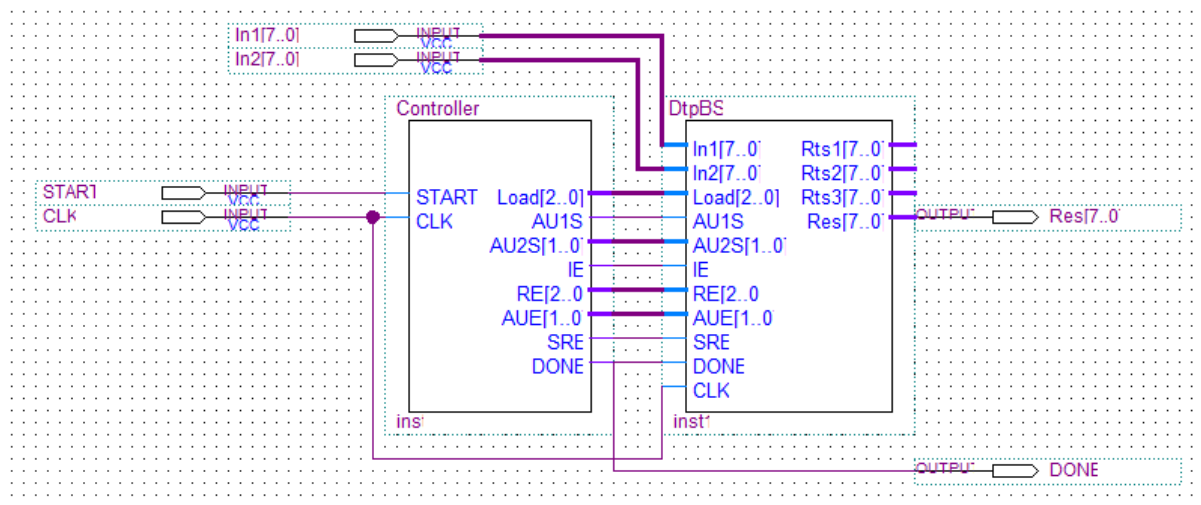


3.7. Waveform controller:

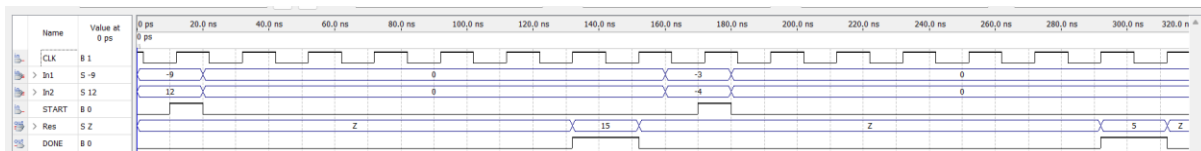


- 0 – 32ns: START = 0: giữ S0 → truyền output cho Datapath để thực hiện nạp Input
- 32ns: START = 1: S0 → S1 → truyền output cho Datapath để thực hiện tính trị tuyệt đối
- 52 – 172ns: S1 → S2 → S3 → S4 → S5 → S6 → S7 → lần lượt truyền output cho Datapath để thực hiện chức năng theo từng trạng thái
- 172 ns: DONE = 1: S7 → S0: bật cờ DONE, bắt đầu quay lại nạp Input mới

4. Thiết kế sử dụng Bus Sharing



5. Waveform

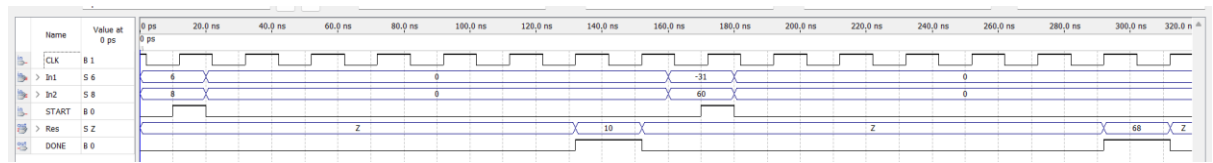


Thực hiện tính toán: $\sqrt{(-9)^2 + 12^2} = 15$ và $\sqrt{(-3)^2 + (-4)^2} = 5$

⇒ Kết quả tính toán đúng

Cờ DONE được bật khi kết quả đã được tính toán xong, cờ DONE đã thực hiện đúng chức năng

Thử với các cặp toán hạng khác:



Thực hiện tính toán: $\sqrt{6^2 + 8^2} = 10$ và $\sqrt{(-31)^2 + 60^2} = \max((0.875 \times 60 + 0.5 \times |-31|), 60) = 68$

Kết quả tính toán đúng và cờ DONE bật khi kết quả đã tính toán xong

L. Thiết kế sử dụng Register Merging

1. Cơ sở lý thuyết

Dựa trên Bus sharing, có thể thấy R1 và R3 sử dụng chung 1 bus (vì không cùng thực hiện cùng 1 chức năng tại mọi thời điểm) nên ta cũng có thể gộp R1 và R3 với nhau thành Register File với:

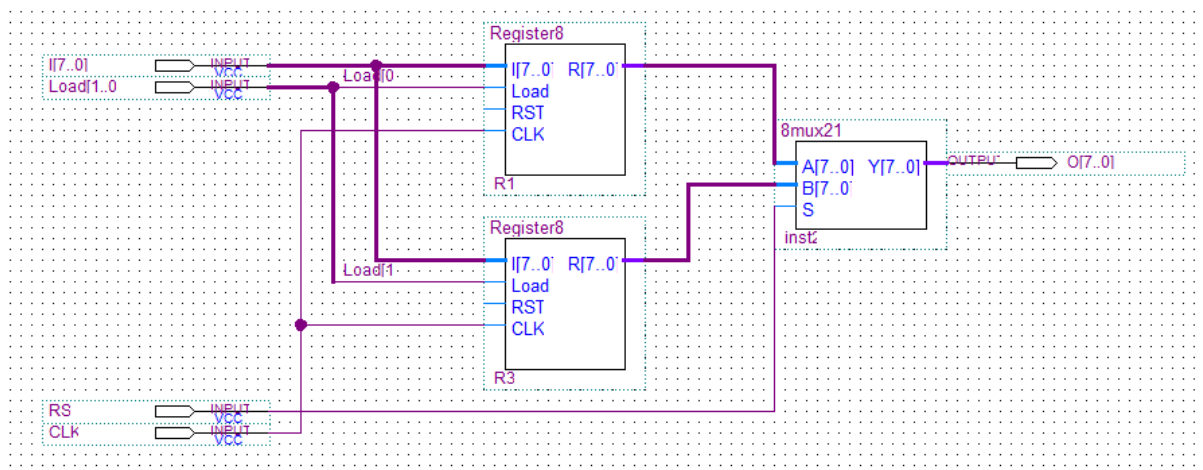
- $I[7..0]$: input cho cả R1 & R3
- $Load[1..0]$: cho phép thanh ghi nào được ghi
- Mux21 8 bit cùng tín hiệu Select RS: cho phép thanh ghi nào được đọc
- Tri-State tại Output: Cho phép đọc hay không

Sử dụng lại các khối chức năng đã thiết kế ở Bus sharing:

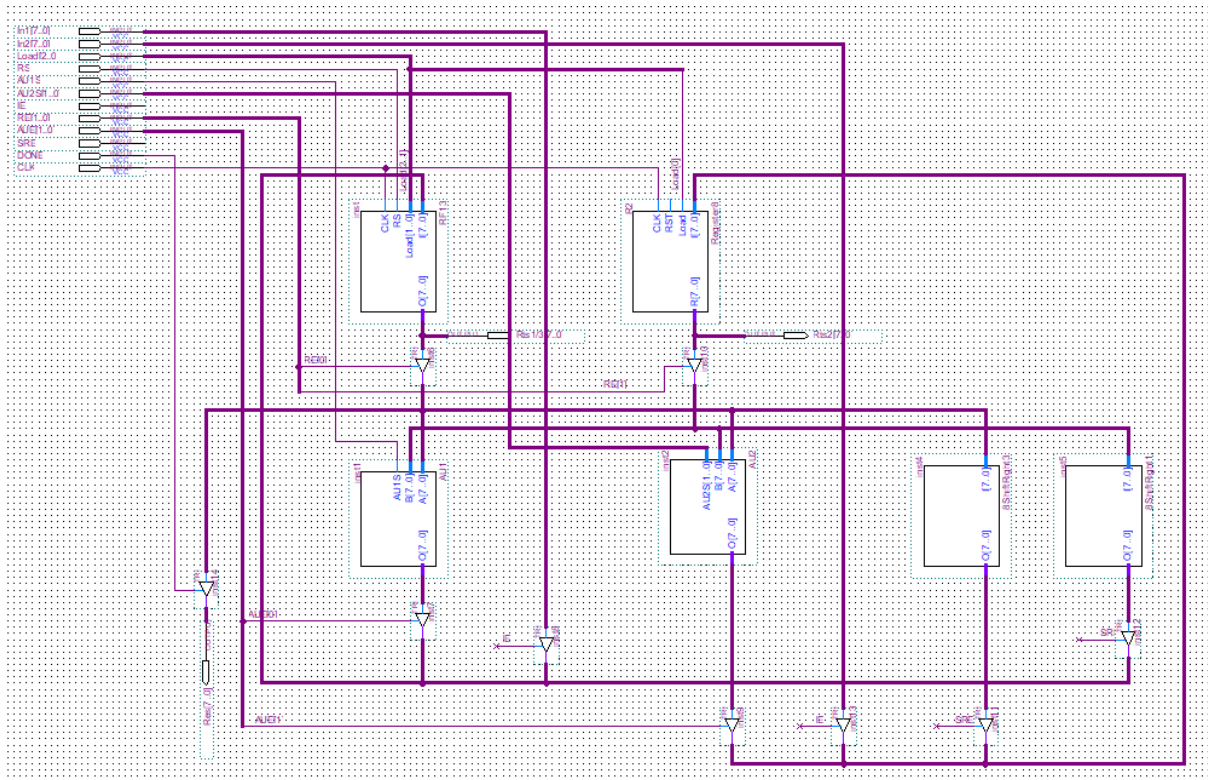
- Trong Datapath: Register 8 bit, AU1, AU2, Shift Right 1 bit & 3 bit cho giá trị 8 bit
- Trong Controller: NextStateLogic, DFFs (vì có số trạng thái bằng nhau (= 8))

2. Thiết kế Datapath

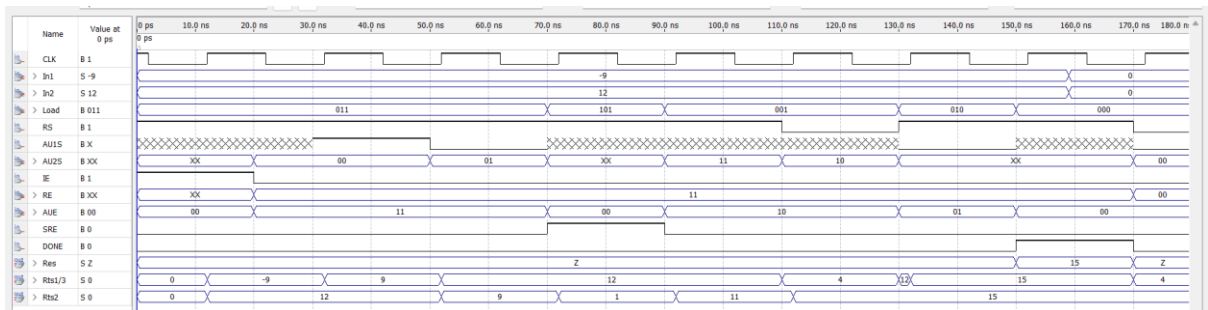
2.1. Thiết kế Register File chứa R1 & R3



2.2. Thiết kế Datapath



2.3. Waveform datapath:



Khoảng thời gian	Chức năng	Output
0 – 32ns	$R1 = In1$ $R2 = In2$	Z
32 – 52ns	$R1 = R1 $ $R2 = R2 $	
52 – 72ns	$R1 = \max(R1, R2)$ $R2 = \min(R1, R2)$	
72 – 92ns	$R2 = R1 \gg 3$ $R3 = R2 \gg 1$	

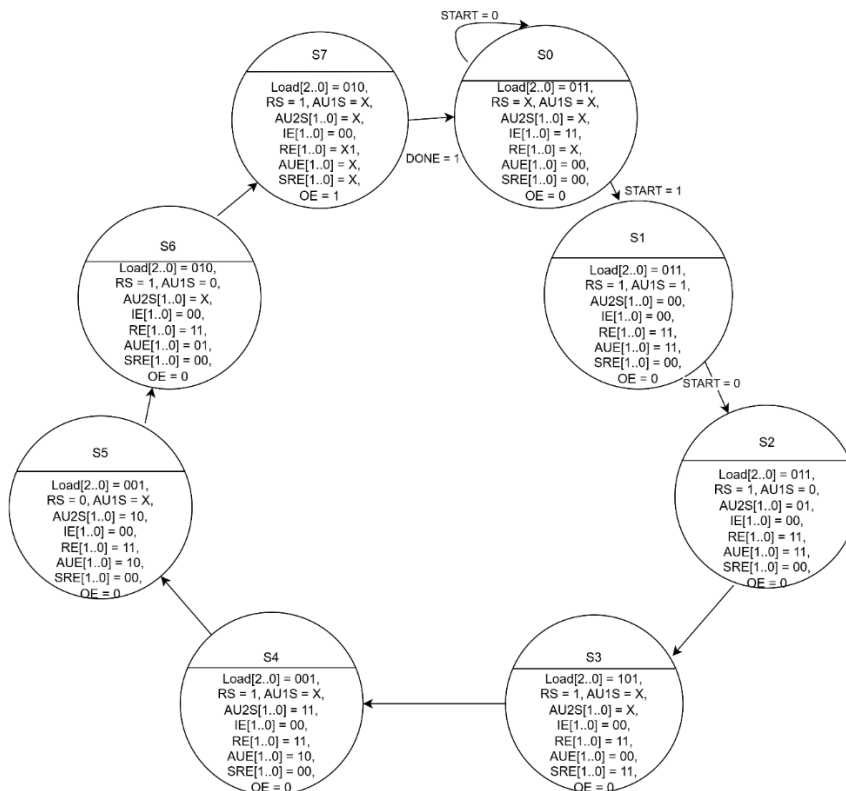
92 – 112ns	$R2 = R1 - R2$	
112 – 132ns	$R2 = R3 + R2$	
132 – 152ns	$R1 = \max(R2, R1)$	
152 – 172ns	Output = R1	15

3. Thiết kế Controller

3.1. Mã hóa trạng thái:

- S0 – 000: nạp Input
- S1 – 001: tính trị tuyệt đối
- S2 – 011: tìm max, min
- S3 – 010: dịch phải
- S4 – 110: tính hiệu
- S5 – 111: tính tổng
- S6 – 101: tìm max
- S7 – 100: xuất Output

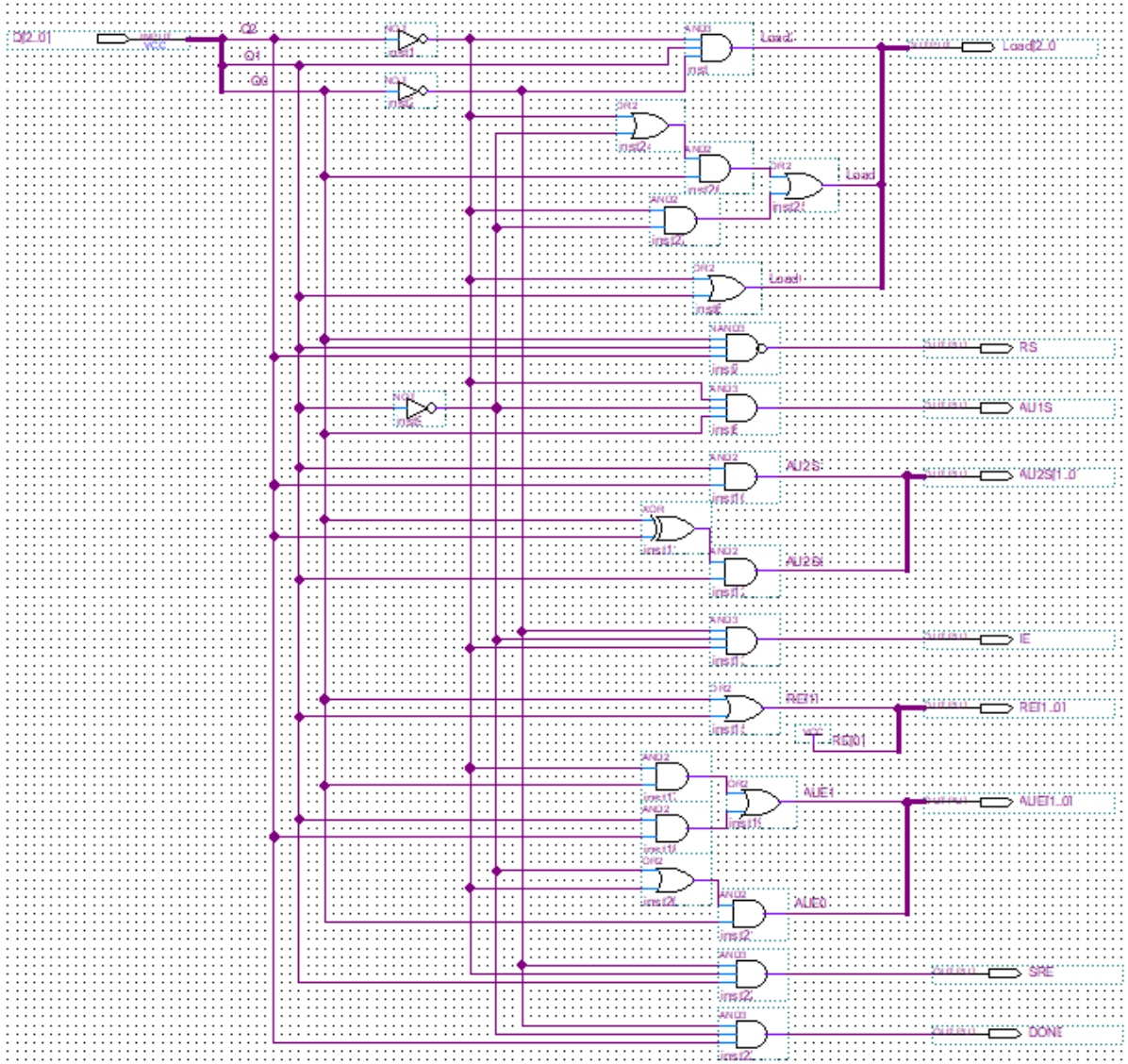
3.2. Sơ đồ chuyển trạng thái:



3.3. Thiết kế NextStateLogic và DFFs

Sử dụng lại thiết kế NextStateLogic và DFFs của Register Sharing

3.4. Thiết kế OutputLogic



Biểu thức output của controller:

Load[2..0]:

- $\text{Load}[2] = Q2'.Q1.Q0'$
- $\text{Load}[1] = Q0.(Q2' + Q1') + Q2'.Q1'$
- $\text{Load}[0] = Q2' + Q1$

$\text{RS} = (Q2.Q1.Q0)'$

$\text{AU1S} = Q2'.Q1'.Q0$

AU2S[1..0]:

- $AU2S[1] = Q2.Q1$
- $AU2S[0] = Q1.(Q2 \oplus Q0)$

IE = $Q2'.Q1'.Q0'$

RE[1..0]:

- $RE[1] = Q1 + Q0$
- $RE[0] = 1$

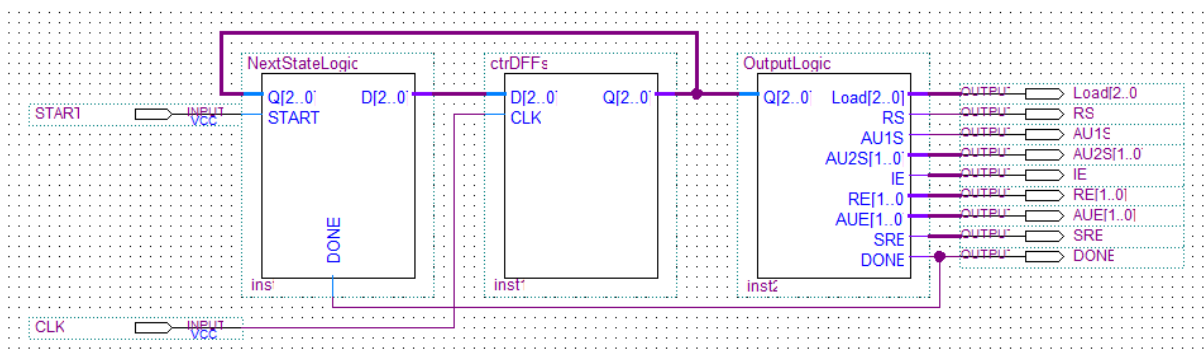
AUE[1..0]:

- $AUE[1] = Q2'.Q0 + Q2.Q1$
- $AUE[0] = Q0.(Q2' + Q1')$

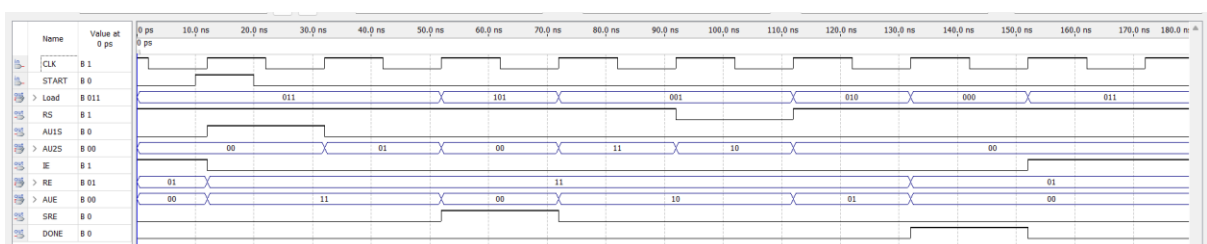
SRE = $Q2'.Q1.Q0'$

DONE = $Q2.Q1'.Q0'$

3.5. Thiết kế Controller tổng quan:

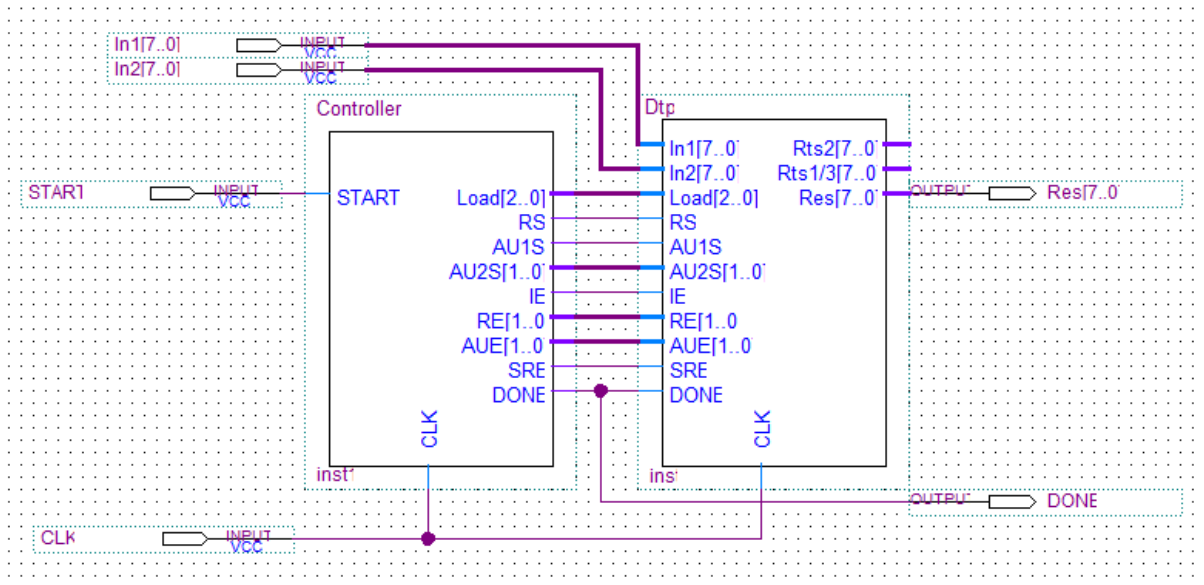


3.6. Waveform:

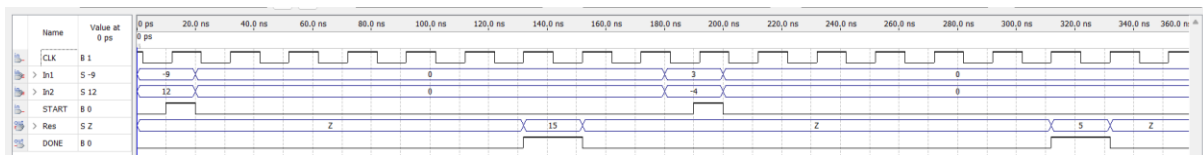


- 0 – 12ns: START = 0: giữ S0, truyền tín hiệu nạp Input vào Datapath
- 12ns: START = 1: S0 → S1
- 12 – 152ns: S1 → S2 → S3 → S4 → S5 → S6 → S7, lần lượt truyền tín hiệu xử lý theo từng trạng thái
- 152ns: DONE = 1: S7 → S0, hoàn thành tính toán, quay về trạng thái nạp Input

4. Thiết kế mạch sử dụng Register Merging



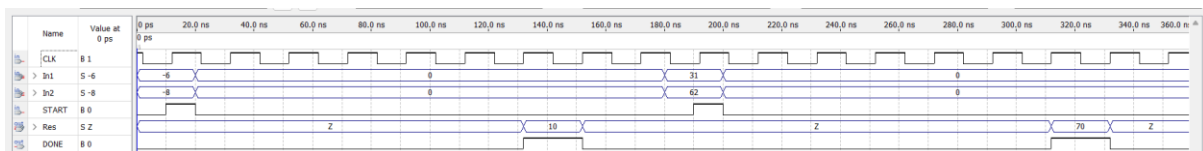
5. Waveform



Thực hiện tính toán $\sqrt{(-9)^2 + 12^2} = 15$ và $\sqrt{3^2 + (-4)^2} = 5$

⇒ Kết quả tính toán đúng và cờ báo DONE = 1 khi có kết quả cuối cùng

Thực hiện tính toán với cặp toán hạng khác:



Thực hiện tính toán $\sqrt{(-6)^2 + (-8)^2} = 10$ và $\sqrt{31^2 + 62^2} \approx$

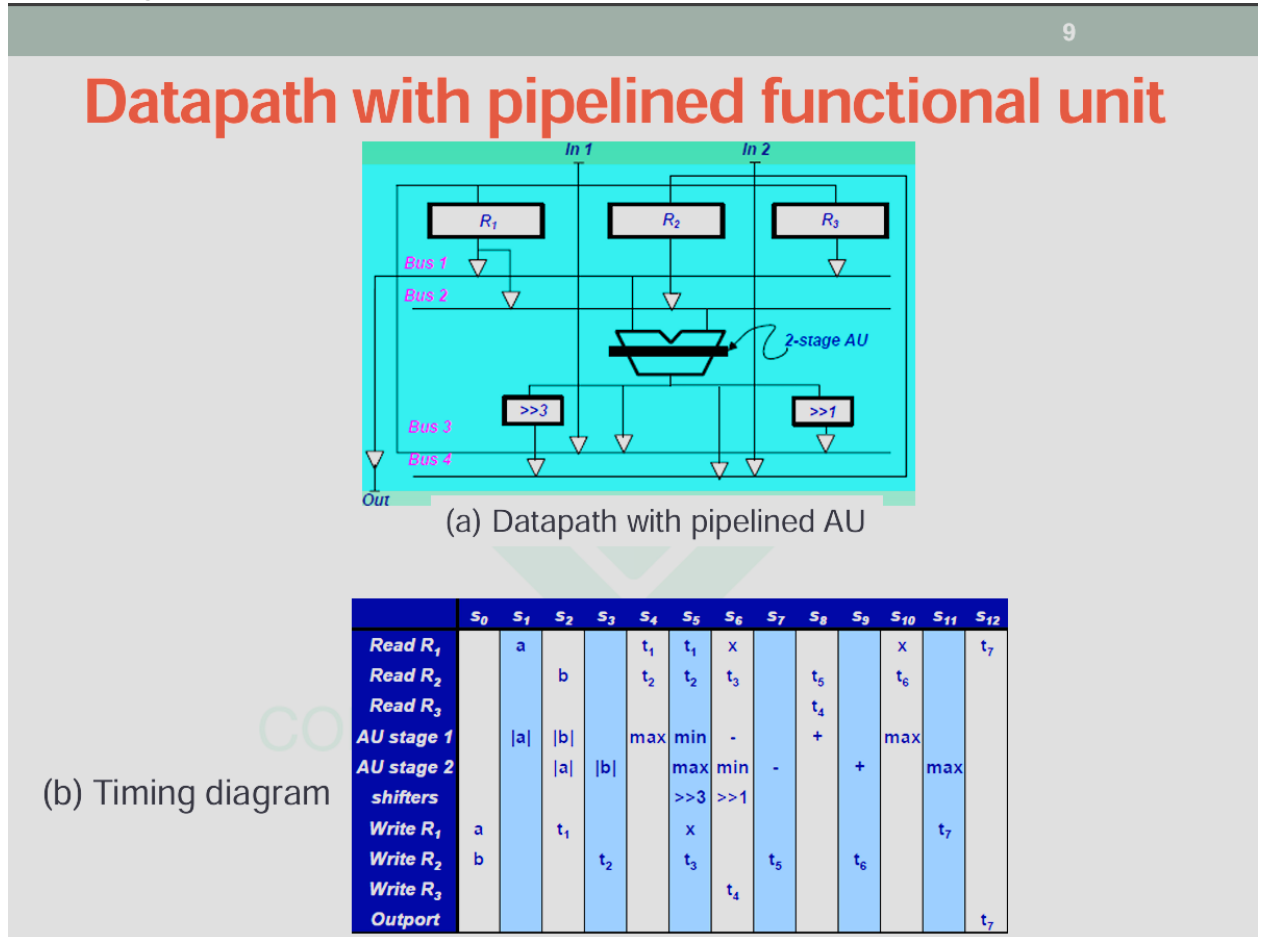
$\max((0.875 \times 62 + 0.5 \times 31), 62) \approx 70$

⇒ Kết quả tính toán đúng và cờ báo DONE = 1 khi có kết quả cuối cùng

M. Thiết kế sử dụng pipelined functional unit

1. Cơ sở lý thuyết

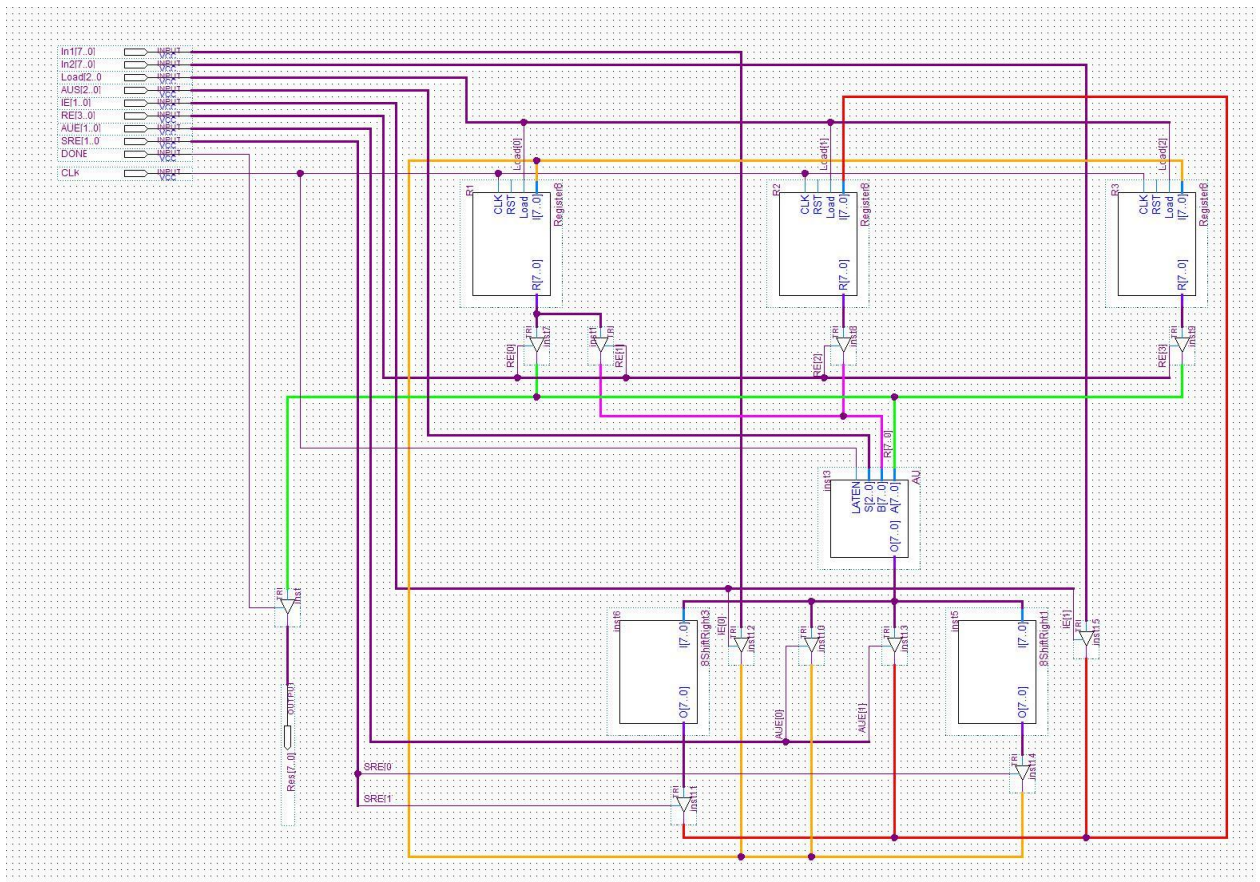
Để thực hiện tính toán song song nhiều cặp giá trị a, b ta thiết kế 1 khối AU 2 giai đoạn để tính toán như hình



2. Thiết kế datapath

Sử dụng datapath của phần bus sharing, ta chỉnh sửa các khối functional unit thành 1 khối AU có 2 giai đoạn

Các khối R_1 , R_2 , R_3 và Shift Right 1 & 3 được sử dụng lại thiết kế từ các bài tập trước

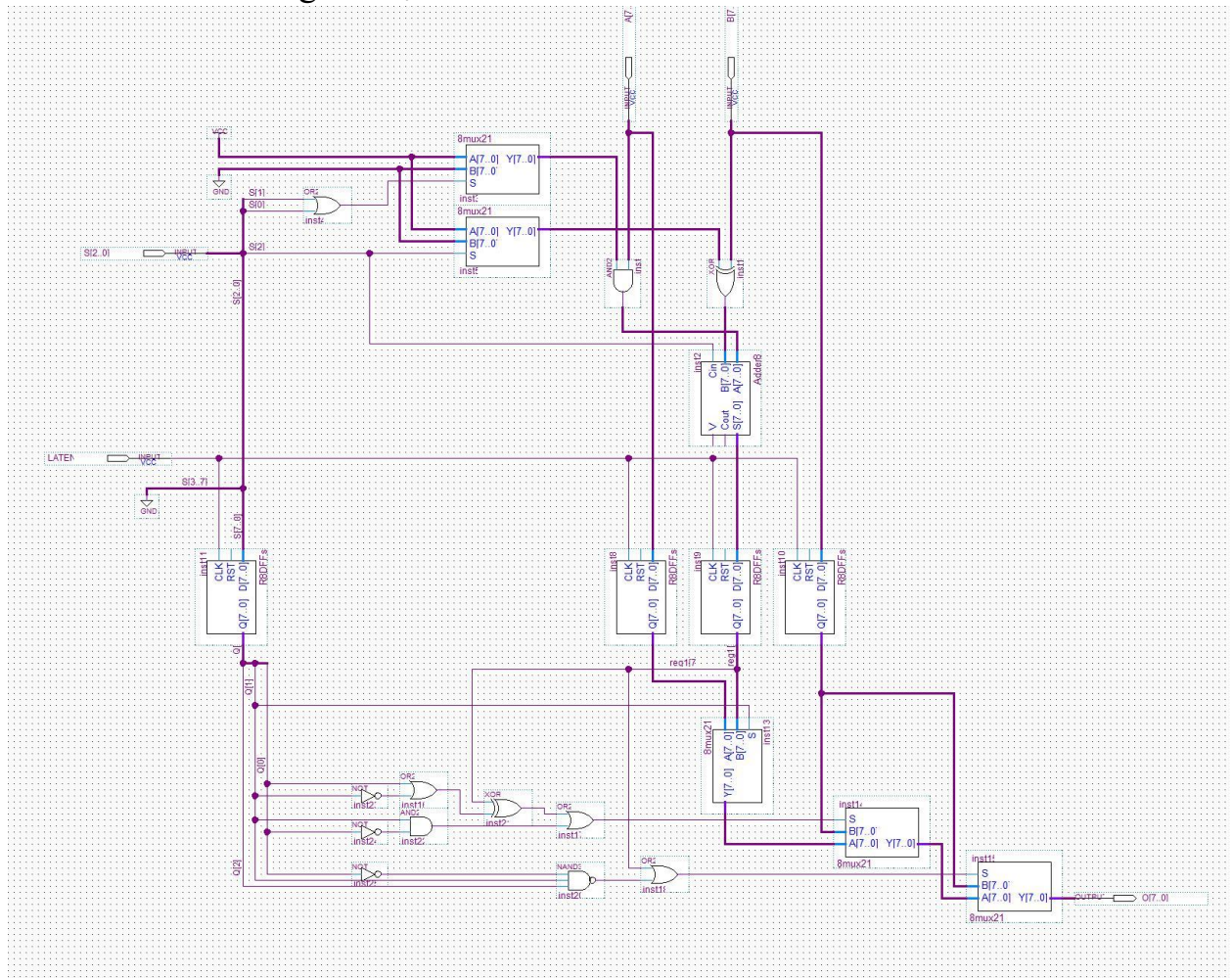


Các input của datapath:

- In1[7..0], In2[7..0]: cặp số a,b để nạp vào biểu thức
- Load[2..0]: tín hiệu điều khiển khi nào thì register load input
 - o Load[0]: R1 load input
 - o Load[1]: R2 load input
 - o Load[2]: R3 load input
- AUS[2..0]: tín hiệu lựa chọn chế độ tính toán của khối AU theo mã hóa sau:
 - o 001: phép cộng
 - o 100: trị tuyệt đối
 - o 101: phép trừ
 - o 110: min
 - o 111: max
- IE[1..0]: Tín hiệu điều khiển việc lấy input In1 và In2
 - o IE[0]: Lấy input In1
 - o IE[1]: Lấy input In2
- RE[3..0]: Tín hiệu lựa chọn output của 3 register
 - o RE[0]: Điều khiển lựa chọn register 1 cho bus 1
 - o RE[1]: Điều khiển lựa chọn register 1 cho bus 2
 - o RE[2]: Điều khiển lựa chọn register 2 cho bus 2
 - o RE[3]: Điều khiển lựa chọn register 3 cho bus 1

- AUE[1..0]: Tín hiệu lựa chọn lấy output của AU đi vào bus 3 và bus 4
 - o AUE[0]: Output của AU đi vào bus 3
 - o AUE[1]: Output của AU đi vào bus 4
- SRE[1..0]: Điều khiển output của bộ dịch phải 3 bit và 1 bit
 - o SRE[0] : Output của bộ dịch phải 1 bit đi vào bus 3
 - o SRE[1] : Output của bộ dịch phải 3 bit đi vào bus 4
- DONE : Tín hiệu done báo hiệu mạch tính toán xong và xuất output

Thiết kế khối AU 2 giai đoạn:



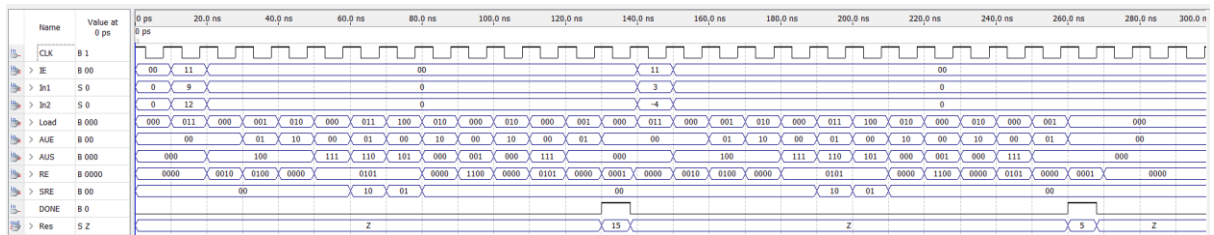
Bảng chức năng của AU:

Ta chia thành 2 giai đoạn:

- Giai đoạn 1: Tính toán cộng trừ từ các input
- Giai đoạn 2: Lựa chọn output phù hợp với yêu cầu

Ở giữa 2 giai đoạn sử dụng 4 Register để lưu output của giai đoạn 1 để làm input cho giai đoạn 2

*waveform datapath:



Giải thích datapath:

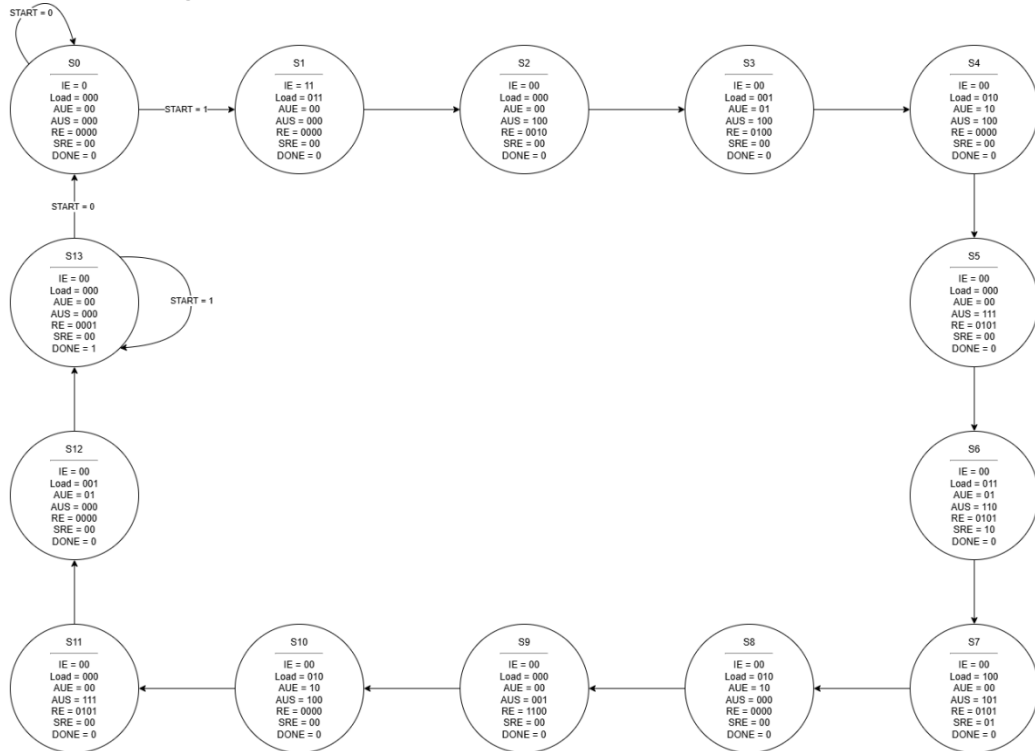
- 0ns – 10ns: Datapath không hoạt động, không có input được nạp vào
 - 18ns: Nhận cặp input [9;12], datapath bắt đầu hoạt động và thực hiện tính toán
 - 18ns – 130ns: Datapath thực hiện tính toán cho cặp input trên
 - 130ns: DONE = 1, tiến hành xuất kết quả tính toán là Res = 15
 - 148ns: Nhận cặp input [3;-4], datapath tiến hành tiếp tục tính toán cho cặp input tiếp theo
 - 148ns – 260ns: Datapath thực hiện tính toán cho cặp input trên
 - 260ns: DONE = 1, tiến hành xuất kết quả tính toán là Res = 5
- Các kết quả tính toán cho các input là đúng

3. Thiết kế controller

3.1. Mã hóa trạng thái:

- S0: 0000 -> Trạng thái chờ
- S1: 0001 -> Load In1 và In2
- S2: 0011 -> Read R1 và tính trị tuyệt đối R1 tại stage 1
- S3: 0010 -> Read R2, tính trị tuyệt đối R2 tại stage 1, tính trị tuyệt đối R1 tại stage 2 và lưu kết quả vào R1
- S4: 0110 -> Tính trị tuyệt đối R2 tại stage 2 và lưu kết quả vào R2
- S5: 0111 -> Read R1 và R2 và tính max tại stage 1
- S6: 0101 -> Read R1 và R2, tính min tại stage 1, tính max tại stage 2, shift right 3 và lưu kết quả R1 = max, R2 = max >>3
- S7: 0100 -> Read R1 và R2, tính trừ tại stage 1, tính min tại stage 2, shift right 1 và lưu kết quả R3 = min >> 1
- S8: 1100 -> Tính trừ tại stage 2 và lưu kết quả R2 = R1 – R2
- S9: 1101 -> Read R2 và R3, tính tổng tại stage 1
- S10: 1111 -> Tính tổng tại stage 2 và lưu kết quả vào R2
- S11: 1110 -> Read R1 và R2, tính max tại stage 1
- S12: 1010 -> Tính max tại stage 2 và lưu kết quả vào R1
- S13: 1011 -> Báo done = 1, đọc R1 và cho ra output = R1

3.2. Sơ đồ trạng thái:



3.3. Biểu thức input cho các D-flipflop:

$$D3 = Q3+ = Q3Q2 + Q2Q1'Q0' + Q3Q1Q0'$$

$$D2 = Q2+ = Q3'Q2 + Q2Q1' + Q2Q0 + Q3'Q1Q0'$$

$$D1 = Q1+ = Q3'Q2'Q0 + Q3'Q1Q0' + Q3Q2Q0 + Q3Q1Q0'$$

$$D0 = Q0+ = Q2'Q1'(Q0 + START) + Q3'Q2Q1 + Q3Q2Q1' + Q3Q2'Q0'$$

3.4. Biểu thức output của controller:

$$Load[0] = Q3'Q1'Q0 + Q2'Q1Q0'$$

$$Load[1] = Q3'Q1'Q0 + Q3Q1'Q0' + Q3'Q2Q1Q0' + Q3Q2Q1Q0$$

$$Load[2] = Q3'Q2Q1'Q0'$$

$$AUS[2] = Q3'Q2 + Q3'Q1 + Q3Q2Q1$$

$$AUS[1] = Q3'Q2Q0 + Q3Q1Q0'$$

$$AUS[0] = Q3Q2 + Q2Q1Q0 + Q3Q1Q0' + Q2Q1'Q0'$$

$$IE[0] = Q3'Q2'Q1'Q0$$

$$IE[1] = Q3'Q2'Q1'Q0$$

$$RE[0] = Q3'Q2Q1' + Q3'Q2Q0 + Q3Q2'Q0 + Q3Q2Q1Q0'$$

$$RE[1] = Q3'Q2'Q1Q0$$

$$RE[2] = Q3'Q2Q1' + Q3'Q2Q0 + Q3Q1'Q0 + Q3Q2Q1Q0' + Q3'Q2'Q1Q0'$$

$$RE[3] = Q3Q2Q1'Q0$$

$$AUE[0] = Q2'Q1Q0' + Q3'Q2Q1'Q0$$

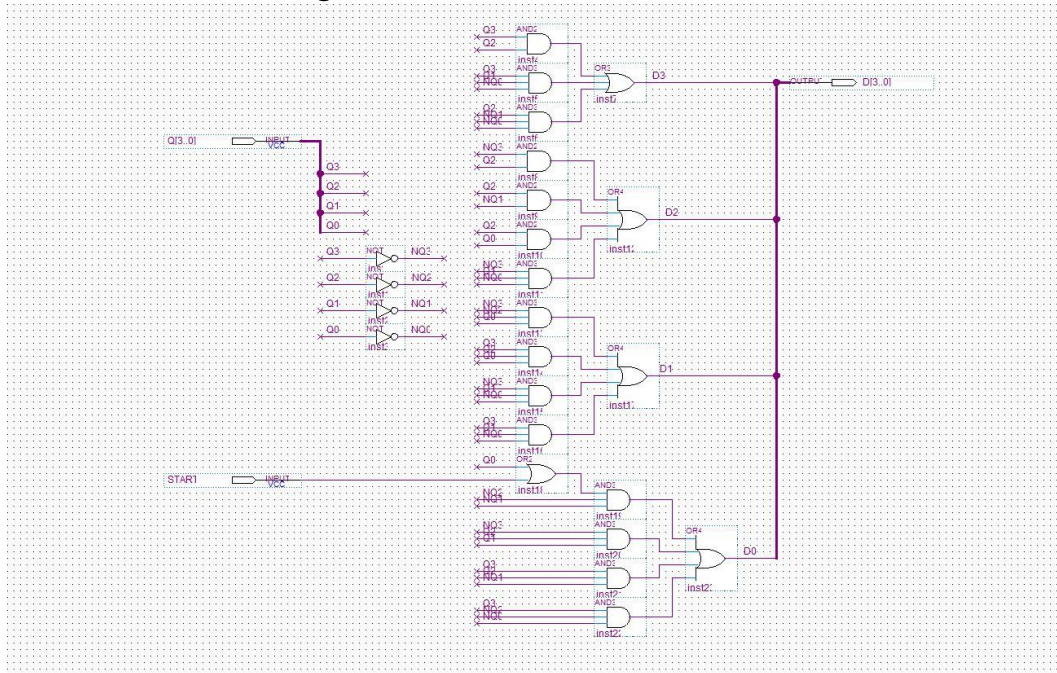
$$AUE[1] = Q3Q1'Q0' + Q3'Q2Q1Q0' + Q3Q2Q1Q0$$

$$SRE[0] = Q3'Q2Q1'Q0'$$

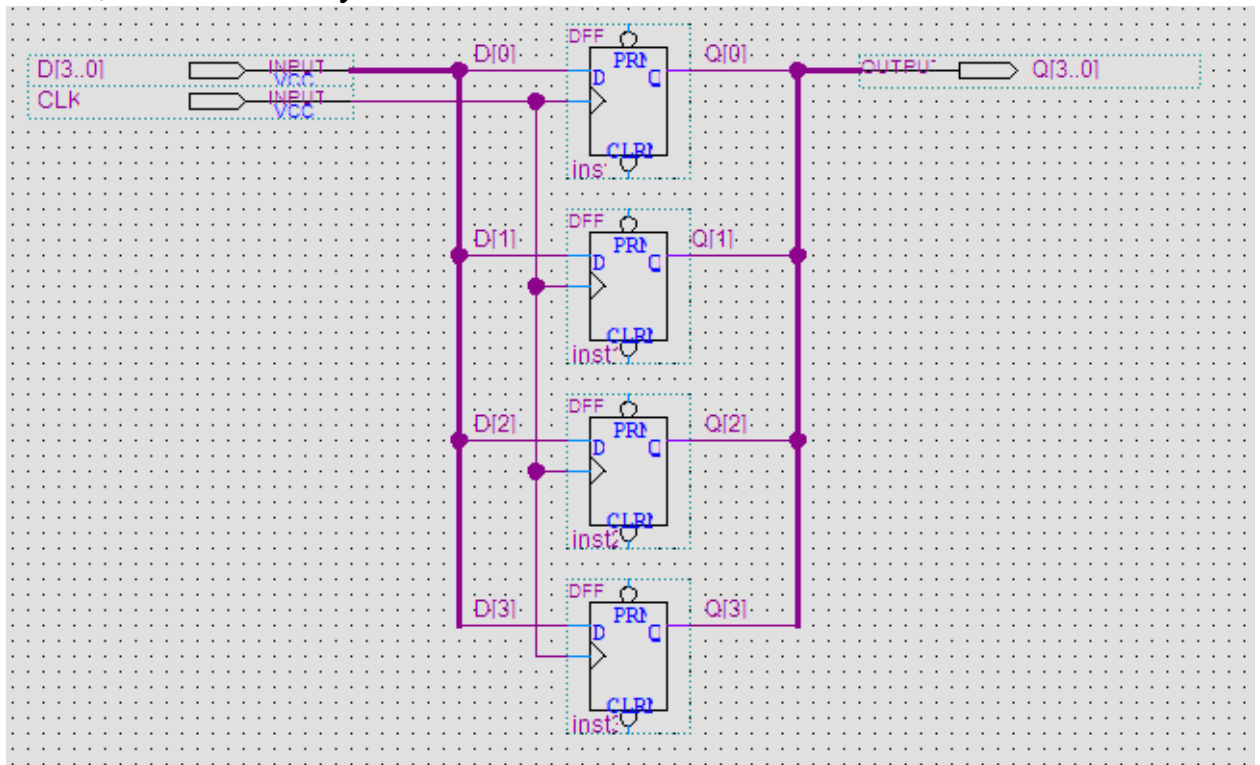
$$SRE[1] = Q3'Q2Q1'Q0$$

$$DONE = Q3Q2'Q1Q0$$

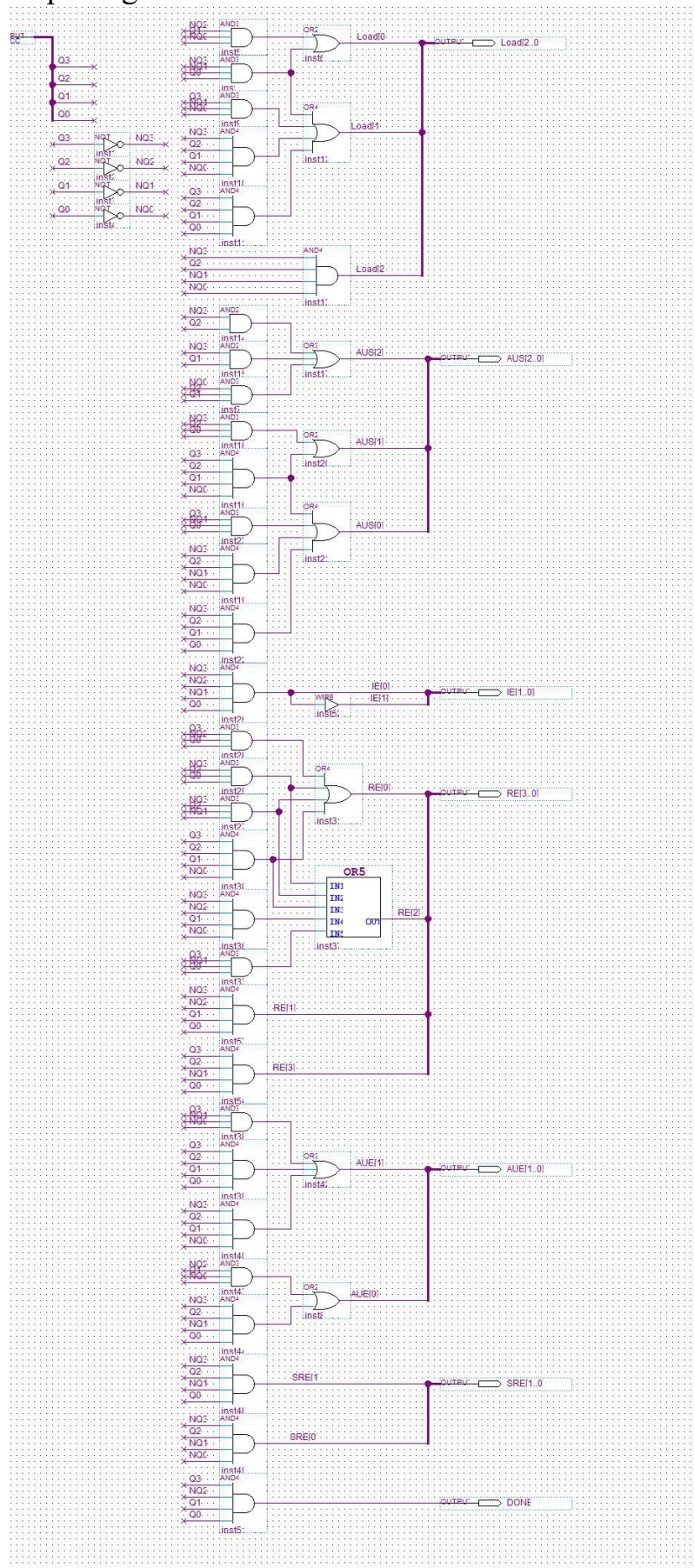
3.5. Mạch next-state logic :



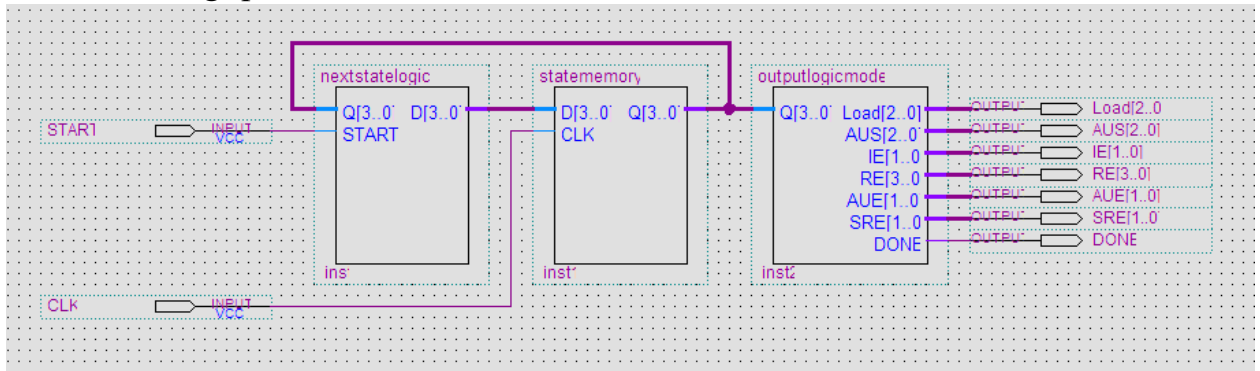
3.6. Mạch state memory :



3.7. Mạch output logic mode :

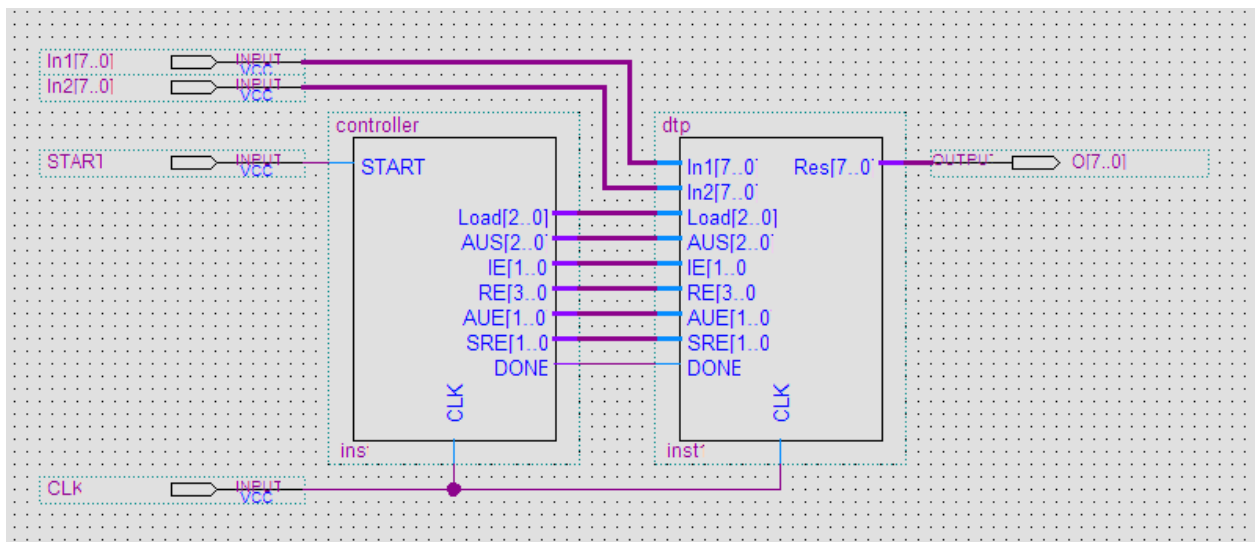


3.8. Mạch tổng quan :



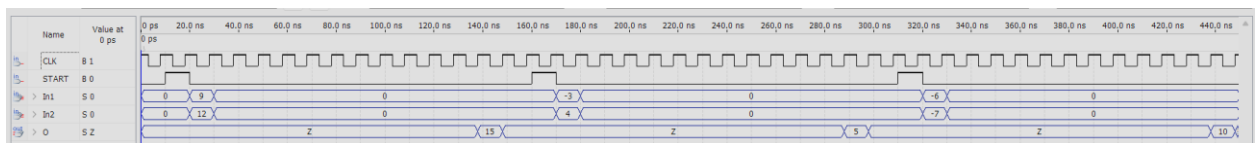
Kết nối 3 khối nextstatelogic, statememory và outputlogic ta được controller hoàn chỉnh

4. Mạch hoàn chỉnh



Đây là hình ảnh mạch sau khi đã kết nối controller và datapath với nhau

5. Waveform



Giải thích waveform :

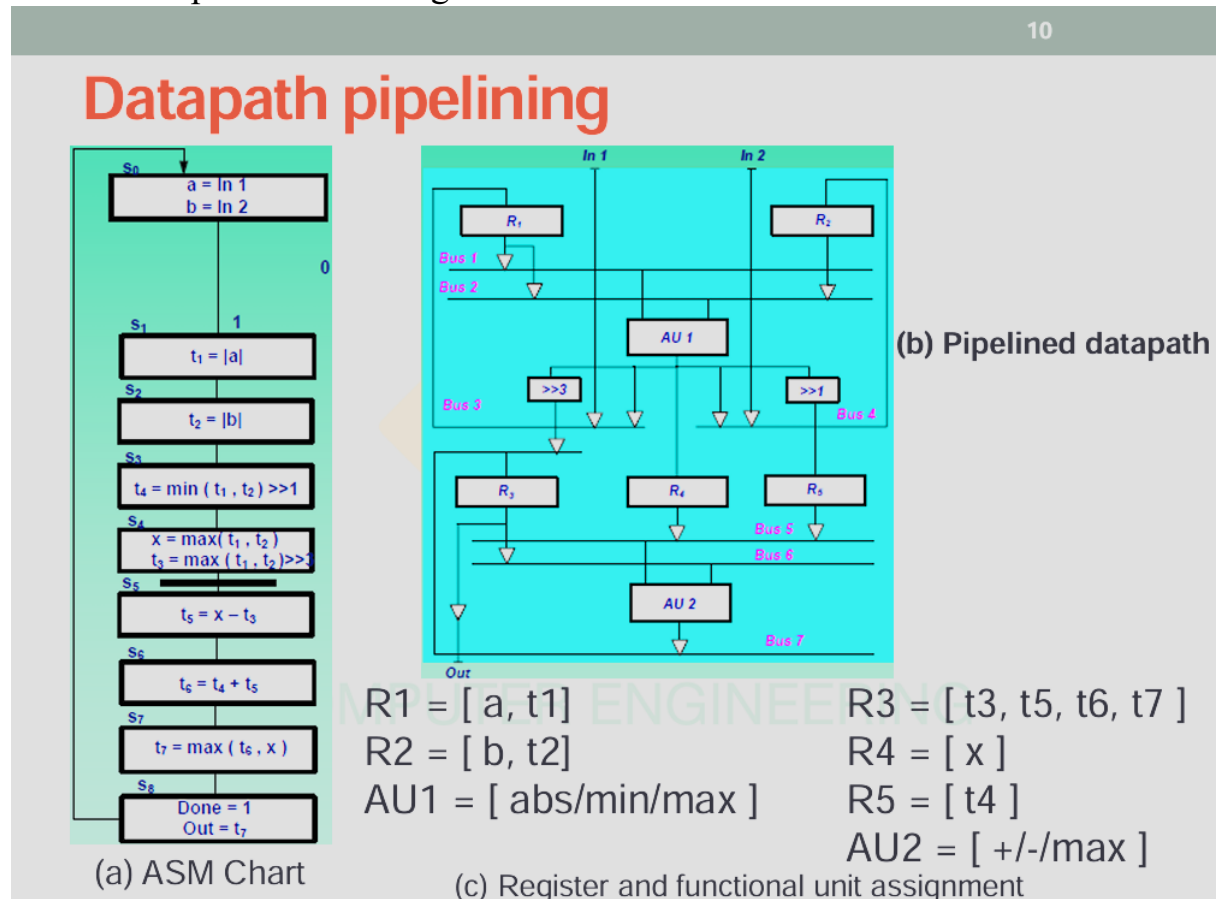
- Cặp $[a; b] = [9; 12]$ (test 2 số nguyên dương ra kết quả là số nguyên) :
 - Tại 10ns : START = 1, và tại cạnh lên 18ns mạch khởi động và bắt đầu thực thi
 - Tại 28ns : Cạnh lên xung clock và mạch nhận input In1 = 9 và In2 = 12
 - Từ 28ns – 138ns: Mạch thực hiện tính toán và tại cạnh lên 138ns thì cho ra output
- Cặp $[a; b] = [-3; 4]$ (test 1 số âm và 1 số dương ra kết quả là số nguyên):

- Tại 160ns : START = 1, và tại cạnh lên 168ns mạch khởi động và bắt đầu thực thi
- Tại 178ns : Cạnh lên xung clock và mạch nhận input In1 = -3 và In2 = 4
- Từ 178ns – 288ns: Mạch thực hiện tính toán và tại cạnh lên 288ns thì cho ra output
- Cặp [a;b] = [-6;-7] (test 2 số âm ra kết quả là số thập phân được làm tròn)
 - Tại 310ns : START = 1, và tại cạnh lên 318ns mạch khởi động và bắt đầu thực thi
 - Tại 328ns : Cạnh lên xung clock và mạch nhận input In1 = -3 và In2 = 4
 - Từ 328ns – 438ns: Mạch thực hiện tính toán và tại cạnh lên 438ns thì cho ra output

N. Thiết kế datapath pipelined

1. Cơ sở lý thuyết

Vì sự hạn chế của mạch datapath với pipelined functional unit là chỉ nạp được 1 cặp input [a;b] trong mỗi lần xử lý, do đó ta tiến hành thiết kế mạch datapath pipelining để có thể xử lý nhiều cặp input [a;b] lồng nhau. Ta chia datapath thành 2 tầng như hình.



Trong đó:

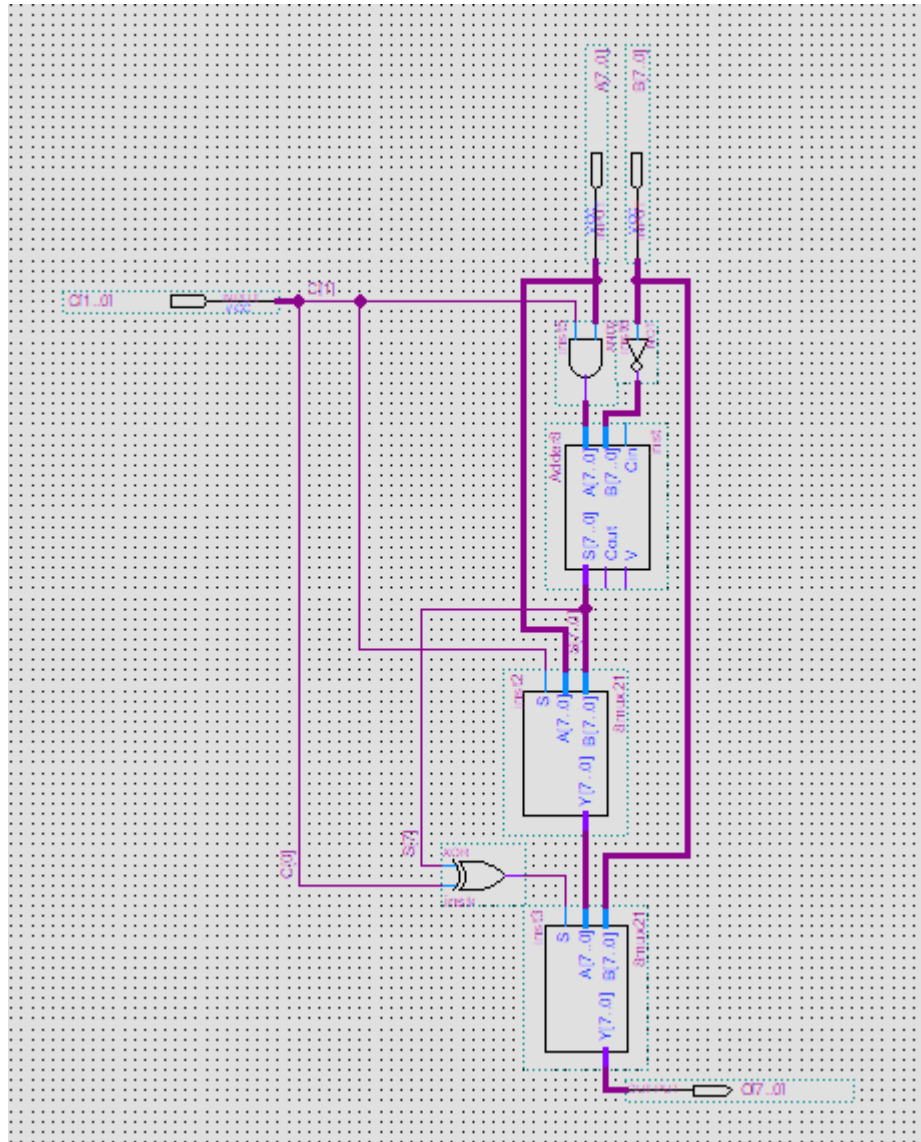
- Tầng 1 bao gồm 2 thanh ghi và 1 AU xử lý abs/min/max
- Tầng 2 bao gồm 3 thanh ghi và 1 AU xử lý +/-/max

Khi phần 1 đã xử lý xong thì có thể nạp tiếp 1 cặp input để thực hiện xử lý đồng thời.

2. Thiết kế datapath

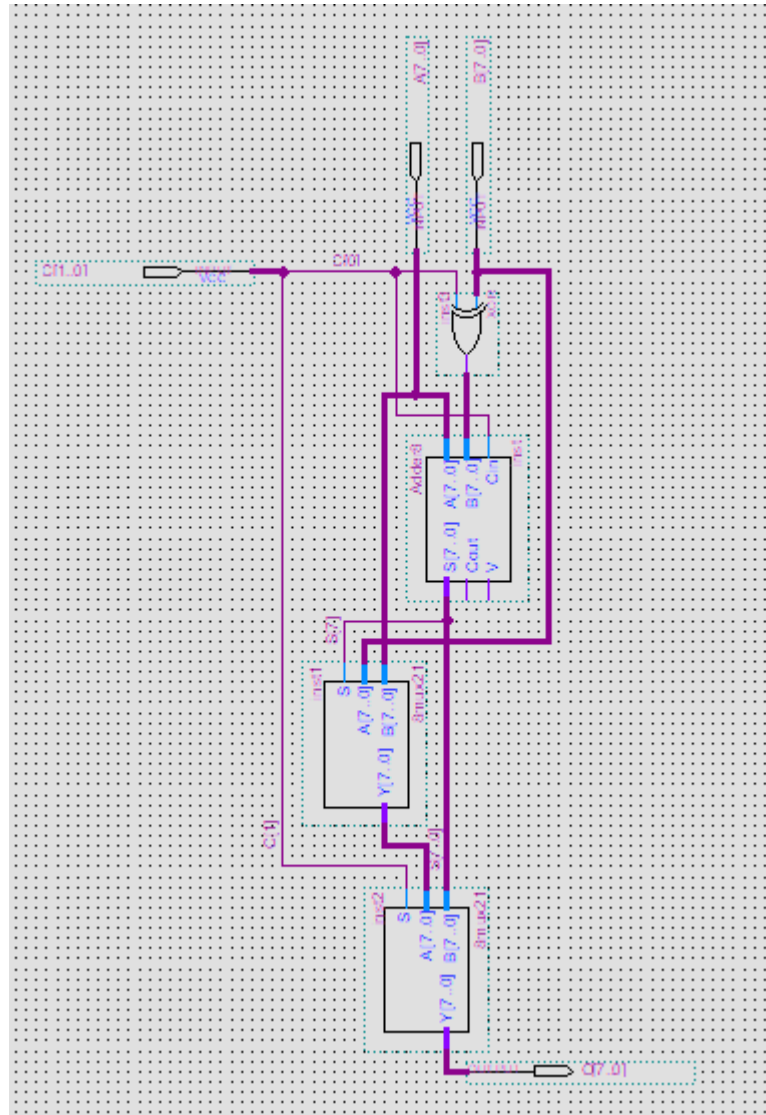
Các khối thanh ghi và shift được tái sử dụng từ mạch datapath với pipelined functional unit

a. Khối AU1



Khối AU1 bao gồm 1 khối adder, 2 khối mux2to1 thực hiện xử lý theo tín hiệu điều khiển sau:

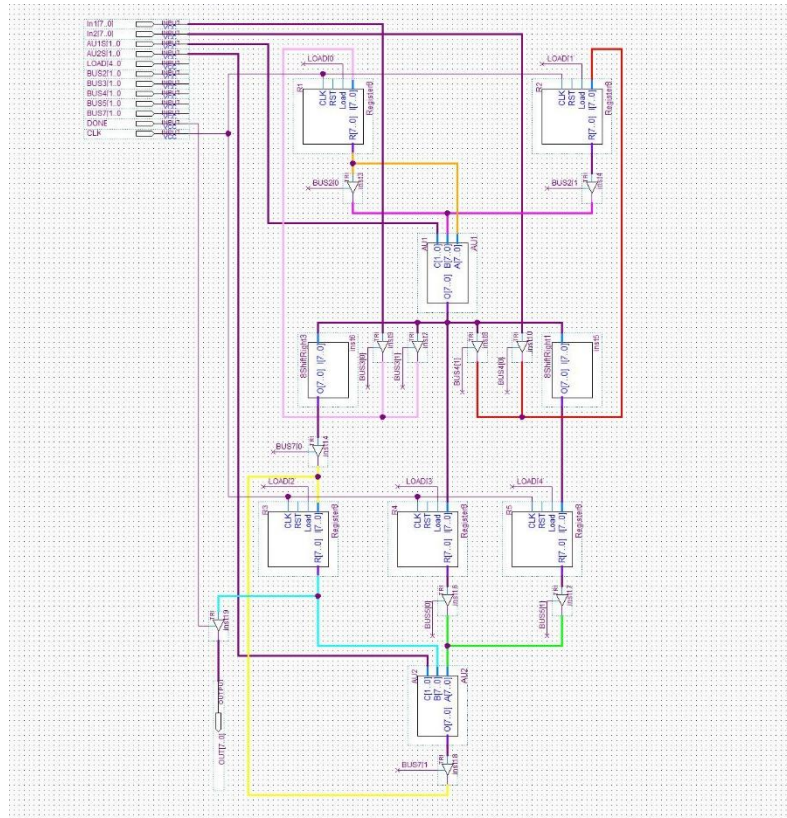
- 01: Tính trị tuyệt đối B ($|B|$)
 - 10: Tính min(A,B)
 - 11: Tính max(A,B)
- b. Khối AU2



Khối AU2 bao gồm 1 khối adder, 2 khối mux2to1 thực hiện xử lý theo tín hiệu điều khiển sau:

- 00: Tính phép cộng ($A+B$)
- 01: Tính phép trừ ($A-B$)
- 11: Tính $\max(A,B)$

c. Datapath hoàn chỉnh

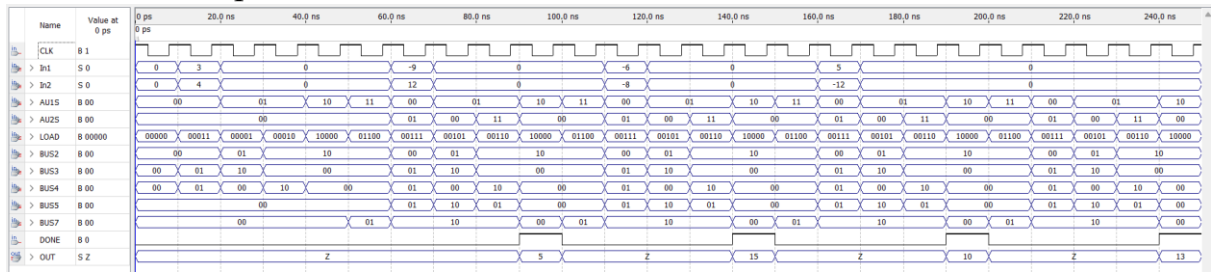


Cấu trúc tổng thể của datapath được thiết kế dựa trên cơ sở lý thuyết ở mục I.

Các input của datapath:

- In1[7..0], In2[7..0]: Input là các cặp [a;b] để tính toán
- AU1S[1..0]: Tín hiệu điều khiển khối AU1
- AU2S[1..0]: Tín hiệu điều khiển khối AU2
- LOAD[4..0]: Tín hiệu lựa chọn register nào sẽ ghi input mới
- BUS2[1..0]: Tín hiệu điều khiển tristate của bus 2
 - o BUS2[0]: Cho output của register 1 vào bus 2
 - o BUS2[1]: Cho output của register 2 vào bus 2
- BUS3[1..0]: Tín hiệu điều khiển tristate của bus 3
 - o BUS3[0]: Cho In1 vào bus 3
 - o BUS3[1]: Cho output của AU1 vào bus 3
- BUS4[1..0]: Tín hiệu điều khiển tristate của bus 4
 - o BUS4[0]: Cho In2 vào bus 4
 - o BUS4[1]: Cho output của AU1 vào bus 4
- BUS5[1..0]: Tín hiệu điều khiển tristate của bus 5
 - o BUS5[0]: Cho output của register 4 vào bus 5
 - o BUS5[1]: Cho output của register 5 vào bus 5
- BUS7[1..0]: Tín hiệu điều khiển tristate của bus 7
 - o BUS7[0]: Cho output của khối Shift right 3 vào bus 7
 - o BUS7[1]: Cho output của AU2 vào bus 7

- DONE: Tín hiệu cho phép xuất kết quả phép tính của datapath
 - CLK: Tín hiệu xung clock
- d. Waveform datapath



Waveform datapath được thiết kế dựa trên bảng timing sau:



Giải thích waveform:

- Từ 0ns – trước 18ns: datapath ở trạng thái nghỉ
- Tại 18ns: Có xung cạnh lên, datapath chuyển sang trạng thái nạp input [a;b] là [3;4]
- Từ 18ns – 58ns: Datapath thực hiện tính toán tầng 1, xử lý cặp input [3;4]
- Từ 60ns trở đi: Tầng 1 được xử lý xong và tiến hành xử lý tầng 2 của cặp [3;4], do đó tầng 1 datapath sẽ trống và có thể nhận cặp input mới
- Tại 68ns: Nạp tiếp theo 1 cặp input [a;b] mới là [-9;12] vào tầng 1 datapath xử lý
- Tại 90ns: Cặp đầu tiên đã xử lý xong và done = 1 để xuất kết quả là 5
- Từ 68ns – 108ns: Datapath thực hiện tính toán tầng 1, xử lý cặp input [-9;12]
- Từ 110ns trở đi: Tầng 1 được xử lý xong và tiến hành xử lý tầng 2 của cặp [-9;12], do đó tầng 1 datapath sẽ trống và có thể nhận cặp input mới

- Tại 118ns: Nạp tiếp theo 1 cặp input [a;b] mới là [-6;-8] vào tầng 1 datapath xử lý
- Và giải thích tương tự cặp [3;4] cho các cặp input sau đó

3. Thiết kế controller

Ta tiến hành thiết kế một máy trạng thái kiểu Moore để điều khiển datapath với các input là:

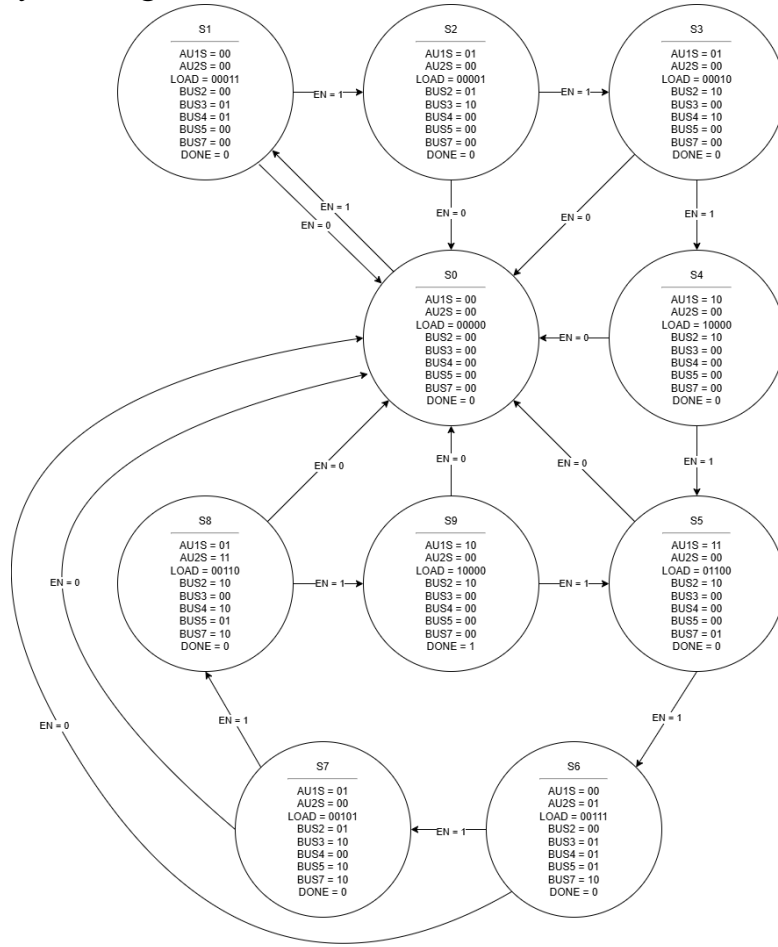
- EN: tín hiệu điều khiển hoạt động của datapath
- CLK: xung clock cho controller

Ta có tổng cộng 10 trạng thái do đó sẽ cần sử dụng 4 D flip flop

Bảng mô tả trạng thái của controller:

Tên trạng thái	Chức năng	Vị trí thực hiện trong datapath	Mã hóa trạng thái
S0	Trạng thái chờ (EN=0)	Cả 2 tầng	0000
S1	Nạp cặp input [a;b] và lưu vào R1, R2	Tầng 1	0001
S2	Đọc R1 và tính $ R1 $ sau đó lưu vào R1		0011
S3	Đọc R2 và tính $ R2 $ sau đó lưu vào R2		0010
S4	Đọc R1, R2, tính min rồi shift right 1 sau đó lưu vào R5		0110
S5	S5: Đọc R1, R2, tính max rồi lưu vào R4, shift right 3 sau đó lưu vào R3		0111
S6	Nạp cặp input [a;b] và lưu vào R1, R2	Tầng 1	0101
	Đọc R3, R4, tính $R4 - R3$ rồi lưu vào R3	Tầng 2	
S7	Đọc R1 và tính $ R1 $ sau đó lưu vào R1	Tầng 1	0100
	Đọc R3, R5, tính $R5 + R3$ rồi lưu vào R3	Tầng 2	
S8	Đọc R2 và tính $ R2 $ sau đó lưu vào R2	Tầng 1	1100
	Đọc R3, R4, tính $\max(R3, R4)$ rồi lưu vào R3	Tầng 2	
S9	Đọc R1, R2, tính min rồi shift right 1 sau đó lưu vào R5	Tầng 1	1101
	Done = 1 và output = R3	Tầng 2	

Sơ đồ chuyển trạng thái:



Số lượng trạng thái đã là tối thiểu và không thể rút gọn thêm

Biểu thức input cho các D-flipflop:

$$D0 = Q0 + EN(Q2Q1'Q0')$$

$$D1 = Q1 + EN(Q3'Q2 + Q3'Q1Q0' + Q3Q2Q1')$$

$$D2 = Q2 + EN(Q3'Q2'Q0 + Q3'Q1Q0' + Q3Q2Q1'Q0)$$

$$D3 = Q3 + EN(Q3'Q2'Q1' + Q3'Q2Q1 + Q3Q2Q1')$$

Biểu thức output của controller:

$$AU1S[1] = S4 + S5 + S9 = Q3'Q2Q1 + Q3Q2Q1'Q0$$

$$AU1S[0] = S2 + S3 + S5 + S7 + S8 = Q3'Q2'Q1 + Q3'Q1Q0 + Q2Q1'Q0'$$

$$AU2S[1] = S8 = Q3Q2Q1'Q0'$$

$$AU2S[0] = S6 + S8 = Q3'Q2Q1'Q0 + Q3Q2Q1'Q0'$$

$$LOAD[4] = S4 + S9 = Q3'Q2Q1Q0' + Q3Q2Q1'Q0$$

$$LOAD[3] = S5 = Q3'Q2Q1Q0$$

$$LOAD[2] = S5 + S6 + S7 + S8 = Q3'Q2Q0 + Q2Q1'Q0'$$

$LOAD[1] = S1 + S3 + S6 + S8 = Q3'Q1'Q0 + Q3'Q2'Q1Q0' + Q3Q2Q1'Q0'$

$LOAD[0] = S1 + S2 + S6 + S7 = Q3'Q2'Q0 + Q3'Q2Q1'$

$BUS2[1] = S3 + S4 + S5 + S8 + S9 = Q3'Q2Q1 + Q3Q2Q1' + Q3'Q2'Q1Q0'$

$BUS2[0] = S2 + S7 = Q3'Q2'Q1Q0 + Q3'Q2Q1'Q0'$

$BUS3[1] = S2 + S7 = Q3'Q2'Q1Q0 + Q3'Q2Q1'Q0'$

$BUS3[0] = S1 + S6 = Q3'Q1'Q0$

$BUS4[1] = S3 + S8 = Q3'Q2'Q1Q0' + Q3Q2Q1'Q0'$

$BUS4[0] = S1 + S6 = Q3'Q1'Q0$

$BUS5[1] = S7 = Q3'Q2Q1'Q0'$

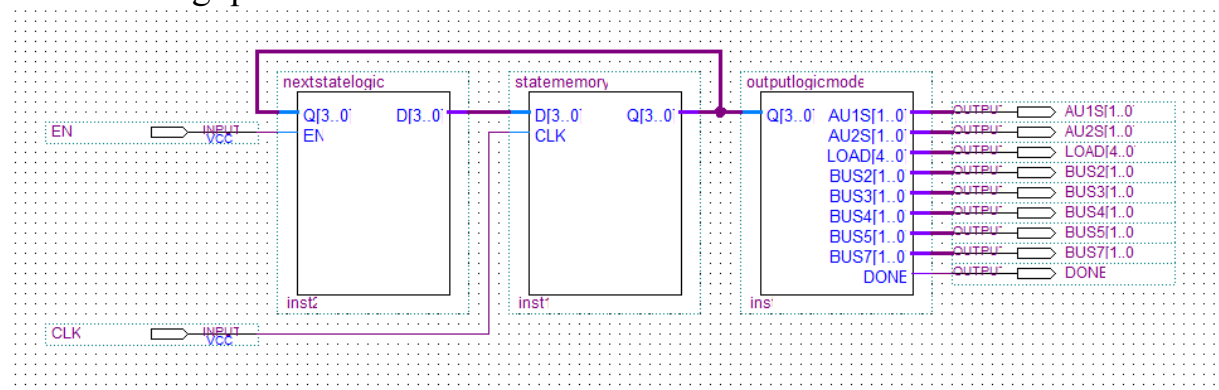
$BUS5[0] = S6 + S8 = Q3'Q2Q1'Q0 + Q3Q2Q1'Q0'$

$BUS7[1] = S6 + S7 + S8 = Q3'Q2Q1' + Q2Q1'Q0'$

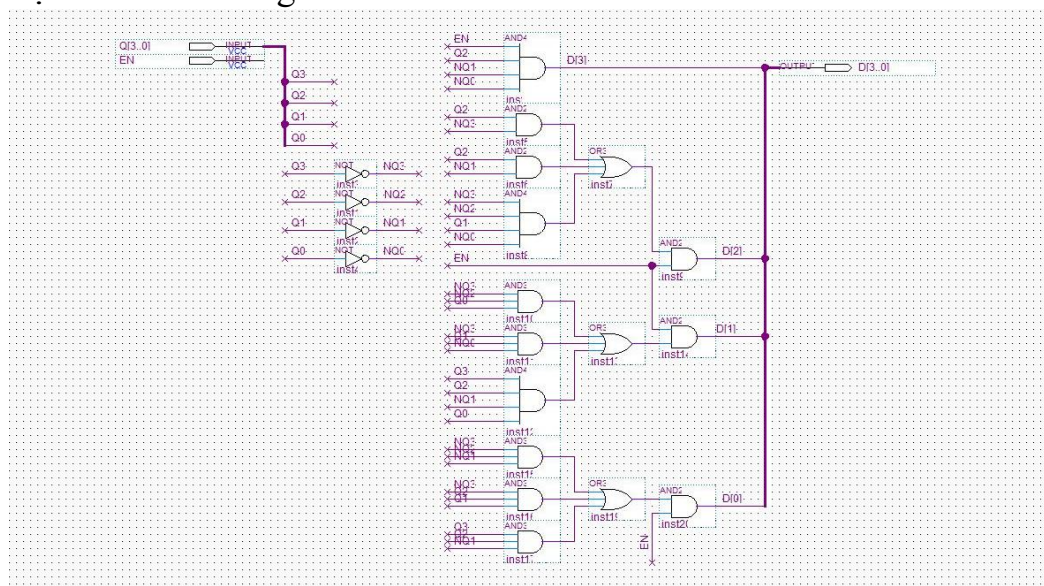
$BUS7[0] = S5 = Q3'Q2Q1Q0$

$DONE = S9 = Q3Q2Q1'Q0$

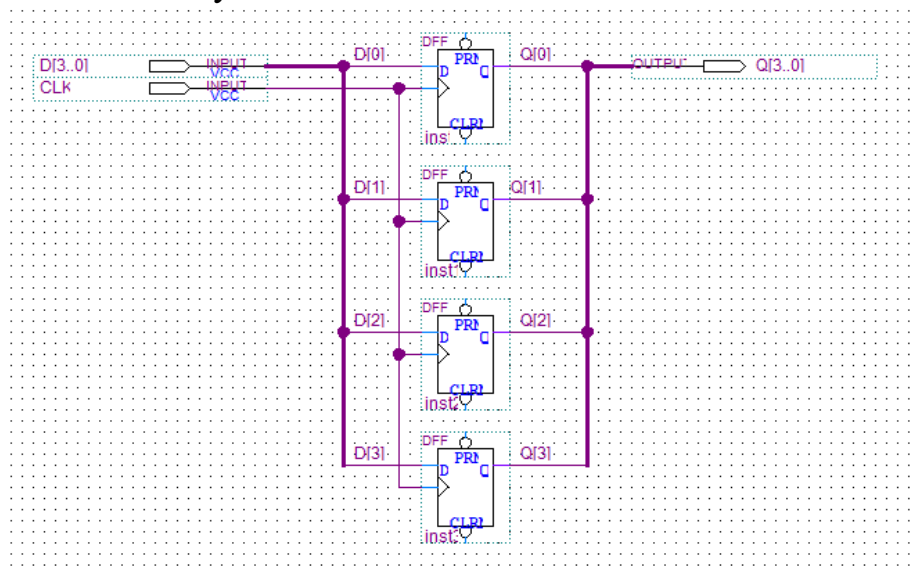
Thiết kế tổng quan controller:



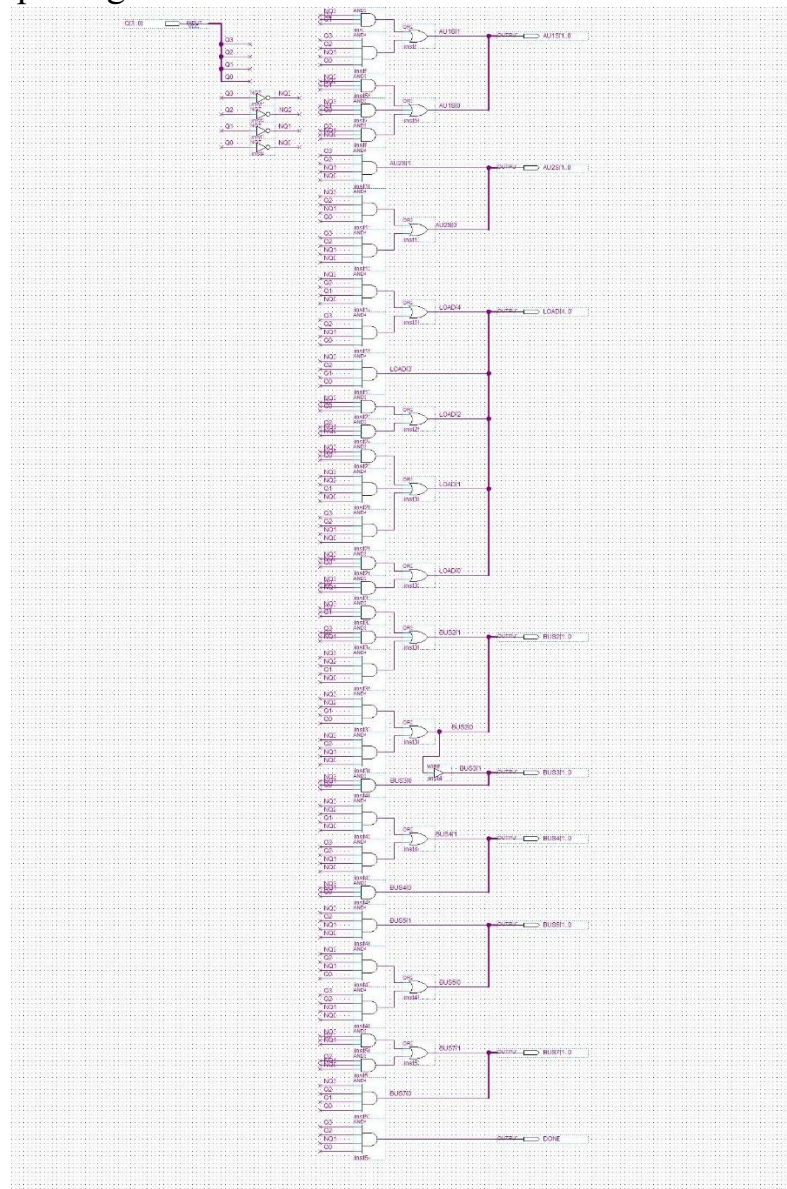
Mạch next-state logic:



Mạch state-memory:



Mạch output-logic:



- 208ns: Cặp input thứ ba [-6;-8] đã được xử lí xong và cho ra output là 10
- 188ns – 230ns: Mạch xử lí giai đoạn 1 của cặp input [5;-12]
- 238ns: Mạch nhận cặp input thứ năm là [5;6] và bắt đầu xử lí ở giai đoạn 1, đồng thời cặp input thứ tư tiến tới giai đoạn 2 xử lí
- 258ns: Cặp input thứ tư [5;-12] đã được xử lí xong và cho ra output là 13
- 238ns – 308ns: Cặp input thứ năm [5;6] đã được xử lí xong và cho ra output là 8
- 320ns: EN = 0, mạch ngưng hoạt động

O. Datapath pipeline với pipelined units

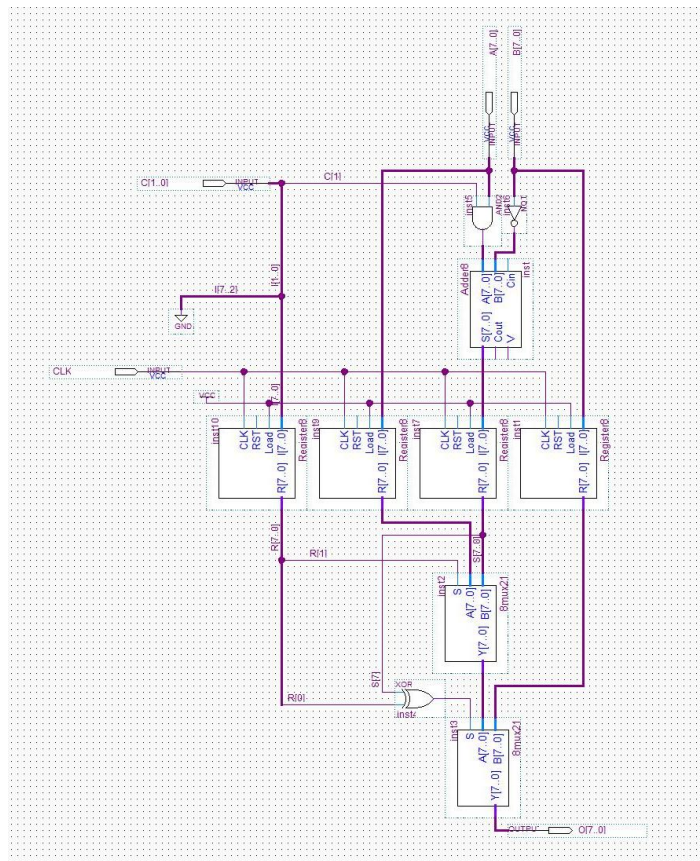
1. Cơ sở lý thuyết

Nhận ra ưu điểm của datapath pipelining chính là việc có thể nạp cặp input thứ $n + 1$ trong khi cặp input thứ n đang xử lý, và ưu điểm của pipelined functional units là có thể xử lý lệnh thứ $n + 1$ trong khi lệnh thứ n đang xử lý. Ta tiến hành thiết kế mạch datapath pipeline with pipelined units để kết hợp ưu điểm của hai mạch trên với nhau.

2. Thiết kế datapath

Các khối thanh ghi và shift được tái sử dụng từ mạch datapath với pipelined functional unit

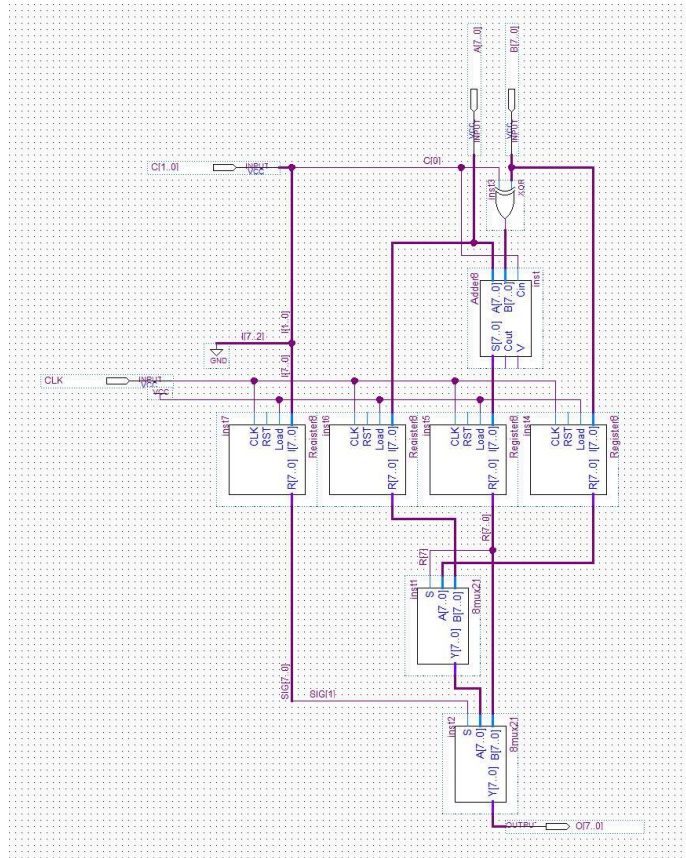
a. Khối AU1



Khối AU1 bao gồm 1 khối adder, 2 khối mux2to1 và được chia thành 2 giai đoạn xử lý nhờ đặt các register lưu giai đoạn ở giữa khối adder và các khối mux và thực hiện xử lý theo tín hiệu điều khiển sau:

- 01: Tính trị tuyệt đối B ($|B|$)
- 10: Tính $\min(A, B)$
- 11: Tính $\max(A, B)$

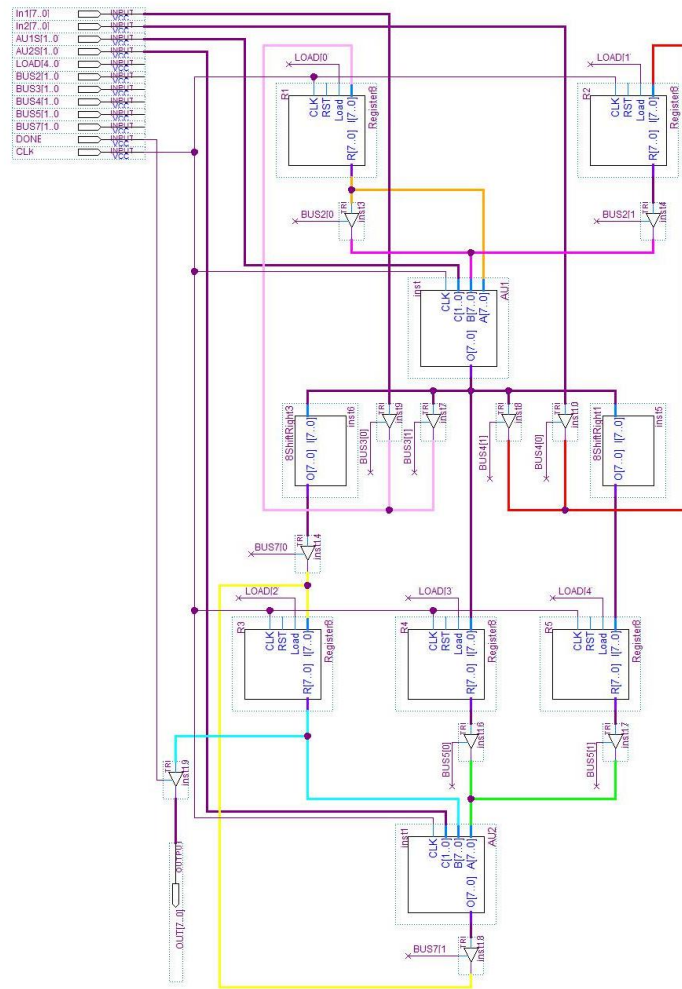
b. Khối AU2



Khối AU2 bao gồm 1 khối adder, 2 khối mux2to1 và được chia thành 2 giai đoạn xử lý nhờ đặt các register lưu giai đoạn ở giữa khối adder và các khối mux và thực hiện xử lý theo tín hiệu điều khiển sau:

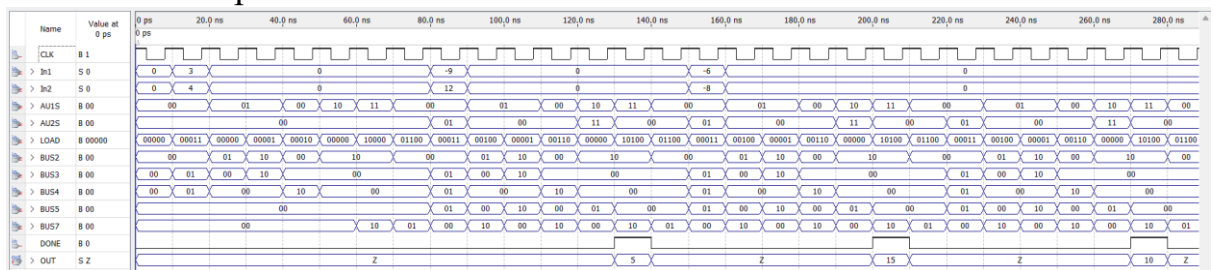
- 00: Tính phép cộng ($A+B$)
- 01: Tính phép trừ ($A-B$)
- 11: Tính $\max(A,B)$

c. Datapath hoàn chỉnh



Các input của datapath tương tự như bài tập datapath pipelining

d. Waveform datapath



Giải thích waveform datapath:

- 0ns – 10ns: Đây là trạng thái chờ, datapath chưa hoạt động
- 10ns – 20ns: Nạp input là cặp [3 ;4] và datapath bắt đầu quá trình tính toán
- 20ns – 80ns: Datapath thực hiện tính toán cặp [3;4] ở tầng 1
- 80ns – 90ns: Nạp tiếp tục 1 cặp input [-9;12], cặp [3;4] chuyển sang tính toán ở tầng 2

- 80ns – 138ns: Cặp input [3;4] được tính toán hoàn tất và bật tín hiệu done để xuất output là 5
- 90ns – 150ns: Datapath thực hiện tính toán cặp [-9;12] ở tầng 1
- 150ns – 160ns: Nạp tiếp tục 1 cặp input [-6;-8], cặp [-9;12] chuyển sang tính toán ở tầng 2
- 150ns – 208ns: Cặp input [-9;12] được tính toán hoàn tất và bật tín hiệu done để xuất output = 15
- 160ns – 278ns: Datapath thực hiện tính toán cặp [-6;-8] và bật tín hiệu done để xuất output = 12

*Các phần done = 0 thì output là Z

3. Controller

Ta tiến hành thiết kế một máy trạng thái kiểu Moore để điều khiển datapath với các input là:

- EN: tín hiệu điều khiển hoạt động của datapath
- CLK: xung clock cho controller

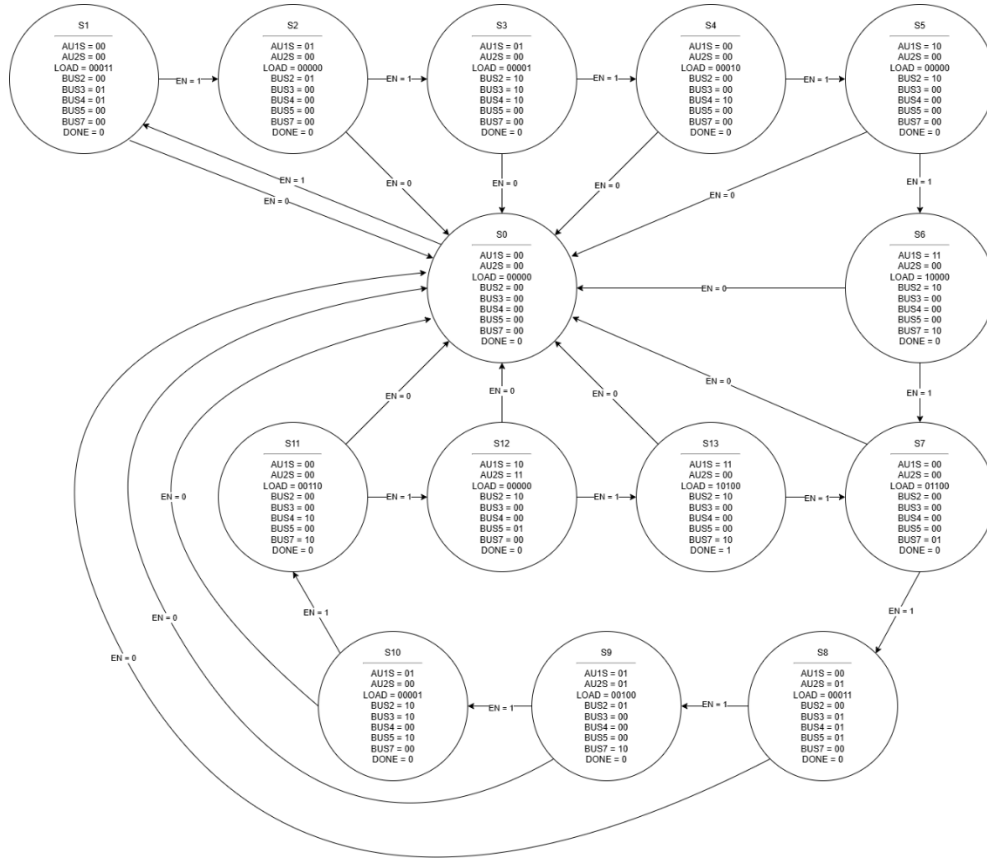
Ta có tổng cộng 14 trạng thái do đó sẽ cần sử dụng 4 D flip flop

Bảng mô tả trạng thái của controller:

Tên trạng thái	Chức năng	Vị trí thực hiện trong datapath	Mã hóa trạng thái
S0	Trạng thái chờ (EN=0)	Cả 2 tầng	0000
S1	Nạp cặp input [a;b] và lưu vào R1, R2	Tầng 1	0001
S2	Đọc R1 và tính R1 ở tầng 1 của AU1		0011
S3	Đọc R2 và tính R2 ở tầng 1 của AU1 và tính R1 ở tầng 2 sau đó ghi vào R1		0010
S4	tính R2 ở tầng 2 sau đó ghi vào R2		0110
S5	Đọc R1, R2, tính min(R1,R2) ở tầng 1 của AU1		0111
S6	Đọc R1, R2, tính max(R1,R2) ở tầng 1 của AU1, tính min(R1,R2) ở tầng 2 lấy kết quả shift right 1 và lưu vào R5		0101
S7	Tính max(R1,R2) ở tầng 2 lấy kết quả lưu vào R4, lấy kết quả shift right 3 lưu vào R3		0100

S8	Nạp cặp input [a;b] và lưu vào R1, R2	Tầng 1	1100
	Đọc R3, R4, tính $R4 - R3$ ở stage 1	Tầng 2	
S9	Đọc R1 và tính $ R1 $ ở tầng 1 của AU1	Tầng 1	1101
	tính $R4 - R3$ ở stage 1 lưu kết quả vào R3	Tầng 2	
S10	Đọc R2 và tính $ R2 $ ở tầng 1 của AU1 và tính $ R1 $ ở stage 2 sau đó ghi vào R1	Tầng 1	1111
	Đọc R3, R5, tính $R3 + R5$ ở stage 1	Tầng 2	
S11	Tính $ R2 $ ở tầng 2 sau đó ghi vào R2	Tầng 1	1110
	Tính $R3 + R5$ ở stage 2 và lưu kết quả vào R3	Tầng 2	
S12	Đọc R1, R2, tính $\min(R1, R2)$ ở tầng 1 của AU1	Tầng 1	1010
	Đọc R3, R4 và tính $\max(R3, R4)$ ở stage 1	Tầng 2	
S13	Đọc R1, R2, tính $\max(R1, R2)$ ở tầng 1 của AU1, tính $\min(R1, R2)$ ở tầng 2 lấy kết quả shift right 1 và lưu vào R5	Tầng 1	1011
	Tính $\max(R3, R4)$ ở stage 2 và lưu kết quả vào R3, $DONE = 1$, $output = R3$	Tầng 2	

Sơ đồ chuyển trạng thái:



Số lượng trạng thái đã là tối thiểu và không thể rút gọn thêm

Biểu thức input cho 4 D-flipflop:

$$D3 = Q3+ = EN(Q3Q2 + Q2Q1'Q0' + Q3Q1Q0')$$

$$D2 = Q2+ = EN(Q3'Q2 + Q3Q2Q1' + Q3Q1Q0 + Q3'Q1Q0')$$

$$D1 = Q1+ = EN(Q3'Q2'Q0 + Q3Q2Q0 + Q1Q0')$$

$$D0 = Q0+ = EN(Q3'Q2'Q1' + Q3'Q2Q1 + Q3Q2Q1' + Q3Q2'Q1Q0')$$

Biểu thức output của controller:

$$AU1S[1] = S5 + S6 + S12 + S13 = Q3'Q2Q0 + Q3Q2'Q1$$

$$AU1S[0] = S2 + S3 + S6 + S9 + S10 + S13 = Q3'Q2'Q1 + Q3Q1Q0 + Q2Q1'Q0$$

$$AU2S[1] = S12 = Q3Q2'Q1Q0'$$

$$AU2S[0] = S8 + S12 = Q3Q2Q1'Q0' + Q3Q2'Q1Q0'$$

$$LOAD[4] = S6 + S13 = Q3'Q2Q1'Q0 + Q3Q2'Q1Q0$$

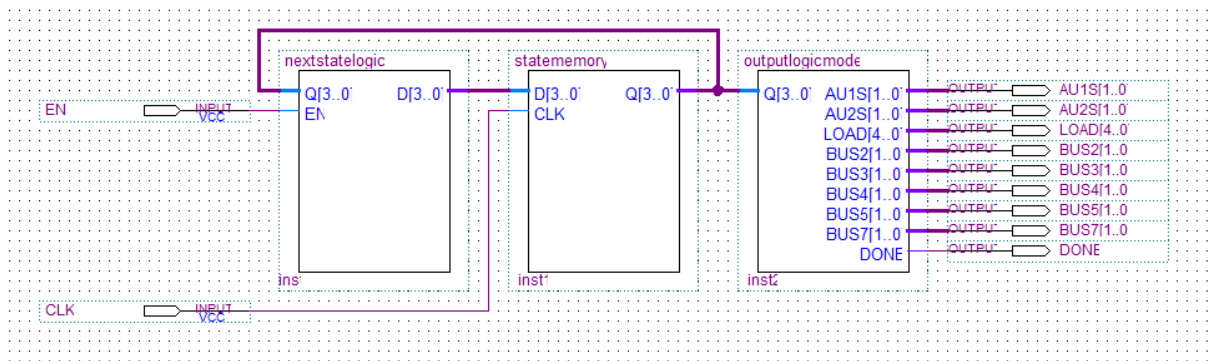
$$LOAD[3] = S7 = Q3'Q2Q1'Q0'$$

$$LOAD[2] = S7 + S9 + S11 + S13 = Q3'Q2Q1'Q0' + Q3Q2Q1'Q0 + Q3Q2Q1Q0' + Q3Q2'Q1Q0$$

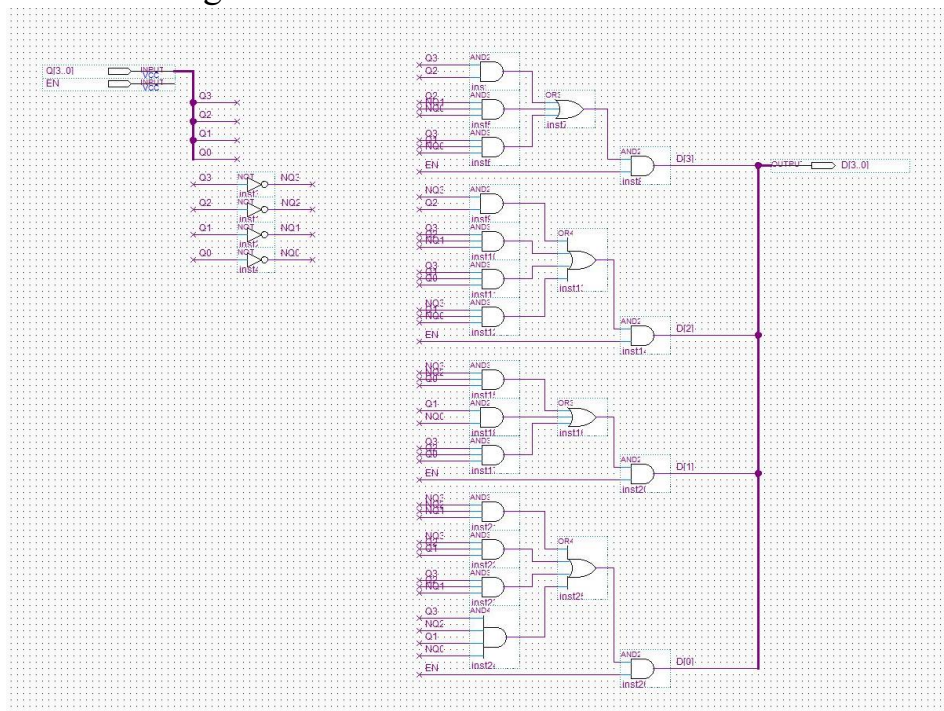
$$LOAD[1] = S1 + S4 + S8 + S11 = Q3Q2Q0' + Q3'Q2'Q1'Q0 + Q3'Q2Q1Q0'$$

$LOAD[0] = S1 + S3 + S8 + S10 = Q3'Q2'Q1'Q0 + Q3'Q2'Q1Q0' + Q3Q2Q1'Q0' + Q3Q2Q1Q0$
 $BUS2[1] = S3 + S5 + S6 + S10 + S12 + S13 = Q3'Q2Q0 + Q3Q1Q0 + Q2'Q1Q0'$
 $BUS2[0] = S2 + S9 = Q3'Q2'Q1Q0 + Q3Q2Q1'Q0'$
 $BUS3[1] = S10 + S3 = Q3Q2Q1Q0 + Q3'Q2'Q1Q0'$
 $BUS3[0] = S1 + S8 = Q3'Q2'Q1'Q0 + Q3Q2Q1'Q0'$
 $BUS4[1] = S4 + S11 = Q2Q1Q0'$
 $BUS4[0] = S1 + S8 = Q3'Q2'Q1'Q0 + Q3Q2Q1'Q0'$
 $BUS5[1] = S10 = Q3Q2Q1Q0$
 $BUS5[0] = S8 + S12 = Q3Q2Q1'Q0' + Q3Q2'Q1Q0'$
 $BUS7[1] = S6 + S9 + S11 + S13 = Q2Q1'Q0 + Q3Q2Q1Q0' + Q3Q2'Q1Q0$
 $BUS7[0] = S7 = Q3'Q2Q1'Q0'$
 $DONE = S13 = Q3Q2'Q1Q0$

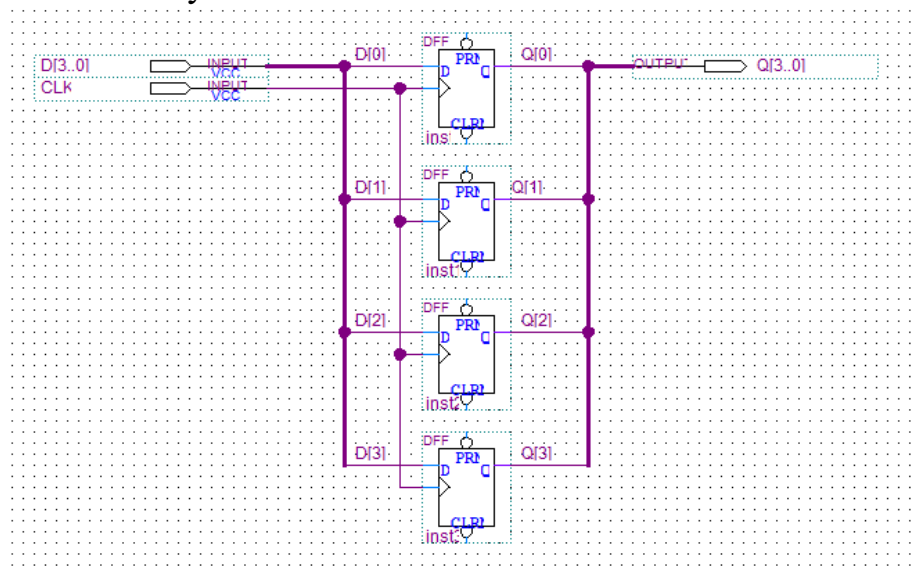
Thiết kế tổng quan controller:



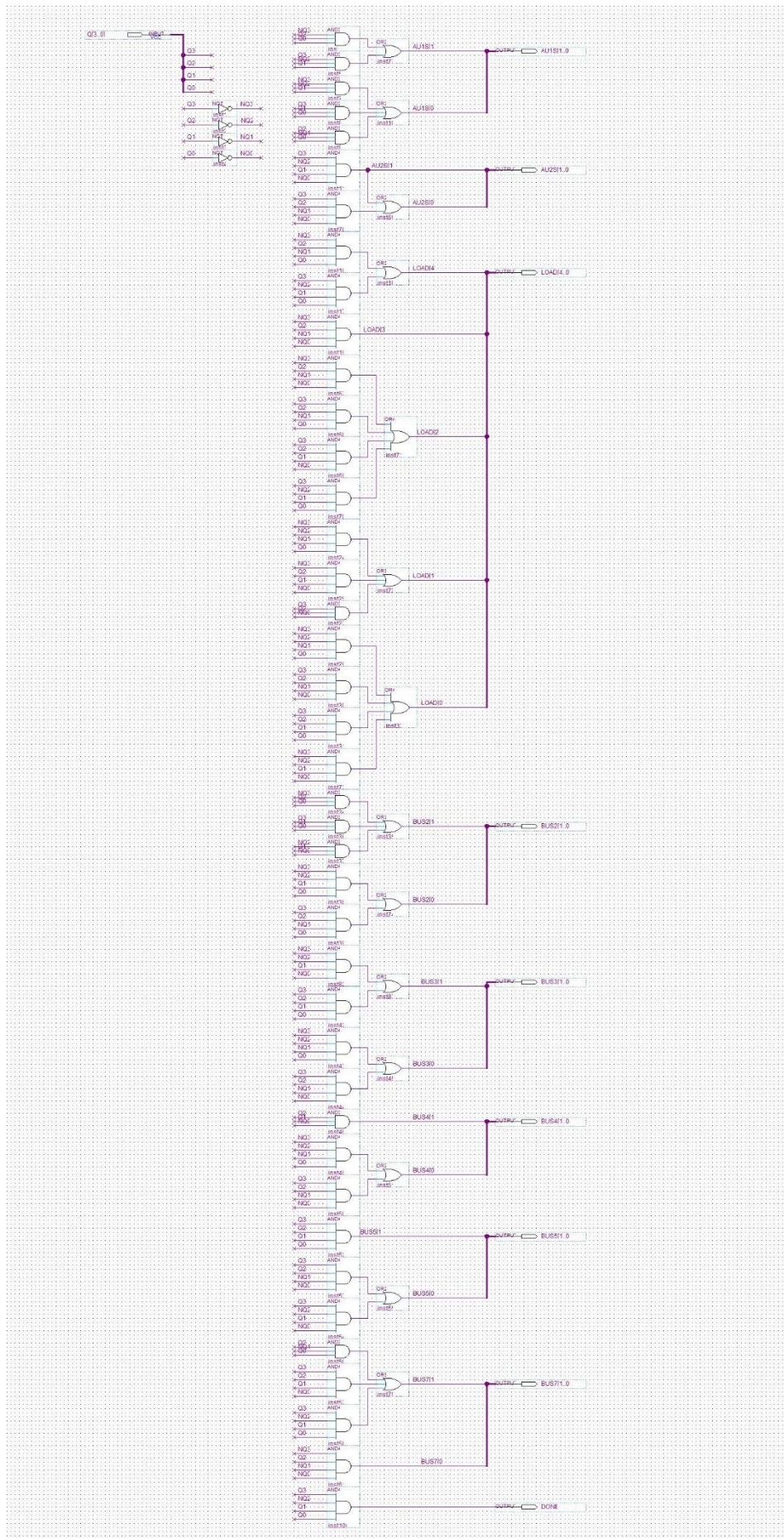
Mạch next-state logic:



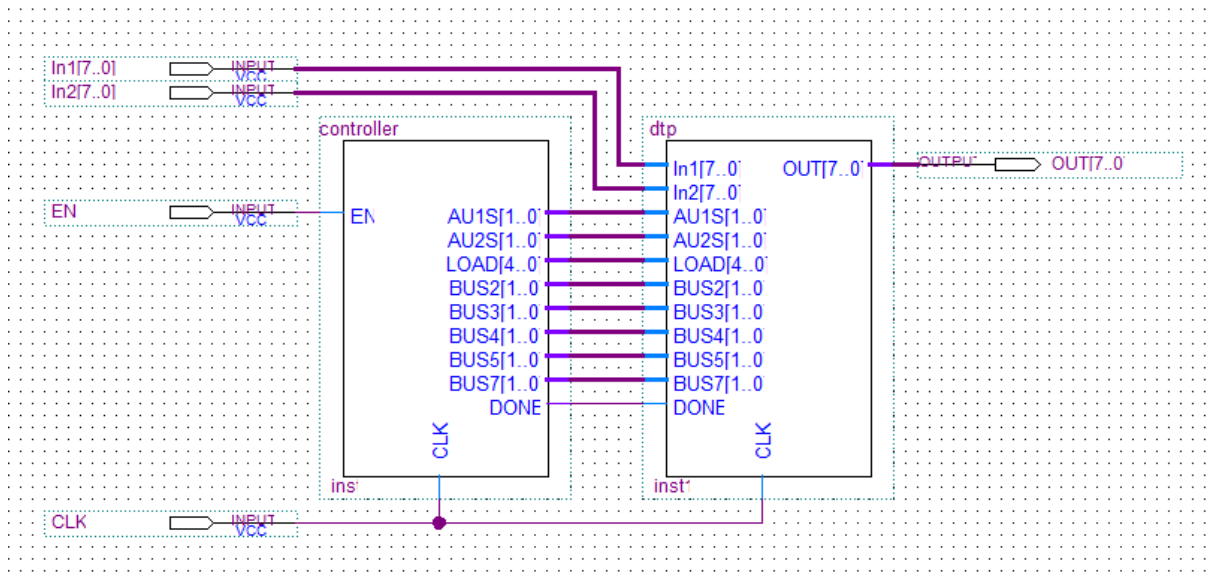
Mạch state memory:



Mạch output logic mode:

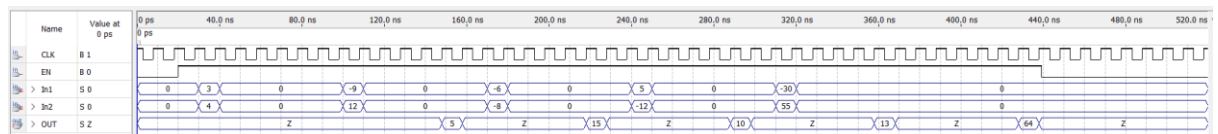


4. Mạch hoàn chỉnh



Ta tiến hành đóng gói controller và datapath sau đó kết nối chúng lại với nhau thành mạch hoàn chỉnh như hình.

5. Waveform



Giải thích waveform:

- 0ns – 20ns: EN = 0, mạch ở chế độ chờ và không hoạt động
- Tại 28ns: EN = 1, mạch bắt đầu hoạt động và nhận input
- 38ns: Mạch nhận cặp input [3;4] và bắt đầu tính toán
- 38ns – 148ns: Cặp [3;4] được tính toán và xuất kết quả là 5
- 108ns: Trong khi cặp [3;4] đang được tính toán ta tiến hành nạp thêm input là cặp [-9;12] và tiến hành bắt đầu tính toán cho cặp này
- 108ns – 218ns: Cặp [-9;12] được tính toán xong và xuất kết quả là 15
- 178ns: Trong khi cặp [-9;12] đang được tính toán ta tiến hành nạp thêm input là cặp [-6;-8] và tiến hành bắt đầu tính toán cho cặp này
- 178ns – 288ns: Cặp [-6;-8] được tính toán xong và xuất kết quả là 10
- 248ns: Trong khi cặp [-6;-8] đang được tính toán ta tiến hành nạp thêm input là cặp [5;-12] và tiến hành bắt đầu tính toán cho cặp này
- 248ns – 358ns: Cặp [5;-12] được tính toán xong và xuất kết quả là 13
- 318ns: Trong khi cặp [5;-12] đang được tính toán ta tiến hành nạp thêm input là cặp [-30;55] và tiến hành bắt đầu tính toán cho cặp này
- 428ns: Cặp [-30;55] được tính toán xong và xuất kết quả là 64

*Nhận xét: Các kết quả của các cặp input đã là chính xác